

Optimizing LLM Inference: Fluid-Based Online Scheduling under Memory Constraints

(Authors' names are not included for peer review)

Authors are encouraged to submit new papers to INFORMS journals by means of a style file template, which includes the journal title. However, use of a template does not certify that the paper has been accepted for publication in the named journal. INFORMS journal templates are for the exclusive purpose of submitting to an INFORMS journal and are not intended to be a true representation of the article's final published form. Use of this template to distribute papers in print or online or to submit papers to another non-INFORM publication is prohibited.

Abstract. Large language models (LLMs) now serve millions of users daily, with some providers incurring compute costs exceeding \$700,000 per day. Each request requires running inference, the process of generating a response token by token, and the scheduler at the server determines how many concurrent requests a given GPU can handle. We study online scheduling for LLM inference under an unusual memory constraint: each request's Key-Value (KV) cache, the intermediate attention state reused across output tokens, grows with every new token, so a request's memory consumption is nonstationary and uncertain at arrival. When memory fills up, the scheduler must evict in-progress requests and discard their prior computation. Even when capacity is theoretically sufficient for stability, a naive policy can trigger cascading evictions that collapse the system; the risk is especially acute when output lengths are unknown at arrival, because the scheduler cannot plan ahead. We formulate LLM inference as a multi-stage online scheduling problem and analyze a fluid approximation that characterizes the stability region and the equilibrium memory level required to sustain it. Guided by this benchmark, we design the WAIT (Waiting for Accumulated Inference Threshold) algorithm, a threshold-based admission rule that keeps the in-system state near the fluid equilibrium and prevents eviction cascades. When output lengths are known, WAIT achieves asymptotically optimal effective throughput, the completion rate net of eviction. For unknown output lengths, we extend the design to Nested WAIT, which uses nested thresholds across decode segments so that short prompts exit early while long prompts advance to later segments, with no output-length prediction required. A safety buffer that is logarithmic in the failure probability prevents memory overflow with high probability. Both algorithms are asymptotically optimal in the long-horizon regime. On Llama-2-7B with an NVIDIA A100 GPU, WAIT and Nested WAIT enlarge the stability region, raise effective throughput, and reduce mean end-to-end latency relative to vLLM and Sarathi, two widely used open-source LLM inference systems, across a wide range of arrival rates.

Key words: Large Language Model, Key-value cache, Memory Constraint, Online scheduling

1. Introduction

Large Language Models (LLMs) now dominate natural language processing (NLP) (Devlin et al. 2019, Brown et al. 2020, Kaplan et al. 2020, Ouyang et al. 2022, Wei et al. 2022a,b, Touvron et al. 2023, OpenAI 2024), powering chatbots (Anthropic 2023, Bai et al. 2023, OpenAI 2024, DeepSeek-AI 2025), search engines (Google 2023, Microsoft 2023), and programming assistants (GitHub 2022, Anthropic 2025). These models generate text through *inference*, which imposes significant computational demands on memory and processing resources (García-Martín et al.

2019). Systems like ChatGPT incur daily inference costs exceeding \$700,000 and contribute to substantial carbon emissions (Patel 2023, Patterson et al. 2021). Efficient inference scheduling can reduce these operational costs and energy consumption (Strubell et al. 2019, Desislavov et al. 2021) by balancing system throughput and response latency (Wu et al. 2022).

Figure 1 illustrates how a query is processed during LLM inference. The user submits a prompt (e.g., “Is apple a fruit?”), which then passes through two computational phases:

- *Prefill phase (stage 0)*. Upon receiving the prompt, the model first tokenizes the input into a sequence of discrete units (e.g., “Is”, “apple”, “a”, “fruit”, “?”). These tokens are then embedded and simultaneously processed in a single forward pass to compute the *Key-Value (KV) cache*, which stores intermediate representations (i.e., attention keys and values) for each token. These precomputed values enable efficient reuse during subsequent decoding steps. This phase corresponds to Stage 0 in Figure 1, where all prompt tokens are embedded and their KV representations are added to the cache.
- *Decode phase (stages 1 to l')*. After the prefill phase, the model enters the decode phase, where it generates the output one token at a time. At each stage, the model queries the existing KV cache to compute the next token, appends the new token to the output sequence, and updates the KV cache with its key-value pair. For instance, the model might first generate “Yes” (Stage 1), then “it” (Stage 2), followed by “is” (Stage 3), and finally the “End of Sequence” (EOS) token (Stage 4). This progression is depicted in the lower part of Figure 1. Each token generation step involves both reading from and writing to the KV cache, so the KV cache grows linearly with the length of the generated sequence.

Figure 1 also highlights key performance metrics in LLM inference:

- *Time-to-first-token (TTFT)* measures the latency from user input to the first generated token.
- *Latency* refers to the total time required to complete the generation of all output tokens.
- *Throughput* captures the average number of tokens generated per unit time.

This inference structure, particularly the growing memory requirements and sequential decoding pattern, introduces fundamental constraints on scheduling and batching. Efficient scheduling must account for both prompt heterogeneity and KV cache dynamics to balance latency and resource use.

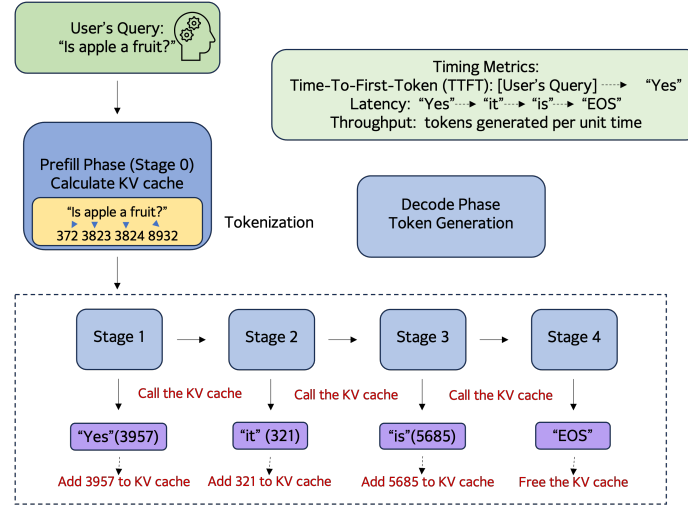


Figure 1 An example of LLM inference.

Scheduling LLM inference tasks involves grouping prompts into *batches* processed concurrently on a GPU. Batching improves throughput: processing multiple prompts together amortizes the fixed overhead of each iteration and better uses GPU parallelism. However, this benefit comes with a fundamental tension: larger batches consume more memory through their combined KV caches. The KV cache improves efficiency, as it prevents the model from recalculating attention history for each new token; without it, computational cost would scale quadratically with sequence length. But the KV cache also grows dynamically during decode, making memory consumption unpredictable. This creates a core trade-off: *larger batches increase throughput but consume more memory, risking capacity overflow*.

Memory overflow can occur in several practical scenarios: (i) *bursty arrivals*, where sudden traffic spikes temporarily exceed capacity; (ii) *unknown output lengths*, where prompts generate longer responses than anticipated; and (iii) *multi-tenant environments*, where multiple users compete for GPU memory. The unknown output length problem is particularly challenging: since the scheduler cannot predict how much memory each prompt will eventually consume, it cannot reliably plan batch composition or admission decisions. Traditional scheduling methods such as Shortest Job First (SJF) assume fixed job sizes and known processing times, making them unsuitable when memory demands grow unpredictably (Tay et al. 2022, Kang et al. 2024, Hooper et al. 2025).

When the total KV cache exceeds GPU memory capacity, the system must evict some in-progress prompts. Two approaches exist: (i) *swapping* to CPU or SSD, which incurs I/O overhead (Sheng et al. 2023, Aminabadi et al. 2022); and (ii) *recomputation*, which discards the KV cache and restarts

the prompt from prefill (Kwon et al. 2023). We focus on recomputation because it is the default in modern production systems such as vLLM (vLLM Team 2025), and because our goal is to *minimize eviction* through scheduling rather than to optimize the eviction mechanism itself (Li et al. 2025a). Eviction creates a vicious cycle: it frees memory temporarily, but restarted prompts consume the memory again and can trigger further evictions. Users experience this as error messages (e.g., “Something went wrong”) or incomplete responses under heavy load.

Memory sufficiency alone does not guarantee stability. The key to maximizing throughput is *load balance*, the equilibrium state at which arrival and completion rates match. Even when total capacity C exceeds the optimal steady-state memory M^* (derived in our fluid analysis), naive policies can push the system away from equilibrium. For instance, first-come-first-served (FCFS) can trigger cascading evictions that reduce effective throughput, the completion rate net of evictions, by as much as 20% when capacity is theoretically sufficient (Example 2). A slight imbalance causes one prompt type to overflow, triggering evictions; the freed memory then fills with another type, which itself overflows, creating a vicious cycle. Thus, a system that *should* be stable becomes unstable under poor scheduling.

Preventing eviction requires *approaching load balance through controlled admission*. Effective scheduling demands not just sufficient capacity but keeping the system near equilibrium by controlling when prompts advance to the next decode stage. This “approaching load balance” principle guides our algorithm design: threshold-based mechanisms hold prompts at decode-stage boundaries until memory conditions permit safe continuation, keeping the system close to the fluid equilibrium. The principle yields two benefits at once. Operating near the fluid equilibrium both *enlarges the stability region*, because eviction cascades no longer destabilize a system whose capacity is theoretically sufficient, and *reduces mean end-to-end latency across the underloaded, near-capacity, and overloaded regimes*. In the underloaded regime, our algorithms and the baselines are all stable, yet tighter load balance still produces lower mean latency; near the stability boundary, baseline latency diverges while ours remains bounded; under sustained overload, our algorithms sustain effective throughput closer to the arrival rate. Section 6 documents all three effects.

Recent system-level optimizations for LLM inference (Yu et al. 2022, Kwon et al. 2023, Agrawal et al. 2023, Pope et al. 2023, DeepSeek-AI 2024, Patel et al. 2024, Zhong et al. 2024) focus on engineering solutions but lack a rigorous mathematical foundation. In practice, output length

predictions are imprecise or costly (Fu et al. 2025), and algorithms designed for known output lengths can degenerate significantly when this assumption fails (see Proposition 5).

We develop a fluid approximation that identifies the equilibrium state, at which prompt arrivals and completions balance, and characterizes the steady-state memory level M^* at which the fluid system sustains the largest arrival rate. This equilibrium serves as a design target: when the stochastic system operates near it, the system sustains high effective throughput while avoiding eviction. The analysis shows that memory is a resource to exploit rather than merely a constraint, because larger batches process more prompts per iteration and increase GPU utilization.

Guided by these fluid insights, we design threshold-based algorithms that keep the stochastic system close to the fluid equilibrium. The WAIT algorithm demonstrates this for known output lengths: threshold-based admission control at each decode stage prevents the system from drifting away from balance. For unknown output lengths, we introduce Nested WAIT, which classifies prompts *on the fly*: short prompts complete early and exit, while longer prompts advance to later segments, all without requiring output length prediction. Both algorithms achieve asymptotic optimality by approaching the fluid equilibrium.

1.1. Summary of Contributions

Our main contributions are as follows.

- *Memory-constrained scheduling model.* We develop a multi-stage online scheduling model that captures the dynamic growth of KV cache memory during LLM inference, where exceeding capacity triggers costly prompt evictions (Section 2). Because the system is open, throughput equals the arrival rate whenever the policy stabilizes the system. The design goal is therefore twofold: to enlarge the stability region, and to operate near the equilibrium memory level M^* required to sustain its boundary. A fluid approximation from queueing theory lets us characterize both (Section 3).
- *Eviction-prevention scheduling algorithms.* We develop threshold-based algorithms that prevent memory overflow and minimize eviction. The WAIT algorithm (Section 4) handles known output lengths; the Nested WAIT algorithm (Section 5) extends to unknown output lengths through on-the-fly type classification. A threshold at each decode-stage boundary performs a *dimensionality reduction*: it decouples prompt types and replaces the continuously growing KV cache dynamics with a discrete-time per-type recursion. Both algorithms are asymptotically optimal.

- *Experimental validation.* We evaluate our algorithms on synthetic and real-world workloads using Llama-2-7B on a single A100 GPU (Section 6). We compare against vLLM (Kwon et al. 2023) and Sarathi (Agrawal et al. 2023), two widely used open-source LLM inference servers that represent, respectively, memory-reactive preemptive recomputation and chunked-prefill scheduling. Consistent with the load-balancing principle, WAIT and Nested WAIT *enlarge the stability region* relative to both baselines and *reduce mean end-to-end latency across the underloaded, near-capacity, and overloaded regimes*; near the stability boundary, baseline latency diverges while ours remains bounded.

Our batching theory also extends naturally to Prefill-Decode Disaggregated (PD) systems such as DistServe (Zhong et al. 2024) and Splitwise (Patel et al. 2024), in which the decode server handles decode-only workloads: the linear iteration-time model in Section 2 then holds exactly, the same thresholds apply, and Appendix G.4 reports numerical results that confirm the extension on representative PD workloads.

1.2. Other Related Work

Online Scheduling Problem and Queueing System. Classical online scheduling problems focus on optimally assigning jobs that arrive sequentially, each with varying completion times. In settings with stochastic arrivals, foundational studies such as Devanur and Hayes (2009), Vee et al. (2010), Cole and Roughgarden (2014), Balkanski et al. (2016), Lattanzi et al. (2020) have developed algorithms that learn arrival patterns or distributions to refine scheduling strategies over time. Beyond individual job processing, works like Im and Moseley (2013), Lucier et al. (2013), Liu and Lu (2015), Li et al. (2020) have explored simultaneous batching and scheduling, grouping similar jobs to enhance efficiency. Within queueing theory, fluid models serve as first-order approximations of stochastic systems, enabling near-optimal control strategies Mandelbaum et al. (1998), Maglaras (2000), Bäuerle (2002), Liu and Whitt (2011), while asymptotic analysis characterizes long-horizon behavior. However, these traditional approaches do not account for key features of LLM inference that complicate asymptotic analysis: (i) iteration time varies with batch composition and current KV cache sizes, precluding fixed service time assumptions; (ii) memory consumption grows dynamically during decode as new tokens are generated; (iii) out-of-memory (OOM) events trigger prompt eviction, creating feedback loops that classical stability and limit theorems cannot accommodate, as those theorems assume jobs complete once started, whereas eviction invalidates standard drift arguments.

LLM Inference. Theoretical frameworks for analyzing Large Language Model (LLM) inference remain scarce despite the scale of production deployment. Unlike classical scheduling, LLM inference combines stochastic arrivals, dynamic memory constraints, and multi-phase prefill/decode processing. System-level works such as Splitwise (Patel 2023) and DistServe (Zhong et al. 2024) split inference into separate prefill and decode pipelines, while scheduling engines such as Orca (Yu et al. 2022) and Sarathi (Agrawal et al. 2023, 2024b) improve throughput through batching and admission control. Other system-level optimizations include multi-head latent attention with low-rank KV compression (DeepSeek-AI 2024) and Kendall's-Tau prioritization by predicted output length (Fu et al. 2025). These works advance system-level efficiency through implementation and engineering. We complement them from the theoretical side, analyzing the underlying scheduling problem to characterize the stability region and the throughput gap induced by memory-growth dynamics. From an operations research perspective, several recent works analyze LLM inference scheduling under online-algorithm frameworks, focusing on competitive ratio and regret bounds. Jaillet et al. (2025) develop an online scheduling algorithm with near-optimal regret guarantees for settings with known output lengths. Wang et al. (2025) study optimization with variable prefill and decode lengths in pipeline-parallel settings. Chen et al. (2025) address robust optimization under prediction uncertainty, considering adversarial scenarios. A different theoretical angle is taken by Li et al. (2025b), whose stochastic processing model allows general batch processing times, including a piecewise-linear form that captures the compute-bound / memory-bound transition, and shows that work-conserving algorithms achieve the fluid-optimal throughput. On output length uncertainty, Jaillet et al. (2025) and Wang et al. (2025) assume lengths are known at arrival, Chen et al. (2025) considers adversarial unknown lengths, while our Nested WAIT algorithm handles stochastic unknown lengths through on-the-fly classification at segment boundaries. These works establish competitive-ratio or regret guarantees relative to hindsight benchmarks. Our work takes a different angle: a fluid-based analysis that characterizes the stability region through a memory-dependent iteration-time model (Equation (1)), showing that threshold-based admission *enlarges* this region by preventing eviction cascades.

1.3. Notations

For integer $n \geq 1$, we denote $[n] = \{1, 2, \dots, n\}$ as the set of integers from 1 to n . For $x \in \mathbb{R}$, denote $\lceil x \rceil$ as the smallest integer not smaller than x and $\lfloor x \rfloor$ as the largest integer not greater than x . Denote $x_+ = \max\{x, 0\}$. For set S , denote $|S|$ as its cardinality. For two functions $f(T)$ and $g(T)$,

we use $f(T) = O(g(T))$ if there exists constant $c_1 > 0$ such that $f(T) \leq c_1 g(T)$ as $T \rightarrow +\infty$ and $f(T) = \Omega(g(T))$ if there exists constant $c_2 > 0$ such that $f(T) \geq c_2 g(T)$ as $T \rightarrow +\infty$.

2. Model

We formalize the online scheduling problem for Large Language Model (LLM) inference under memory constraints. Unlike traditional scheduling where job sizes are fixed, LLM inference involves *dynamically growing memory demands*: the Key-Value (KV) cache expands continuously as prompts generate output tokens, and the two-phase structure (prefill and decode) creates fundamentally different resource consumption patterns. This dynamic growth, combined with unknown output lengths, distinguishes our problem from classical queueing models. We first introduce the inference process, then describe prompt characteristics, batching rules, system constraints, and performance metrics.

2.1. Notation

We summarize the key notation used throughout this paper in Table 1.

Table 1 Summary of Notation

Symbol	Description
m	Number of prompt types
j	Index of prompt type, $j \in \{1, \dots, m\}$
λ_j	Arrival rate of type- j prompts (Poisson process)
l_j	Input length (tokens) of type- j prompts after prefill
l'_j	Output length (tokens) of type- j prompts in decode phase
s	Stage index: $s = 0$ for prefill, $s \in \{1, \dots, l'_j\}$ for decode
k	Segment index in Nested WAIT, $k \in \{1, \dots, m\}$
b	Batch index (iteration count)
n_{js}^t	Number of type- j prompts at stage s at time t
n_j	Threshold for type- j prompts in WAIT algorithm
n_k	Threshold for segment k in Nested WAIT
C	GPU memory capacity
$\mathcal{G}^t$	Set of prompts with KV cache on GPU at time t
B^t	Batch of prompts processed at iteration t ($B^t \subseteq \mathcal{G}^t$)
τ	Iteration time to process a batch
d_0	Fixed overhead time per batch iteration
d_1	Time cost per unit of KV cache memory
M^*	Equilibrium memory usage in fluid model
n_j^*	Equilibrium number of active type- j prompts
Throughput*	Equilibrium throughput (fluid benchmark)
Π	Class of admissible scheduling policies

2.2. Prompts and Inference Process

User queries, called *prompts*, arrive at a single Graphics Processing Unit (GPU) for processing. We focus on maximizing how fully the GPU is used, and our algorithms extend to multi-GPU settings such as pipeline parallelism, where each prompt is processed across different GPUs in different layers of the LLM. When communication costs are negligible, our algorithms apply directly to these settings.

During the inference process, each prompt is divided into discrete units known as *tokens* during the prefill phase and generates output text through an inference process in the decode phase. A token represents a distinct text element, such as a word or subword, and its processing relies on a memory structure called the Key-Value (KV) cache. The KV cache improves inference efficiency by storing the computational history of prior tokens, thus avoiding redundant computations. The cache size increases as tokens are processed, requiring careful memory management to maintain system performance.

Prompt Characteristics: Prompts arrive stochastically and are classified into m distinct types according to their input / output lengths, indexed by $j \in \{1, \dots, m\}$. We assume that prompts of type j arrive according to a Poisson process with rate λ_j . Each prompt's memory consumption depends on its processing stage. A prompt of type j has the following properties:

$$\begin{cases} \text{a length of } l_j \text{ tokens after prefill;} \\ \text{a length of } l_j + l'_j \text{ tokens after decode.} \end{cases}$$

Therefore, a type- j prompt must undergo one prefill iteration followed by l'_j decode iterations to complete its inference processing. During the prefill phase, the prompt's input context and question are embedded as l_j tokens in a single iteration, consuming a KV cache of size l_j units. In the subsequent decode phase, one output token is generated per iteration, each incurring an additional unit of KV cache memory.

To track processing progress, we define a prompt to be in **stage** $s = 0$ if it is about to undergo the prefill phase, and in **stage** s (where $s \in \{1, \dots, l'_j\}$) if it is scheduled to receive its s -th output token during the decode phase. Thus, a prompt at stage s occupies memory of size $l_j + s$ units.

Though the total number of different types m can be large (for example, from 1 to 10000), our algorithms and their theoretical guarantees exhibit only a weak (logarithmic) dependence on m .

The analysis relies primarily on the total arrival rate across all types, ensuring that our approach scales efficiently regardless of the number of types.

While our analysis focuses on fixed arrival rates λ_j , our threshold-based algorithms can be extended to settings with time-varying arrivals by dynamically adjusting thresholds based on the arrival rates (see Section 7).

2.3. Batching and Iteration Time

To use GPU resources efficiently, the GPU processes tasks in *batches*, combining prompts for prefilling and prefilled prompts for decoding. Figure 2 illustrates the temporal dynamics of batching and scheduling in GPU-based inference. Two prompts, $P1$ and $P2$, arrive sequentially and are processed according to their current stage—prefill or decode—at each time step. Initially, $P1$ arrives and enters the prefill phase, embedding its input sequence “How are you?” in a single iteration. Shortly after, $P2$ arrives with the query “Is 5 a prime?”, which is also scheduled for prefill. The GPU batches prompts in the same phase together; thus, $P1$ is processed for prefill alone, followed by $P2$. At the next step, $P1$ enters the decode phase, generating its first output token “I’m,” while $P2$ undergoes prefill in parallel. Subsequently, both prompts advance in the decode phase, each generating one token per iteration: “Yes” for $P2$ and “I’m fine” for $P1$. This example highlights how the scheduler interleaves prompts in different stages—prefill and decode—into shared GPU batches, thereby improving throughput. Prefill stages involve initializing KV caches based on input tokens, while decode stages extend outputs one token at a time, with incremental KV cache updates.

In general, consider a batch composed of:

- n_1 prompts in the prefill phase (stage $s = 0$), where prompt i has input length $l_{j(i)}$ tokens corresponding to its type $j(i)$. These prompts are processed to initialize their KV caches.
- n_2 prompts in the decode phase, where prompt i is at stage $s_i \in \{1, \dots, l'_{j(i)}\}$, having generated s_i output tokens. The total KV cache size for prompt i is $l_{j(i)} + s_i$ tokens.

Experiments in Agrawal et al. (2023), Kwon et al. (2023), Zhong et al. (2024) show that *iteration time* scales linearly with total KV cache size. We model this as:

$$\tau = d_0 + d_1 \cdot \underbrace{\left(\sum_{i=1}^{n_1} l_{j(i)} + \sum_{i'=1}^{n_2} (l_{j(i')} + s_{i'}) \right)}_{\text{Total KV cache size in batch}}, \quad (1)$$

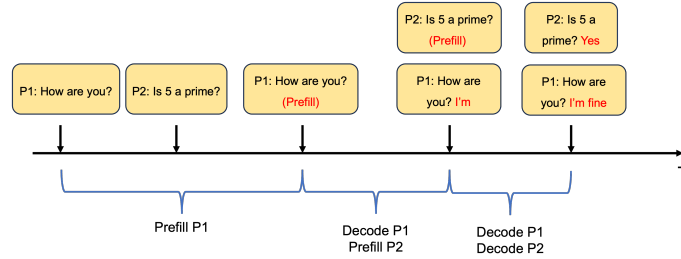


Figure 2 Example of batching and scheduling.

where d_0 is the fixed per-batch overhead and d_1 is the marginal time cost per unit of KV cache memory. This form makes the throughput trade-off explicit: larger batches amortize d_0 over more prompts, but the resulting increase in KV cache load pushes the system toward the memory constraint (2) and raises eviction risk. Figure 3 validates this linear approximation on a single L20 GPU using Llama-2-7B and Llama-2-13B. For the A100-based experiments in Section 6, Appendix G.2 reports a separate calibration of the same model against direct A100 measurements, yielding $d_0 \approx 276$ ms and $d_1 \approx 0.0115$ ms per KV cache token over the batch-size range used there.

The linear model (1) is most accurate in decode-dominated regimes, where each iteration processes one token per prompt and the computational cost is dominated by KV cache memory access. It is exact under prefill-decode disaggregation (Zhong et al. 2024, Patel et al. 2024), where prefill and decode are processed on different devices and the decode-side server handles only decode operations. In mixed prefill-decode batches, it remains a good approximation when prefill lengths are similar across prompts.

More generally, iteration time can be piecewise linear across operating regimes. For example, Li et al. (2025b) models it as $\tau(b') = c + a \cdot \max\{0, b' - b_0\}$, so iteration time is constant for small batches and grows linearly once the batch size exceeds a threshold b_0 . This form captures the transition from compute-bound to memory-bound operation. Our algorithms and analysis extend naturally to such models: the threshold mechanism in WAIT (Section 4) and Nested WAIT (Section 5) still controls the number of prompts in each decode stage, and the design principle of balancing arrivals and completions to approach the fluid equilibrium remains unchanged as long as iteration time scales predictably with batch composition. We adopt the simplified linear model (1) for tractability, and Figure 3 shows that it is accurate in the decode-dominated regime where our algorithms operate.

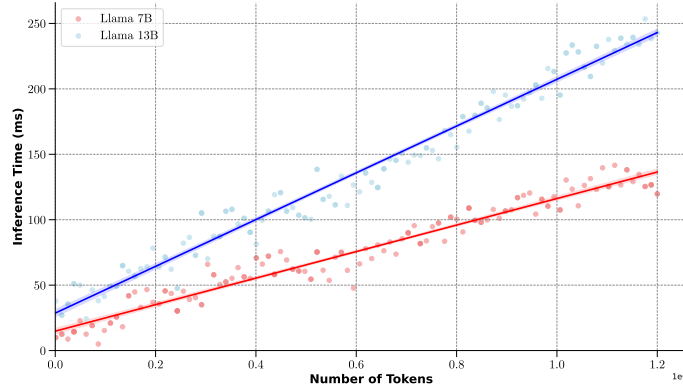


Figure 3 Inference time of Llama-2-7B and Llama-2-13B with varying number of tokens per batch on a single L20 GPU.

2.4. Memory Constraint and Challenges

Unlike traditional scheduling where job sizes are fixed, LLM inference faces a unique challenge: prompts occupy memory that **grows during decode**. Each generated token adds one unit to the prompt's KV cache, causing its memory consumption to grow continuously until completion. This dynamic growth creates eviction risks even when total capacity appears sufficient.

Memory Constraint: At all times t , the total KV cache stored on GPU must not exceed capacity C :

$$\sum_{i \in \mathcal{G}^t} (l_{j(i)} + s_i^t) \leq C, \quad (2)$$

where $\mathcal{G}^t$ denotes the set of prompts whose KV caches are stored on GPU at time t . This includes both prompts actively processed in the current batch $B^t \subseteq \mathcal{G}^t$ and prompts that are preempted but retain their KV caches on GPU. Here $j(i)$ is the type of prompt i , and $s_i^t \in \{0, 1, \dots, l'_{j(i)}\}$ is its current stage. Unlike traditional scheduling where job sizes are fixed, the per-prompt memory consumption $l_{j(i)} + s_i^t$ grows *dynamically* as decode progresses (s_i^t increases each iteration). This dynamic growth creates the risk of *eviction*: if memory becomes insufficient to satisfy the constraint (2), ongoing prompts must be removed from the GPU and their KV caches discarded. When eviction is necessary, prompts are removed in **LIFO order** (most recently admitted prompts are evicted first) until sufficient memory is freed. Evicted prompts lose all accumulated computation and must restart from the prefill phase (stage $s = 0$), re-entering the scheduling queue as new arrivals. This creates a vicious cycle: eviction temporarily frees memory, but restarted prompts compete for resources again and can trigger further evictions, a phenomenon we call *cascading failure*. The following example illustrates this mechanism.

EXAMPLE 1 (DYNAMIC MEMORY GROWTH AND EVICTION). Consider a simple prompt type: users submit “Hello” queries, each expecting a 1-token response “Hi.” Each prompt has prefill length $l = 1$ token and decode length $l' = 1$ token. Memory consumption evolves as follows:

- **Stage $s = 0$ (awaiting prefill):** No memory allocated yet.
- **After prefill:** The prompt’s KV cache occupies $l = 1$ unit of memory.
- **After decode (stage $s = 1$):** The prompt occupies $l + s = 1 + 1 = 2$ units before completion.

Suppose the GPU has capacity $C = 12$ units. A batch processes 4 prompts at prefill (using $4 \times 1 = 4$ units) and 4 prompts at decode (using $4 \times 2 = 8$ units), totaling $4 + 8 = 12$ units with 4 completions per iteration.

Now consider a scenario where 6 new prompts arrive simultaneously while 3 prompts are already decoding:

1. **Current state:** 3 prompts in decode occupy $3 \times 2 = 6$ units.
2. **Admit all 6 for prefill.** After prefill, these 6 prompts advance to decode. The 3 existing decode prompts complete. The system now has 6 prompts in decode, occupying $6 \times 2 = 12$ units, so memory is full.
3. **Next batch:** 4 new prompts arrive. To prefill them (needing $4 \times 1 = 4$ units), we must **evict** 2 decode prompts to free space.
4. **Evicted prompts restart:** The 2 evicted prompts lose their KV caches and return to stage $s = 0$, re-entering the queue as new arrivals.
5. **Cascade:** These restarted prompts compete with new arrivals. The imbalance persists, triggering repeated evictions.

This example reveals the **cascading eviction problem** our algorithms target. Even with $C \geq M^*$ (the equilibrium memory from our fluid analysis, Section 3), poor admission control triggers this cascade, destabilizing a system whose capacity is theoretically sufficient for stability. The eviction-restart cycle can persist indefinitely, reducing the effective throughput, the completion rate net of evictions, by as much as 20% (Example 2 quantifies this for FCFS).

Core Challenge. When output lengths are unknown, optimistic admission risks overflow; pessimistic admission wastes capacity. Our threshold-based algorithms resolve this dilemma by: (i) preventing evictions through controlled admission, (ii) exploiting memory fully via batching, (iii) classifying prompts on-the-fly as they complete decode stages. This approach maintains the system

near the load-balanced equilibrium derived in Section 3, where arrivals match completions and memory use is stable.

2.5. Performance Metrics

We define the metrics guiding scheduling optimization:

- **Throughput:** Average number of tokens generated in decode phase per unit time, measuring system efficiency. We count throughput only after prompt completion to ensure validity of the output.
- **Latency:** Average elapsed time from prompt arrival to completion, capturing responsiveness.
- **Time to First Token (TTFT):** Average delay from prompt arrival to the generation of the first token, reflecting initial user-perceived delay.

2.6. Optimization Problem and Policy Space

We formalize the problem as optimizing throughput subject to latency and TTFT constraints over continuous time horizon $[0, T]$:

Policy Space Π . We define the class of admissible scheduling policies Π . At each decision epoch t , a policy $\pi \in \Pi$ observes the system state and makes scheduling decisions subject to the following constraints:

Information Structure. The system state at time t includes: (i) the set of all prompts that have arrived by time t , along with their input lengths l_i ; (ii) the current stage s_i^t of each prompt i (i.e., how many output tokens have been generated); (iii) whether each prompt's KV cache is currently stored on GPU. Policies are *non-anticipating*: decisions at time t depend only on information available at or before time t . In particular, the output length l'_i of a prompt may be unknown until the prompt completes (generates an end-of-sequence token). This unknown output length is a key challenge addressed by our Nested WAIT algorithm (Section 5).

Admissible Actions. At each decision epoch, a policy may:

1. **Admit:** Select a subset of waiting prompts (at stage $s = 0$) to enter the system. Admitted prompts wait until included in a batch for prefill.
2. **Batch:** Form a batch B^t of prompts to process in the current iteration. After prefill completes, a prompt's KV cache is allocated on GPU.

3. **Preempt**: Pause a prompt while retaining its KV cache on GPU. The prompt can resume later without recomputation.

4. **Evict**: Remove a prompt's KV cache from GPU to free memory when the constraint (2) cannot be satisfied. Eviction follows a **LIFO (Last In, First Out)** policy: the most recently admitted prompts are evicted first. The evicted prompt returns to the waiting queue at stage $s = 0$, losing all prior computation.

Memory Constraint. At all times, the total KV cache stored on GPU must satisfy the capacity constraint (2).

Preemption and eviction are both permitted, but eviction is costly: the evicted prompt re-enters the system as a new arrival and must repeat all processing stages from the beginning. Our WAIT algorithms are designed to *prevent eviction* by controlling admission through thresholds, thereby avoiding the memory pressure that causes eviction.

3. Fluid Model and Equilibrium

Unlike traditional queueing systems where job sizes are fixed, LLM inference faces a unique challenge: KV cache memory grows dynamically as prompts progress through decode stages. Each additional generated token increases the prompt's memory consumption, making it difficult to predict when total memory usage will exceed capacity C and trigger costly evictions. These evictions waste the partial computation already invested in interrupted prompts, degrading both throughput and latency.

We develop a fluid model to characterize the system's equilibrium state, a balanced configuration in which arrival rates match completion rates and memory usage stabilizes. This equilibrium serves two purposes. First, it quantifies the maximum sustainable throughput **Throughput*** under memory constraint C , establishing a benchmark for any scheduling policy. Second, it reveals the load-balancing principle that guides our algorithm design (Section 4): maintain the system close to equilibrium to prevent evictions. When prompts distribute evenly across processing stages, memory consumption remains predictable and overflow risk minimizes.

We first analyze a system with a single prompt type to build intuition about equilibrium dynamics, then extend to multiple heterogeneous types. The equilibrium provides a theoretical benchmark; our WAIT algorithm (Section 4) achieves asymptotic optimality by approaching this benchmark through threshold-based admission control that keeps the system near the balanced state.

3.1. Single-Type Fluid Model

We begin with a system handling prompts of a single type j . Each prompt's memory usage depends on its processing stage: l_j tokens in the prefill stage ($s = 0$) and $l_j + s$ tokens in the s -th decode stage ($s = 1, \dots, l'_j$). When the prompt completes stage $s = l'_j$, it finishes decoding and its KV cache is cleared from GPU memory.

In equilibrium, we assume the system maintains n_j^* active prompts distributed evenly across all $(l'_j + 1)$ stages, with $n_j^*/(l'_j + 1)$ prompts at each stage. This balanced distribution is crucial: if prompts accumulate at later decode stages, where memory consumption is highest ($l_j + l'_j$ tokens), total usage may exceed capacity C , forcing evictions that waste partial computation. By spreading prompts evenly, we minimize overflow risk while maximizing throughput. This equilibrium model allows us to derive the system's fundamental capacity limit.

Before deriving the equilibrium parameters, we identify when the system becomes fundamentally unstable regardless of the scheduling policy used. Intuitively, each prompt requires total processing time $d_1(l'_j + 1)(l_j + l'_j/2)$ to load its KV cache across all $(l'_j + 1)$ stages. When the arrival rate exceeds the inverse of this processing time, the queue grows unbounded.

PROPOSITION 1. *When the condition $\lambda_j d_1(l'_j + 1) \left(l_j + \frac{l'_j}{2}\right) \geq 1$ is satisfied, the system becomes unstable in the sense that the expected average latency under any scheduling policy with deterministic arrival rate λ_j grows linearly with time. Formally,*

$$\mathbb{E}[\text{Latency}^{(T, \pi)}] = \Omega(T) \quad \text{as } T \rightarrow \infty.$$

This condition represents the fundamental capacity limit: arrivals outpace completions, causing latency to grow linearly with time. The proof constructs an equilibrium batch and shows that even in this balanced state, arrivals exceed completions when the condition holds (Appendix A.1). We therefore assume $\lambda_j d_1(l'_j + 1) \left(l_j + \frac{l'_j}{2}\right) < 1$ for the remainder of this section, ensuring the system can reach equilibrium.

The total memory usage in equilibrium depends on the distribution of prompts across stages:

$$M_j^* = \frac{n_j^*}{l'_j + 1} \sum_{s=0}^{l'_j} (l_j + s) = n_j^* \left(l_j + \frac{l'_j}{2} \right), \quad (3)$$

where we sum over all $(l'_j + 1)$ stages, each containing $n_j^*/(l'_j + 1)$ prompts. The factor $(l_j + l'_j/2)$ represents the average memory per prompt across its lifecycle, from prefill (l_j) to final decode stage ($l_j + l'_j$).

To find n_j^* , we use the equilibrium condition: arrivals must equal completions. Each iteration processes one stage for all n_j^* prompts, taking time $(d_0 + d_1 M_j^*)$. During this time, $\lambda_j (d_0 + d_1 M_j^*)$ new prompts arrive. For equilibrium, this must equal the $n_j^*/(l'_j + 1)$ prompts that complete:

$$\underbrace{(d_0 + d_1 M_j^*)}_{\text{Time per iteration}} \lambda_j = \left(d_0 + d_1 n_j^* \left(l_j + \frac{l'_j}{2} \right) \right) \lambda_j = \frac{n_j^*}{l'_j + 1}.$$

Solving for n_j^* yields:

$$n_j^* = \frac{d_0 \lambda_j (l'_j + 1)}{1 - d_1 \lambda_j (l'_j + 1) (l_j + \frac{l'_j}{2})}.$$

Substituting this into the memory equation, the equilibrium memory equals:

$$M_j^* = n_j^* \left(l_j + \frac{l'_j}{2} \right) = \frac{d_0 \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}{1 - d_1 \lambda_j (l'_j + 1) (l_j + \frac{l'_j}{2})}. \quad (4)$$

This equilibrium batch size balances arrivals and completions. The denominator $(1 - d_1 \lambda_j (l'_j + 1) (l_j + l'_j/2))$ represents the fraction of iteration time available for processing; as this fraction approaches zero, the batch size grows unbounded (approaching instability). The average throughput at equilibrium is the number of decode tokens generated per iteration divided by the time consumption for one iteration. In each iteration, $n_j^*/(l'_j + 1)$ prompts complete (those at stage l'_j), each generating l'_j decode tokens:

$$\text{Throughput}_j^* = \frac{n_j^* \cdot l'_j}{(l'_j + 1) (d_0 + d_1 n_j^* (l_j + \frac{l'_j}{2}))} = \lambda_j l'_j, \quad (5)$$

where the equality follows from the equilibrium condition in line 46. This shows how arrival rate, memory, and throughput relate in equilibrium, where arrivals equal completions.

3.2. Multiple-Type Fluid Model

The single-type analysis provides intuition, but real systems handle multiple prompt types simultaneously, each with different input lengths l_j and output lengths l'_j . This heterogeneity introduces a key challenge: types compete for the same memory capacity C . Admitting a long prompt (large l'_j) may force evicting multiple short prompts, wasting their partial computation; this cascading failure is analyzed in Section 4. We now extend the equilibrium model to capture this multi-type interaction.

We assume the system maintains n_j^* active prompts of each type $j \in \{1, \dots, m\}$ in equilibrium. Following the derivation of (4), the total memory occupancy equals:

$$M^* = \sum_{j=1}^m n_j^* \left(l_j + \frac{l'_j}{2} \right) \quad (6)$$

with:

$$\underbrace{(d_0 + d_1 M^*)}_{\text{Time per iteration}} \lambda_j (l'_j + 1) = n_j^*, \forall j \in \{1, 2, \dots, m\}$$

since the arrivals during each iteration should equal the number of completions in equilibrium, and there are $(l'_j + 1)$ stages total. Substituting the above equations into (6) yields

$$M^* = \frac{d_0 \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}{1 - d_1 \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)} \quad (7)$$

Aggregating the results above, we get the processing time per iteration in equilibrium:

$$\begin{aligned} &\text{Processing Time}^* \\ &= d_0 + d_1 M^* \\ &= \frac{d_0}{1 - d_1 \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}, \end{aligned} \quad (8)$$

as well as the average throughput:

$$\text{Throughput}^* = \sum_{j=1}^m \frac{n_j^* \cdot l'_j}{(l'_j + 1) \cdot (d_0 + d_1 M^*)} = \sum_{j=1}^m \lambda_j l'_j \quad (9)$$

The equilibrium throughput benchmarks all non-predictive policies within the policy class Π . The following proposition demonstrates that the expected throughput of any policy in Π cannot exceed the equilibrium throughput **Throughput**^{*}, as specified in Equation (9).

PROPOSITION 2. *Suppose that the memory capacity satisfies $C \geq M^*$, where M^* denotes the equilibrium memory given in (7). Then, for any online policy $\pi \in \Pi$, the expected throughput satisfies*

$$\mathbb{E}[\text{Throughput}^{(T,\pi)}] \leq \text{Throughput}^*,$$

where Throughput^* is given in (9).

This result has two implications for algorithm design. First, it quantifies the theoretical limit: no non-predictive policy can exceed $\text{Throughput}^* = \sum_{j=1}^m \lambda_j l'_j$ when $C \geq M^*$. This establishes a performance benchmark for any scheduling policy. Second, it suggests a design principle: any policy that prevents evictions and maintains balanced stages can approach this throughput. This motivates our WAIT algorithm (Section 4), which uses thresholds to keep the system near equilibrium, preventing the cascading failures that cause FCFS to fall short (see Example 2).

The equilibrium analysis reveals a critical design principle: maintain balanced prompt distribution across stages. Having exactly $n_j^*/(l'_j + 1)$ prompts at each stage prevents two failure modes. First, if prompts accumulate at later decode stages (where each consumes more memory), total usage may exceed capacity C , forcing evictions that waste partial computation. Second, if prompts concentrate at early stages while later stages starve, the system leaves processing capacity unused, reducing throughput.

This load-balancing principle directly motivates our WAIT algorithm (Section 4). By controlling when prompts advance to the next stage via thresholds, WAIT keeps the system close to the balanced equilibrium, preventing the cascading evictions that plague FCFS (see Example 2 in Section 4). The fluid model thus provides both a performance benchmark and a design guide.

In classical queueing theory, fluid models typically lead to ordinary differential equations (ODEs) describing the continuous transition of queue lengths over time. Constructing such ODEs for LLM inference, however, presents technical challenges. Eviction events, which are triggered when memory usage exceeds capacity, cause discontinuous jumps in the system state, as KV caches of evicted prompts are cleared entirely rather than decreasing smoothly. At boundaries where certain stage populations reach zero or where drift terms vanish, the direction of state transition becomes ambiguous under standard ODE analysis. We therefore adopt a discrete, iteration-based framework that directly characterizes equilibrium and uses it as a performance benchmark pending a complete characterization of such an asymptotic regime.

4. Scheduling with Known Arrival Types: The WAIT Algorithm

This section introduces the *Waiting for Accumulated Inference Threshold* (WAIT) algorithm, a scheduling policy that achieves asymptotically optimal effective throughput under known prompt arrival types. Using the fluid model established in Section 3, WAIT forms each batch by accumulating prompts until specific thresholds are met. This approach uses resources efficiently in stochastic environments by maintaining the system near equilibrium while respecting memory constraints and optimizing latency and time-to-first-token (TTFT). We assume that the memory capacity $C \geq M^*$, the equilibrium memory capacity given in (7) (otherwise the throughput can never approach the optimal fluid throughput (9) by Proposition 2).

REMARK 1 (ON THE ASSUMPTION $C \geq M^*$). Our theoretical analysis assumes $C \geq M^*$, as this is the necessary condition under which the system can achieve stable operation without accumulating backlog. When $C < M^*$ (the overloaded regime), our algorithms remain effective: the eviction prevention mechanism still outperforms baselines, as demonstrated in Section 6. Avoiding eviction cascades provides value across all operating regimes.

4.1. Motivation: Why Naive Scheduling Fails

We first demonstrate why naive scheduling policies fail even when memory capacity is sufficient, motivating the design of WAIT.

Consider what happens under a First-Come-First-Serve (FCFS) policy that processes prompts immediately upon arrival. Even when the memory capacity C equals the equilibrium memory M^* (a capacity theoretically sufficient to handle the workload), FCFS can trigger catastrophic failure. The following proposition formalizes this observation.

PROPOSITION 3. *When the memory capacity C equals the equilibrium memory M^* , there exists an instance where the First-Come-First-Serve (FCFS) policy incurs a throughput gap of $\text{Throughput}^* - \mathbb{E}[\text{Throughput}^T] = \Omega(1)$ as $T \rightarrow \infty$.*

The proof is provided in Appendix. To illustrate the mechanism behind this failure concretely, consider the following example.

EXAMPLE 2 (EVICTION CASCADE UNDER FCFS). Consider a single prompt type where each prompt is a simple query such as “Hello” with a 1-token response “Hi” (prefill length $l = 1$ and

decode length $l' = 1$). Each prompt occupies 1 unit of memory after prefill and 2 units after decode. Let the memory capacity be $C = 12$ and arrival rate $\lambda = 4$. The fluid equilibrium has 4 prompts at each stage (prefill and decode), using exactly $4 \times 1 + 4 \times 2 = 12$ units of memory with 4 completions per iteration.

Now suppose the system starts slightly imbalanced with 6 prompts at stage 0 (awaiting prefill) and 3 prompts at stage 1 (in decode). Under FCFS:

1. **Iteration 1:** Process all 9 prompts. The 3 decode prompts complete, but the 6 prefill prompts advance to decode, requiring $6 \times 2 = 12$ units. Memory is exactly full with only 3 completions.
2. **Iteration 2:** During iteration 1, approximately 4 new prompts arrive. To admit them for prefill ($4 \times 1 = 4$ units needed), we must evict 2 decode prompts (freeing $2 \times 2 = 4$ units). Now we have 4 prefill + 4 decode = $4 + 8 = 12$ units.
3. **Iteration 3:** The 4 decode prompts complete (4 completions). The 2 previously evicted prompts, having lost their KV caches, must now restart from the prefill phase (stage $s = 0$), re-entering the queue as new arrivals and competing for resources alongside genuinely new prompts. This restart mechanism perpetuates the imbalance.

This eviction-restart cycle persists, yielding approximately 3–3.5 completions per iteration instead of the 4 sustained at the fluid equilibrium. Even though $C = M^* = 12$ (theoretically sufficient capacity for stability), FCFS triggers cascading evictions that reduce the effective throughput, the completion rate net of evictions, by as much as 25%.

This example shows that *memory sufficiency alone does not guarantee stability*: a system whose capacity is theoretically sufficient for stability can become unstable under poor scheduling. The root cause is that FCFS does not maintain *load balance*: it admits prompts greedily, causing the system to deviate from the fluid equilibrium and triggering the eviction-restart cycle.

4.2. The WAIT Algorithm

The WAIT algorithm prevents eviction cascades by maintaining load balance through a threshold mechanism. The core idea is to control admission at each decode stage so that the rate of new arrivals matches the rate of completions. This keeps the system close to the fluid equilibrium derived in Section 3, *approaching load balance* where arrival and completion rates match. This principle of maintaining the system near equilibrium through controlled admission is central to preventing the cascading failures illustrated in Example 2.

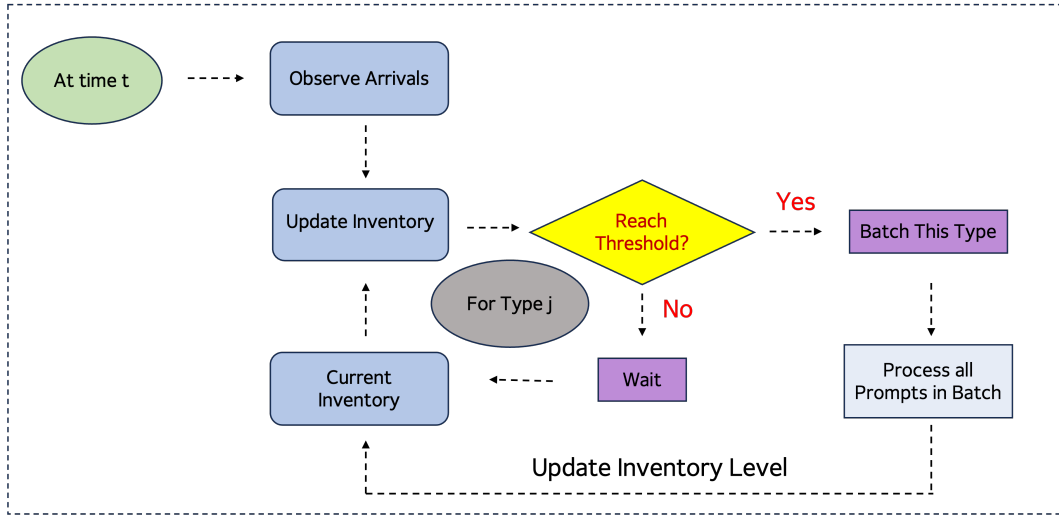


Figure 4 Pipeline of Algorithm 1

The threshold-and-wait mechanism achieves three goals. First, it prevents *eviction cascades* by controlling admission. Second, it *decouples the complex multi-stage, multi-type dynamics*, transforming the system into m independent single-type analyses. Third, by waiting until thresholds are met, the batch approaches the fluid equilibrium size, *using memory capacity fully to maximize throughput*. The decoupling is crucial: without thresholds, delays in one type would affect memory availability for others through shared capacity constraints, creating feedback loops that resist classical drift analysis. The threshold mechanism breaks these inter-type dependencies, enabling tractable per-type analysis. The third goal reflects a complementary principle: memory is a resource to exploit rather than merely a constraint, since larger batches amortize the fixed overhead d_0 across more prompts per iteration, and waiting until the threshold is met ensures each batch approaches the equilibrium size that maximizes this amortization while staying within safe limits.

Algorithm Description. For each prompt type $j \in [m]$, WAIT sets a threshold n_j derived from the fluid model. At time t , the algorithm monitors the inventory $n_{j,s}^t$, the number of waiting prompts of type j at stage s . Type j prompts are included in a batch only when:

$$n_{j0}^t \geq n_j.$$

When this condition is met, WAIT constructs a batch by selecting $\min\{n_j, n_{j,s}^t\}$ prompts of type j at each stage s for all types meeting the threshold. If no type satisfies the condition, the algorithm waits for more arrivals. KV caches for waiting prompts are preserved, avoiding redundant computation. The complete specification is in Algorithm 1 and Figure 4.

Algorithm 1 WAIT: Waiting for Accumulated Inference Threshold

Input: Memory C , arrival rates λ_j , thresholds n_j satisfying (10), $\forall j \in [m]$

- 1: Initialize prompt inventory $n_{js} \leftarrow 0$ for all $j \in [m], s \in \{0, 1, \dots, l'_j\}$
- 2: Initialize event queue with arrival events for each type j
- 3: Set current time $t \leftarrow 0$
- 4: **while** True **do**
- 5: Wait for the next event (arrival or batch completion)
- 6: **if** event is an arrival of type j **then**
- 7: Update inventory: $n_{j0} \leftarrow n_{j0} + 1$ ▷ Add new prompt to waiting queue of prefill phase
- 8: **else if** event is a batch completion **then**
- 9: Update inventory: For each j in batch B , $n_{js} \leftarrow n_{js} - \min\{n_j, n_{js}\}$ for $s = 0, \dots, l'_j$
- 10: Advance prompts: For each j , move $\min\{n_j, n_{js}\}$ prompts from stage s to $s + 1$
- 11: Clear KV caches for prompts reaching stage $l'_j + 1$ ▷ Free memory for completed prompts
- 12: **end if**
- 13: Check if $n_{j0} \geq n_j$ for any $j \in [m]$ ▷ Check threshold for batching
- 14: **if** condition $n_{j0} \geq n_j$ is met for some j **then**
- 15: Form batch B by selecting $\min\{n_j, n_{js}\}$ prompts of type j at each stage s for all j meeting the condition
- 16: Process batch B , keep prompts outside the batch waiting and do not delete their KV caches
- 17: **end if**
- 18: **end while**

4.3. Asymptotic Analysis

We evaluate the performance of WAIT in an asymptotic regime that captures long-run system behavior. Specifically, we scale the arrival rates to $\lambda_j^{(\zeta)} = \zeta \lambda_j$ and processing speeds accordingly $((d_0^{(\zeta)}, d_1^{(\zeta)}) = (\zeta^{-1} d_0, \zeta^{-1} d_1))$ while keeping memory capacity C fixed. This can equivalently be viewed as scaling the time horizon from T to ζT while keeping arrival rates and processing speeds unchanged. As $\zeta \rightarrow \infty$, we obtain infinite-horizon limits that characterize the system's long-run average performance, allowing us to study throughput stability under sustained load.

Performance Metric: We focus on the average throughput, denoted $\text{Throughput}^{(\zeta, \pi)}$, where π represents the WAIT policy, defined as:

$$\text{Throughput}^{(\zeta, \pi)} = \frac{1}{\zeta T} \sum_{j=1}^m N_j^{(\zeta, \pi)},$$

with $N_j^{(\zeta, \pi)}$ being the total number of type- j tokens processed under scaling ζ . The average latency $\text{Latency}^{(\zeta, \pi)}$, $\text{TTFT}^{(\zeta, \pi)}$ and TTFT are defined similarly by dividing the quantity by ζ . Our objective is to demonstrate that $\text{Throughput}^{(\zeta, \pi)}$ approaches the fluid equilibrium benchmark $\text{Throughput}^* = \sum_{j=1}^m \lambda_j l'_j$ (9) as $\zeta \rightarrow \infty$ while giving bound to the latency and TTFT.

4.4. Theoretical Analysis

When memory capacity satisfies $C \geq M^*$ (where M^* is the equilibrium memory from Equation (7)), consider the WAIT policy π parameterized by thresholds $[n_1, \dots, n_m]$, where n_j is the threshold

for type- j prompts at each stage. WAIT is asymptotically optimal when the thresholds satisfy:

$$\begin{aligned} \Delta T(n_{1:m}) &:= d_0 + d_1 M^\pi \leq \frac{n_j}{\lambda_j}, \forall j \in [m], \\ M^\pi &= \sum_{j=1}^m n_j (l'_j + 1) (l_j + \frac{l'_j}{2}) \end{aligned} \quad (10)$$

The intuition follows from Equation (8): when type j is included in a batch, the iteration time ΔT ensures that expected arrivals of type j do not exceed n_j . This keeps the system close to the fluid equilibrium, using memory capacity fully for throughput while avoiding overflow.

THEOREM 1. *Assume the thresholds $\pi = [n_1, \dots, n_m]$ satisfy (10), then we have*

$$\begin{aligned} \text{Throughput}^* - \mathbb{E} [\text{Throughput}^{(\zeta, \pi)}] &= O((\zeta T)^{-\frac{1}{2}}), \\ \mathbb{E} [\text{Latency}^{(\zeta, \pi)}], \mathbb{E} [\text{TTFT}^{(\zeta, \pi)}] &= O((\zeta T)^{\frac{1}{2}}). \end{aligned}$$

Moreover, when we have $\Delta T_{[1, \dots, m]} < n_j / \lambda_j, \forall j \in [m]$ where $\Delta T_{[1, \dots, m]}$ is defined in (10), it holds that

$$\begin{aligned} \text{Throughput}^* - \mathbb{E} [\text{Throughput}^{(\zeta, \pi)}] &= O(1/(\zeta T)), \\ \mathbb{E} [\text{Latency}^{(\zeta, \pi)}], \mathbb{E} [\text{TTFT}^{(\zeta, \pi)}] &= O(1). \end{aligned}$$

The WAIT algorithm prevents eviction cascades through a simple but powerful principle: controlled admission via thresholds. By requiring $n_{j0}^t \geq n_j$ before admitting type- j prompts, the algorithm ensures that expected arrivals during each iteration do not exceed completions. This keeps each prompt type close to its equilibrium inventory level, preventing the overflow that triggers eviction cascades in FCFS (Example 2).

The threshold mechanism simultaneously achieves three design goals: (1) *eviction prevention* by controlling when prompts enter batches; (2) *approaching load balance* by maintaining the system near the fluid equilibrium where arrivals match completions; and (3) *memory exploitation* by waiting until batches can be formed close to the equilibrium size M^* , maximizing throughput.

From an analytical perspective, the threshold decouples the m prompt types: we can analyze each type independently, as if it were the only type in the system. Without this decoupling, delays in one type would affect memory availability for others through the shared capacity constraint, creating feedback loops that resist classical drift analysis. With the types decoupled, we construct a dominating process per type and apply Lindley-style drift analysis to establish the asymptotic-optimality bounds. The decoupling itself is what makes the analysis tractable; the complete proof is in Appendix B.

Theorem 1 provides performance bounds that match the worst-case lower bounds for latency and Time to First Token (TTFT). The following proposition establishes these lower bounds.

PROPOSITION 4. *There exists an instance where $C = M^*$, and for any non-predictive online policy π , the expected average latency and TTFT satisfy*

$$\mathbb{E}[\text{Latency}^{(\zeta, \pi)}], \quad \mathbb{E}[\text{TTFT}^{(\zeta, \pi)}] = \Omega((\zeta T)^{1/2}).$$

Moreover, we demonstrate that the latency for type j prompts depends exclusively on the relationship between $\Delta T_{[1, \dots, m]}$ and n_j/λ_j . Specifically, when $\Delta T_{[1, \dots, m]} < n_j/\lambda_j$, the average latency and TTFT for type j prompts are $O(1)$, offering a more precise performance estimate than that provided in Theorem 1.

A limitation of WAIT is its reliance on known arrival types and rates, which may not hold in realistic environments (though we can approximate it by using predictors, see e.g. (Fu et al. 2025)). In the next section, we address this issue by extending WAIT to handle unknown arrival types, introducing a nested variant that adapts to stochastic settings with unknown prompt types upon arrival.

5. Scheduling with Unknown Arrival Types: The Nested WAIT Algorithm

The WAIT algorithm achieves near-optimal throughput when we know prompt types at arrival. In practice, however, we cannot predict how many tokens a prompt will generate—output lengths reveal themselves only as decode progresses. The *Nested Waiting for Accumulated Inference Threshold* (Nested WAIT) algorithm exploits this gradual revelation: **prompts classify themselves on-the-fly** as they decode. Short prompts complete early and exit; longer prompts naturally advance to later segments. **No prediction is needed**—the algorithm simply lets each prompt reveal its type through its decode trajectory.

EXAMPLE 3 (UNKNOWN OUTPUT LENGTHS). Continuing Example 2, suppose output lengths are now *unknown* at arrival: each “Hello” prompt receives either “Hi” (1 token) or “Hi there!” (2 tokens). The scheduler faces a dilemma. If it optimistically assumes all outputs are short (1 token each), it admits 6 prompts with capacity $C = 12$, requiring $6 \times 2 = 12$ units after decode. But if some outputs turn out to be 2 tokens (requiring 3 units each), memory overflows and eviction cascades begin. Conversely, if the scheduler conservatively assumes all outputs are long, it admits only 4 prompts to stay safe, wasting 25% of capacity when most outputs are actually short.

This dilemma (optimism leads to eviction, pessimism wastes capacity) leads to our design principles: (i) *avoid eviction* by not over-committing memory; (ii) *classify on-the-fly* by letting prompts reveal their type as they decode; (iii) *don't waste throughput* by exploiting available capacity; (iv) *let shorter prompts finish first* so they free memory without being blocked by longer ones.

5.1. Algorithm Overview

Example 3 reveals a fundamental dilemma: optimism risks eviction, pessimism wastes capacity. We resolve this through on-the-fly classification. Rather than predicting output lengths at arrival, Nested WAIT lets prompts reveal their output as they decode. Short prompts complete early and exit; longer prompts naturally advance to later segments.

The core idea is **nested thresholds**: segment 1 handles all prompts from decode stage 0 to l'_1 ; segment 2 handles prompts that survive past l'_1 (i.e., types 2 through m); segment 3 handles prompts past l'_2 ; and so on. Each segment k has its own threshold n_k controlling when it processes prompts. This nesting achieves two goals: (i) short prompts complete in early segments without being blocked by long prompts; (ii) each segment operates at high memory utilization for its remaining types.

Formally, let output lengths satisfy $l'_1 < l'_2 < \dots < l'_m$, where type j has decode length l'_j . We organize inference into m nested segments, where segment k spans decode stages from $l'_{k-1} + 1$ to l'_k (with $l'_0 = 0$). The algorithm uses thresholds $\pi = [n_1, \dots, n_m]$ where n_k controls when segment k begins processing and limits batch size per stage.

Segment 1 processes all newly arrived prompts (at combined rate $\lambda_1 + \dots + \lambda_m$) through decode stages 0, 1, $\dots$, l'_1 . After $l'_1 + 1$ iterations, two outcomes emerge: type-1 prompts have completed their output and **exit the system**, freeing their memory; all other prompts (types 2 through m) **advance to segment 2**. Segment 2 now processes only these longer prompts (at reduced rate $\lambda_2 + \dots + \lambda_m$) through stages $l'_1 + 1, \dots, l'_2$. This pattern repeats: after $l'_2 - l'_1$ more iterations, type-2 prompts exit, and types 3 through m advance to segment 3. **Prompts classify themselves** by completing (short) or continuing (long) at each segment boundary.

If segment k^* has fewer than n_{k^*} prompts, we pause processing for that segment and all later ones, retaining their KV caches on GPU. We continue batching exactly n_k prompts per stage in segments $k < k^*$ and perform iterations. Since new long prompts arrive continuously, segment k^*

resumes once its threshold is met. Algorithm 2 formalizes this procedure, and Figure 5 illustrates the pipeline.

We assume identical prefill lengths $l_1 = l_2 = \dots = l_m$ for clarity, though the general case extends similarly to Algorithm 1 by forming segment groups for each prefill length. The number of output lengths m minimally impacts performance, which depends primarily on total arrival rates. The number of segments can also differ from m while retaining near-optimal guarantees (Section 7).

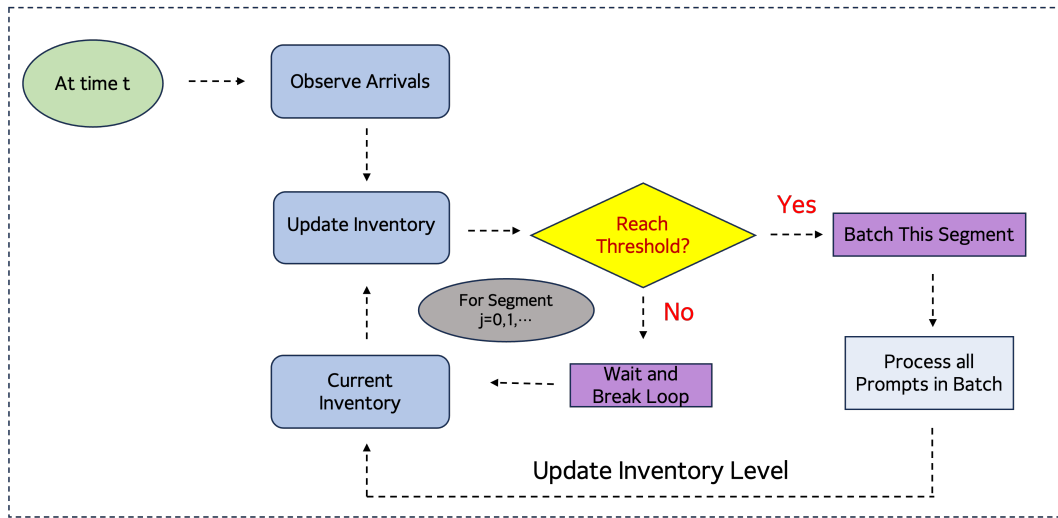


Figure 5 Pipeline of the Nested WAIT Algorithm

Consider Example 3 with two prompt types: “Hello” $\rightarrow$ “Hi” (1 token, type 1) and “Hello” $\rightarrow$ “Hi there!” (2 tokens, type 2). At arrival, we cannot distinguish them. After one decode iteration, however, **the prompts reveal themselves**: type-1 prompts have completed their output (“Hi”) and exit; type-2 prompts have generated only their first token (“Hi”) and continue. Nested WAIT exploits this revelation by partitioning processing into two segments. **Segment 1** processes all newly arrived prompts (both types) through stages 0, 1 (up to $l'_1 = 1$) at combined rate $\lambda_1 + \lambda_2$. **Segment 2** processes only type-2 prompts (those that survived segment 1) through stage 2 (up to $l'_2 = 2$) at reduced rate λ_2 . **No prediction is needed**—prompts simply classify themselves by completing or continuing at the segment 1 boundary.

Figure 6 visualizes the algorithm in action. At time $t = 0$, prompts $P1$ and $P2$ arrive but are insufficient to trigger processing, as Segment 1 has fewer than 3 prompts. At $t = 1$, $P3$ arrives, satisfying the Segment 1 threshold, and the batch is processed. Since these prompts have just begun

decoding, none qualify for Segment 2. At $t = 5$, a new wave of prompts ($P4, P5$) arrives, again below the threshold. By $t = 7$, the threshold is met with $P6, P7$, and processing proceeds. Now, $P3$, having reached a deeper decode stage, transitions to Segment 2. However, since the Segment 2 threshold is still unmet, only Segment 1 is processed. Finally, at $t = 14$, enough prompts have advanced into Segment 2 ($P3, P5, P6$, etc.) to satisfy both segment thresholds. At this point, the Nested WAIT algorithm jointly schedules both segments, batching according to type-specific readiness. This example shows how the algorithm defers processing to ensure type separation and stage-aligned batching.

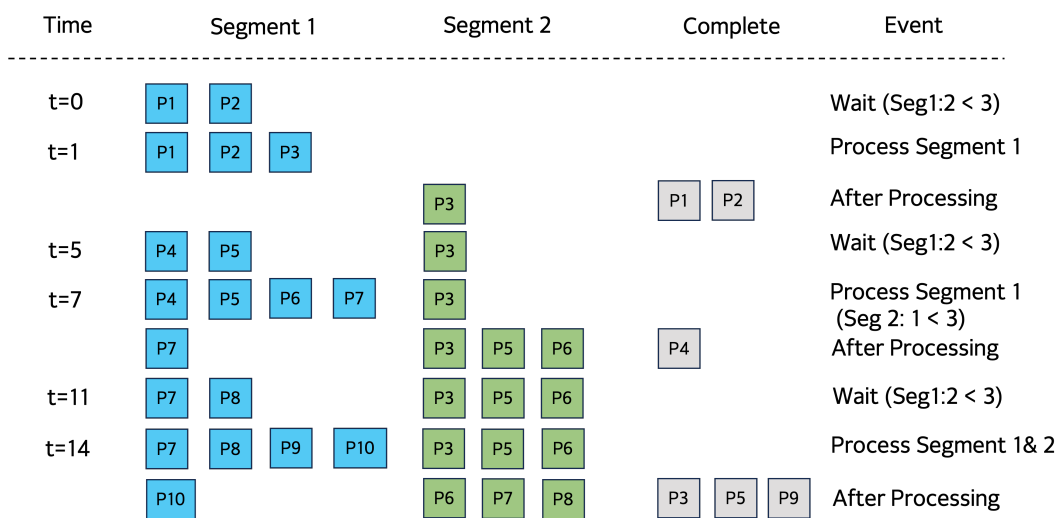


Figure 6 Example of the Nested WAIT Algorithm

This nested structure directly implements the four design principles from Example 3: (i) we **avoid eviction** by never exceeding thresholds; (ii) prompts **classify on-the-fly** at segment boundaries; (iii) we **don't waste throughput** because each segment operates at its own optimal threshold; (iv) **short prompts finish first** within segment 1, freeing memory without waiting for long prompts. Crucially, **no prediction is needed**—prompts simply reveal their output by completing or continuing. This achieves high memory utilization **for all types simultaneously**, not just the shortest or longest. We now formalize this intuition.

5.2. Main Theorem

We now analyze Nested WAIT in the asymptotic regime, following the same framework as Section 4. The unknown-output-length setting introduces **additional stochastic variability**: since we cannot predict which prompts will be long, some may queue at segment boundaries waiting for

Algorithm 2 Nested WAIT: Nested Waiting for Accumulated Inference Threshold

Input: Memory capacity C , arrival rates λ_j for $j \in [m]$, thresholds n_k for segments $k \in [m]$

Input: Output lengths $l'_1 < l'_2 < \dots < l'_m$ for prompt types $j \in [m]$

▷ $Q_{k,s}$: number of prompts in segment k at decode stage s ; entry stage of segment k is l'_{k-1} (with $l'_0 = 0$)

- 1: Initialize $Q_{k,s} \leftarrow 0$ for all segments $k \in [m]$ and stages $s \in [l'_{k-1}, l'_k - 1]$
- 2: Initialize event queue; set current time $t \leftarrow 0$
- 3: **while** True **do**
- 4: Wait for the next event (arrival or batch completion)
- 5: **if** event is an arrival **then**
- 6: $Q_{1,0} \leftarrow Q_{1,0} + 1$ ▷ New prompts enter segment 1 at stage 0
- 7: **else if** event is a batch completion for segment k **then**
- 8: **for** each stage s in segment k **do**
- 9: Advance $\min\{n_k, Q_{k,s}\}$ prompts from stage s to $s + 1$
- 10: **end for**
- 11: **if** prompts reach stage l'_k (end of segment k) **then**
- 12: **if** $k < m$ **then**
- 13: Move to segment $k + 1$ at stage l'_k ▷ These are type- $(k + 1)$ or longer
- 14: **else**
- 15: Complete and clear KV caches
- 16: **end if**
- 17: **end if**
- 18: **end if**
- 19: Find largest k such that $Q_{k',l'_{k'-1}} \geq n_{k'}$ for all $k' \leq k$ ▷ All segments up to k meet threshold
- 20: **if** such k exists **then**
- 21: Form batch from segments $1, \dots, k$: select $\min\{n_{k'}, Q_{k',s}\}$ prompts per stage
- 22: Process batch; prompts outside batch wait with KV caches retained
- 23: **end if**
- 24: **end while**

thresholds. This affects both memory requirements and throughput guarantees. We characterize average throughput $\text{Throughput}^{(\zeta, \pi)}$, latency $\text{Latency}^{(\zeta, \pi)}$, and time-to-first-token $\text{TTFT}^{(\zeta, \pi)}$ under threshold policy $\pi = [n_1, \dots, n_m]$.

Before showing Nested WAIT achieves near-optimal throughput, we establish a lower bound showing that **some** extra memory beyond M^* is necessary when output lengths are unknown.

Intuition: When memory capacity equals exactly $C = M^*$ (the fluid equilibrium), any algorithm that cannot predict output lengths faces a dilemma at every iteration. If we admit too many prompts optimistically, some will reveal longer outputs than expected, causing overflow and eviction. If we admit too few conservatively, we waste capacity. Either scenario creates a throughput gap of $\Omega(1)$ over time.

PROPOSITION 5. *When memory capacity C equals the equilibrium memory M^* , there exists an instance where any online non-predictive policy π , unaware of prompt types upon arrival, incurs a throughput gap $\text{Throughput}^* - \mathbb{E}[\text{Throughput}^{(\zeta, \pi)}] = \Omega(1)$.*

This lower bound shows that uncertainty has a cost: we need more memory than the fluid equilibrium to hedge against unknown output lengths. Theorem 2 shows that only a **logarithmic** amount of extra memory suffices under Nested WAIT.

As in Section 4, thresholds must balance two goals: (i) **throughput efficiency** (don't wait too long to form batches); (ii) **segment stability** (ensure later segments receive enough prompts). This leads to two constraints, analogous to Equation (10).

Similar to Equation (10), we define $\Delta T_{[1,\dots,m]}(n_1, \dots, n_m)$ as the time cost per iteration with exactly n_k prompts per stage in segment k , for all $k \in [1, \dots, m]$. Thresholds must satisfy:

$$\begin{aligned} \Delta T_{[1,\dots,m]}(n_1, \dots, n_m) &< \frac{n_1}{\sum_{j=1}^m \lambda_j}, \\ \frac{n_{k+1}}{n_k} &> p_k, \forall k \in \{1, \dots, m-1\}, \end{aligned} \quad (11)$$

where $p_k = (\sum_{j=k+1}^m \lambda_j) / (\sum_{j=k}^m \lambda_j) < 1$. To state the memory requirement, we define the memory usage when running iteration with exactly n_k prompts per stage in segment k , for all $k \in [1, \dots, m]$ as:

$$\begin{aligned} M^\pi &= \sum_{k=1}^m n_k \left(l + \frac{1}{2} L'_k \right) \Delta l'_k, \\ \text{where } L'_k &= \sum_{r=1}^k l'_r, \Delta l'_k = l'_k - l'_{k-1}, l'_0 = 0. \end{aligned} \quad (12)$$

Here, the formula captures the peak batch memory when exactly n_k prompts occupy each segment k : the term n_k is the threshold (maximum prompts concurrently decoding in segment k), $\Delta l'_k = l'_k - l'_{k-1}$ counts the decode stages spanned by segment k , and $(l + \frac{1}{2} L'_k)$ represents the average memory per prompt within segment k , where $L'_k = l'_1 + \dots + l'_k$ is the cumulative decode length. We define the total number of iterations over time horizon $[0, T]$ as B . Therefore, we have $T \geq \sum_{b=1}^B \Delta T_{b\text{-th Batch}} \geq B \cdot \min \Delta T_{\text{Batch}}$, where $\Delta T_{b\text{-th Batch}}$ denotes the time cost of the b -th iteration and $\Delta T_{\min} = \min_b \{\Delta T_{\text{Batch}}\}$. Theorem 2 establishes asymptotic optimality for Nested WAIT in the asymptotic regime with unknown prompt types, given thresholds $\pi = [n_1, \dots, n_m]$ satisfying Equation (11).

THEOREM 2. Assume thresholds $\pi = [n_1, \dots, n_m]$ satisfy Equation (11), and memory M satisfies:

$$\begin{aligned} M^{(\zeta, \pi)} &\geq M^\pi + \sum_{k=2}^m (l + l'_{k-1}) \left(n_k + \theta_k^{-1} \ln \left(\frac{m\zeta B}{\delta} \right) \right) \\ &= O \left(2M^\pi + \sum_{k=1}^m (l + l'_{k-1}) \theta_k^{-1} \ln \left(\frac{m\zeta B}{\delta} \right) \right), \end{aligned} \quad (13)$$

where $\theta_k \geq 8(n_k - n_{k-1}p_k)/n_k$. Then:

$$\begin{aligned} \text{Throughput}^* - \mathbb{E}[\text{Throughput}^{(\zeta, \pi)}] &= O((\zeta T)^{-1}), \\ \mathbb{E}[\text{Latency}^{(\zeta, \pi)}], \mathbb{E}[\text{TFTT}^{(\zeta, \pi)}] &= O(1). \end{aligned}$$

Moreover, memory is not exceeded with probability at least $1 - \delta$ during time horizon $[0, T]$.

Nested WAIT extends the dimensionality reduction approach from WAIT to handle unknown output lengths. The core insight is that thresholds n_k in each segment k bound the state space, just as in WAIT—but now prompts classify themselves on-the-fly by completing or continuing at segment boundaries, rather than revealing their type at arrival. There is even no need to know the actual output—we only observe whether each prompt completes or continues. This on-the-fly classification creates a key technical challenge: prompts entering segment 2 at time t are precisely those that survived segment 1, creating a delayed arrival process with rate $\lambda_2 + \dots + \lambda_m$ that depends on the history of earlier segments. We address this through a coupling argument: the number of prompts transitioning from segment k to $k + 1$ couples to a Poisson process with rate $\sum_{j=k+1}^m \lambda_j$, allowing us to analyze each segment's queue independently despite the temporal dependency.

This coupling reveals why the safety buffer is necessary. When segment k has fewer than n_k prompts, processing pauses and prompts queue at the boundary. Because we cannot predict which prompts will advance to later segments, this queueing introduces stochastic fluctuations beyond the fluid limit. Doob's maximal inequality bounds the maximum queue length over time horizon $[0, T]$ with high probability, showing the bound grows only as $O(\ln(T/\delta))$ —this logarithmic overhead is the safety buffer in Equation (13). The key is that despite not knowing output lengths, the threshold mechanism keeps each segment close to its equilibrium batch size, maintaining the system near the load-balanced state where arrivals match completions. This prevents the cascading evictions that occur in FCFS (Example 2) while achieving the same design goals as WAIT: eviction prevention,

approaching load balance, and memory exploitation. The full proof in Appendix C formalizes the coupling construction and derives explicit constants.

The memory requirement in Equation (13) consists of two components with clear operational interpretations:

1. **Base memory M^π .** This is the equilibrium memory from the fluid model, the memory needed when exactly n_k prompts occupy each stage in segment k . It represents the steady-state memory consumption.

2. **Safety buffer $\sum_{k=2}^m (l + l'_{k-1})(n_k + \theta_k^{-1} \ln(m\zeta B/\delta))$:** This additional memory hedges against stochastic fluctuations when output lengths are unknown. It has three components:

- n_k : Accounts for prompts queueing at segment k boundaries, waiting for thresholds.
- $\theta_k^{-1} \ln(m\zeta B/\delta)$: High-probability protection against queue buildup (overflow occurs with probability $\leq \delta$).
- $(l + l'_{k-1})$: Memory consumption of prompts at the boundary between segments $k - 1$ and k .

This safety buffer is *logarithmic* in the time horizon T (via B) and confidence level $1/\delta$, making it a modest overhead compared to the base memory M^π . This is why Proposition 5 shows that *some* extra memory beyond M^* is necessary for unknown output lengths, but our Theorem 2 demonstrates that only a logarithmic amount suffices.

Number of types: The algorithm's performance depends primarily on arrival rates and output lengths, not the number of types m . Even for large m (e.g., output lengths $\{1, 2, \dots, m\}$), descending thresholds $n_1 > n_2 > \dots > n_m$ remain effective when arrival rates satisfy condition (11).

Safety buffer overhead: The memory requirement in (13) is typically dominated by the base term M^π . The safety buffer (second term) grows only logarithmically in the time horizon and is modest in practice. Section 6 provides numerical validation.

Segment-wise generalization: In practice, we use a **segment-wise variant** that groups m types into $L \leq m$ segments (Section 7). This design addresses two concerns: (i) robustness to varying or uncertain type distributions; (ii) for long decode lengths, maintaining separate thresholds for every type may be infeasible. Grouping into segments provides stable scheduling with similar guarantees. Having established theoretical foundations for both known and unknown output lengths, we now validate our algorithms experimentally (Section 6).

6. Numerical Experiments

We evaluate the WAIT and Nested WAIT algorithms on two classes of workloads: synthetic Poisson arrivals (Section 6.1) and a real prompt distribution sampled from lmsys-chat-1m (Section 6.2). On every workload we benchmark our algorithms against two widely deployed inference schedulers, vLLM (Kwon et al. 2023) and Sarathi (Agrawal et al. 2023); vLLM serves new arrivals in first-come-first-served (FCFS) order, while Sarathi additionally uses chunked prefill. We use Vidur (Agrawal et al. 2024a) as the discrete-event simulator, configured to model a single NVIDIA A100 80 GB serving Llama-2-7B. Vidur’s iteration-time predictions agree with direct vLLM measurements on a physical A100 within a mean absolute percentage error (MAPE) of 1.5% over the batch-size range used in our experiments; the validation details, together with end-to-end real-GPU runs of WAIT, are reported in Appendix G.2, and the per-workload parameter values are tabulated in Appendix F.

The presentation in this section follows a standard queueing diagnostic. Because the system is open, throughput equals the arrival rate λ whenever the scheduling policy is stable (i.e., the queue is positive recurrent); differences in throughput between policies therefore reflect differences in the extent of their stability regions, rather than differences in per-request processing speed. We plot mean end-to-end latency against λ and identify each policy’s *stability boundary* λ^* as the largest λ at which the mean latency remains bounded, i.e., the point at which the latency curve diverges. The same λ^* marks the transition in the effective-throughput curve: effective throughput tracks λ below λ^* , and saturates or declines above it (the latter happens when the policy suffers eviction cascades at high load).

Hardware. All experiments use Vidur to simulate a single NVIDIA A100 80 GB serving Llama-2-7B. The iteration-time model is Equation (1); its A100 calibration and GPU-side validation are reported in Appendix G.2.

Baseline configurations (vLLM, Sarathi). Both baselines use their default Vidur configurations and admit arrivals in FCFS order. Both use a 1% memory watermark, which is checked only when a new request is admitted; it does not bound the cumulative growth of already-admitted requests, so the pool can still fill up and trigger eviction under sustained load. Sarathi additionally bounds the prefill work per iteration to chunks of at most 512 tokens, which prevents a single large prefill from monopolizing the batch and starving decode requests. Neither baseline enforces an explicit admission threshold on the total number of in-flight requests.

WAIT and Nested WAIT settings. In the experiments below, we do not set the thresholds n_j and n_k directly. Under Nested WAIT, let m denote the number of decode-stage segments. We use WAIT for the two single-type synthetic workloads, p512d20 and p512d1000, and Nested WAIT for the two-type synthetic workload with arrival mix (0.7 p512d20, 0.3 p512d50), meaning that each arrival is p512d20 with probability 0.7 and p512d50 with probability 0.3, and for the real lmsys workload. For each workload and arrival rate, we choose a single total in-flight limit tl from the grid $\{20, 25, \dots, 40\}$ and derive the threshold vector. Under WAIT, tl determines the overall scale of the per-type thresholds n_j , and Equation (10) determines their relative magnitudes across types. Under Nested WAIT, tl plays the same role for the per-segment thresholds n_k , and Equation (11) determines their ratios across segments. A segment becomes active once its entry-stage inventory reaches the relevant threshold or once the segment is already full at its in-flight limit. For the two-type synthetic workload (0.7 p512d20, 0.3 p512d50), we use Nested WAIT with $m = 2$, corresponding to decode-stage segments 1–20 and 21–50, so that $n_2/n_1 > 0.3$ and $20n_1 + 30n_2 \approx tl$. For example, if $tl = 30$, one consistent choice is $(n_1, n_2) \approx (1, 1/3)$, which integerizes to segment in-flight limits of about 20 and 10. On the real lmsys workload, Nested WAIT again uses a decode-stage partition, with the sensitivity to the segment count reported in Section 6.2. Thus, the single scalar tl determines both the scale of the threshold vector and the total in-flight limit. Under the baseline settings used for the rate-sweep figures, WAIT and Nested WAIT do not trigger eviction.

6.1. Synthetic Workloads

We consider three synthetic Poisson workloads. The single-type workload p512d20 has prefill length 512 and decode length 20, isolating the effect of WAIT's threshold mechanism (Algorithm 1). The two-type workload (0.7 p512d20, 0.3 p512d50), meaning that each arrival is p512d20 with probability 0.7 and p512d50 with probability 0.3, tests the segment design of Nested WAIT (Algorithm 2) on prompts that share the prefill profile but differ in decode length. The long-decode workload p512d1000 keeps the same prefill length but increases the decode length to 1000, allowing us to examine the same threshold mechanism when KV cache growth is much more pronounced.

Single-type workload (WAIT). Figure 7 shows mean end-to-end latency (left, on a stretched-symlog scale that expands the stable region) and effective throughput (right), i.e., the completion rate net of work lost to eviction, versus arrival rate λ . Because all requests are homogeneous, this workload highlights the effect of threshold-based admission without the additional segment

structure used in Nested WAIT. WAIT sustains bounded latency up to $\lambda \approx 23$, whereas Sarathi and vLLM lose stability near $\lambda \approx 22$ and $\lambda \approx 19$, respectively. At low load the three policies are close, but the gap widens rapidly as λ approaches the boundary: WAIT attains the smallest mean latency at every rate in the swept range $\lambda \in \{12, \dots, 24\}$, with the gap to Sarathi growing from $\sim 2\%$ at $\lambda = 12$ to over 70% at $\lambda = 23$. This pattern matches the paper's mechanism. Because there is only one request type, the gain does not come from type separation; it comes from threshold-based admission itself, which keeps the in-system population controlled and delays the onset of instability. The right panel shows the same insight in throughput terms: WAIT follows the ideal λ -line up to a higher arrival rate than Sarathi and vLLM.

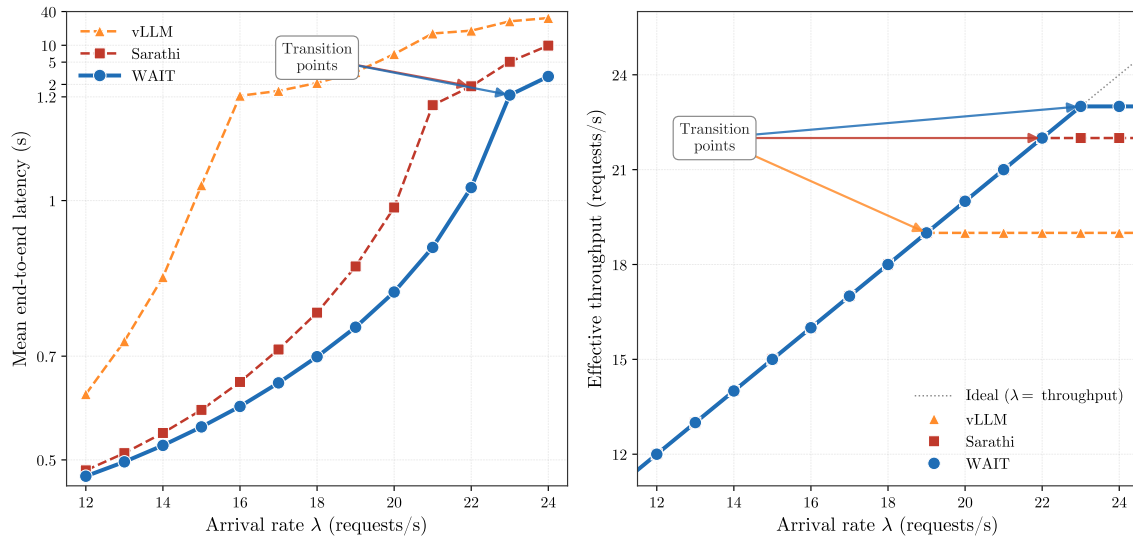


Figure 7 Single-type workload (p512d20, Poisson arrivals): mean end-to-end latency (left) and effective throughput (right) versus arrival rate λ . The arrows mark each policy's stability boundary λ^* ; the dotted line in the right panel indicates the ideal $\lambda = \text{throughput}$.

Two-type workload (Nested WAIT). Figure 8 shows the same comparison for the two-type workload (0.7p512d20, 0.3p512d50). The stability ordering is again vLLM first ($\lambda^* \approx 18$), Sarathi second ($\lambda^* \approx 21$), and Nested WAIT last ($\lambda^* \approx 22$). Nested WAIT attains the lowest mean latency at every rate, with the gap to Sarathi growing from $\sim 6\%$ at low load to over 40% near the boundary. The mechanism also differs from the single-type case. Here the two request classes have the same prefill length and differ only in decode length, so the main source of congestion is the accumulation of the long-decode minority in later decode stages. Nested WAIT's segment thresholds keep that downstream inventory from expanding too far, which prevents those delays from propagating to the short-decode majority. The resulting gain is therefore not only at entry: by controlling downstream

buildup, Nested WAIT preserves low latency for the short jobs and enlarges the stable operating region beyond what a single global admission threshold can achieve.

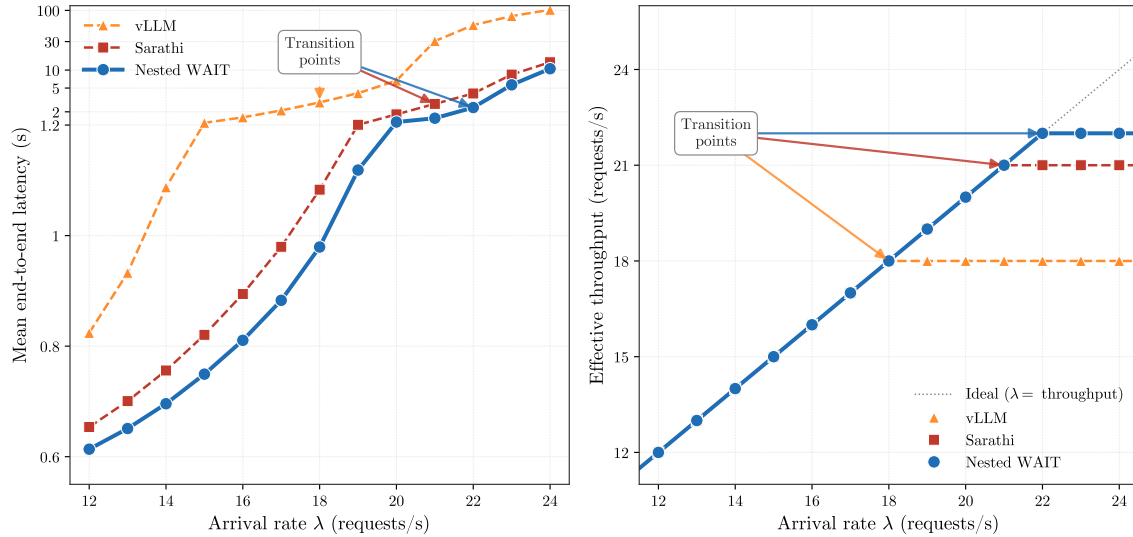


Figure 8 Two-type workload (0.7p512d20, 0.3p512d50) (Poisson arrivals): mean end-to-end latency (left) and effective throughput (right) versus λ . The stability boundaries vLLM ($\lambda^* = 18$) < Sarathi ($\lambda^* = 21$) < Nested WAIT ($\lambda^* = 22$) are visible in both panels.

Long-decode regime. We also study the long-decode workload p512d1000 with Poisson arrivals at $\lambda = 0.5, 1.0, \dots, 5.0$. Relative to the single-type workload p512d20, each request now occupies about 30 $\times$ more KV cache, so all three policies lose stability at much smaller arrival rates. Figure 9 shows that at low rates $\lambda \leq 2.5$ the three policies are nearly indistinguishable, but near the boundary WAIT again has the lowest latency: at $\lambda = 4.5$ it attains 32.3 s versus 41.7 s for Sarathi and 53.8 s for vLLM. The intuition is that long-decode requests stay in the system for many more iterations, so excess admission persists longer and makes it harder to keep arrivals and completions in balance. WAIT performs better in this regime because it keeps the in-system inventory closer to that balance point. The same pattern is already visible at $\lambda = 4.0$, where the baseline-memory sweep gives 26.4 s for WAIT versus 30.2 s for Sarathi. Table 2 reports the accompanying near-capacity eviction diagnostic at the same arrival rate: in the tightened-memory run, Sarathi incurs a 2.88% eviction-induced restart rate, whereas WAIT still avoids eviction altogether. Under the server's last-in-first-out eviction rule, each evicted request loses its partial decode progress and begins again from prefill, so the restart gap is another manifestation of the same imbalance.

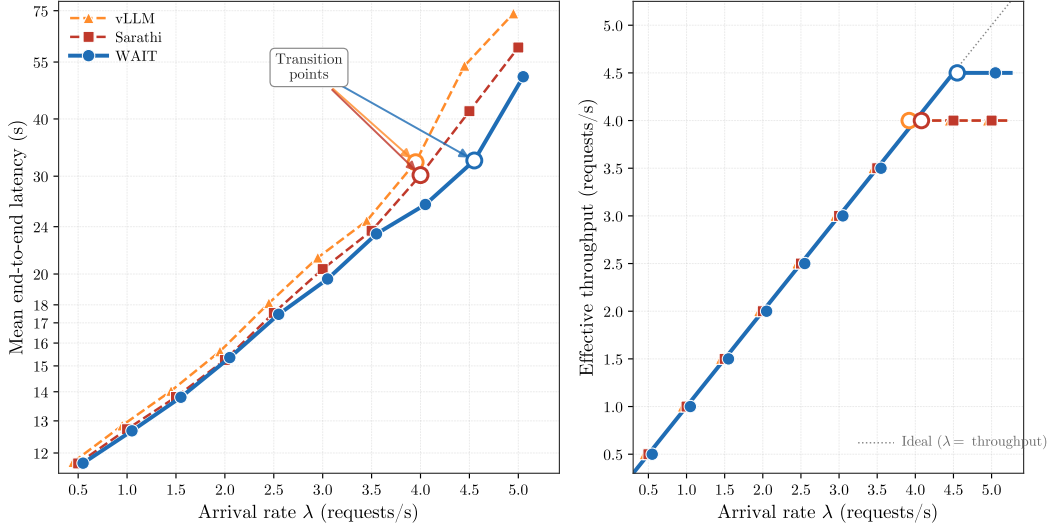


Figure 9 Long-decode workload (p512d1000, prefill 512, decode 1000): mean end-to-end latency (left) and effective throughput (right) versus arrival rate λ . At low rates the three policies are largely comparable, with only small mixed latency differences; the main separation emerges from $\lambda \approx 3.0$ onward. Near the boundary, WAIT attains the lowest latency and sustains the highest throughput.

Metric	Sarathi	WAIT (ours)
Mean end-to-end latency ($\lambda = 4.0$)	30.2 s	26.4 s
Eviction-induced restart rate ($\lambda = 4.0$)	2.88%	0%

Table 2 Near-capacity eviction diagnostic for the long-decode workload p512d1000 at $\lambda = 4.0$. Under tightened memory, Sarathi incurs nonzero eviction-induced restart while WAIT avoids eviction altogether; the same comparison also shows lower latency under WAIT.

Long-horizon validation of the stability boundary. To validate the estimated boundary, we vary the simulation horizon at the two arrival rates nearest Nested WAIT’s transition. Figure 10 plots mean latency on (0.7p512d20, 0.3p512d50) for $\lambda \in \{22, 23\}$ as N increases from 2,000 to 20,000. At $\lambda = 22$, Nested WAIT approaches a finite latency plateau, whereas Sarathi’s mean latency grows approximately linearly in N , consistent with instability. At $\lambda = 23$, both policies are unstable, but Nested WAIT remains 30%–50% below Sarathi at every horizon we consider. Taken together, these comparisons are consistent with Nested WAIT being stable at $\lambda = 22$ and unstable at $\lambda = 23$, so the transition seen in the main figures persists as the horizon lengthens.

6.2. Application to the lmsys-chat-1m Dataset

We next evaluate the policies on lmsys-chat-1m, a real dataset with heterogeneous prompt and response lengths, where decode lengths are unknown at admission. This empirical setting lets

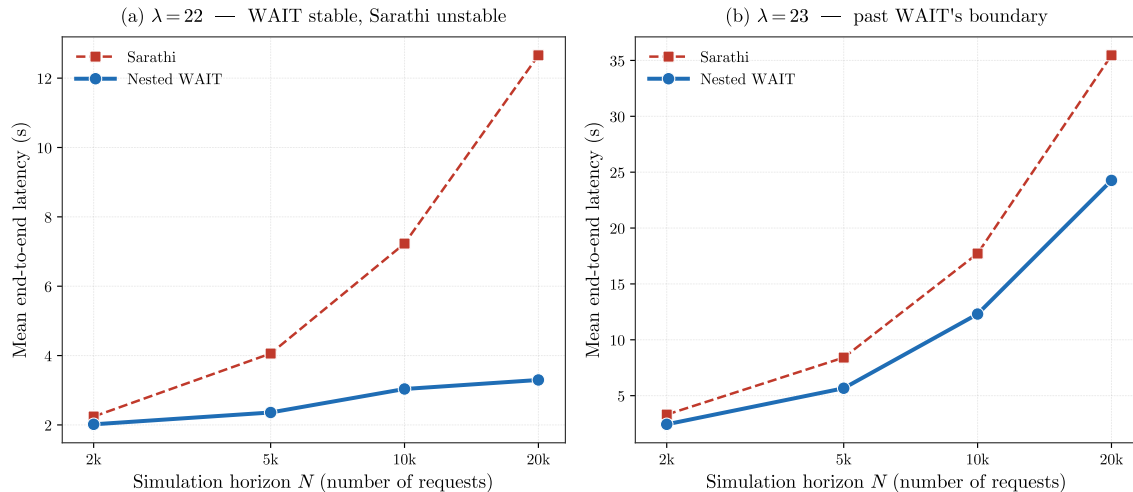


Figure 10 Long-horizon validation on the two-type workload ($0.7p_{512d20}, 0.3p_{512d50}$). **(a)** At $\lambda = 22$, Nested WAIT approaches a finite latency plateau, whereas Sarathi's mean latency grows roughly linearly in N . **(b)** At $\lambda = 23$, both policies are unstable, but Nested WAIT remains substantially below Sarathi at every horizon we consider.

us assess the robustness of Nested WAIT's state-dependent admission rule under a heterogeneous workload distribution, and characterize how the induced capacity allocation varies across decode stages.

Dataset. The dataset contains approximately 1 million human–LLM conversations; we treat each user message as the prompt (prefill) and the corresponding assistant response as the generated output (decode). Figure 11 shows the resulting prefill-length and decode-length distributions. For simulation, we sample 50,000 prompt–response pairs from this dataset. In the sampled dataset, prefill lengths are concentrated near 35 tokens (median 30, 95th percentile 120), while decode lengths range from 1 to 500 tokens (median 95, 95th percentile 380). Short responses remain common: about 22% of prompts decode for at most 50 tokens, whereas about 10% exceed 300. Arrivals follow a Poisson process with rate λ , measured in requests per second. All runs use the same simulator, hardware, and baseline settings as in Section 6.1.

Segment construction for Nested WAIT. Nested WAIT (Algorithm 2) does not classify a request by its eventual decode length. Instead, it tracks realized decode progress and applies the threshold associated with the decode-stage segment currently occupied by the request. A request begins in the head segment and moves to later segments only if it continues decoding, so the policy can control downstream inventory more tightly without requiring any prediction of the final response length.

The main design choice is the segment count m . If m is too small, decode stages with materially different memory implications are pooled under the same threshold. If m is too large, the implied

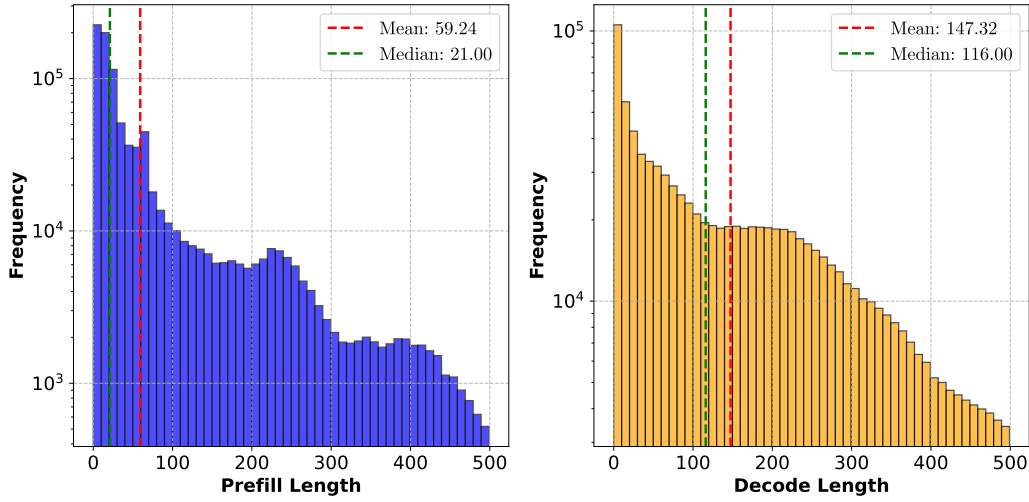


Figure 11 Raw prompt-length distributions in `lmsys-chat-1m`: prefill lengths on the left and decode lengths on the right. The dashed lines mark the mean and median of each distribution.

per-segment in-flight limits become too granular and fragment the batch. Table 3 reports this trade-off across representative values of m .

Threshold allocation across segments. The operator does not set the thresholds $n_1, \dots, n_m$ individually. Instead, the operator chooses only the scalar total in-flight limit tl , and the per-segment thresholds are derived automatically to satisfy the fluid-optimality ratio constraint established for Nested WAIT (Equation (11)):

$$\frac{n_{k+1}}{n_k} > p_k, \quad p_k = \frac{\sum_{j>k} \lambda_j}{\sum_{j \geq k} \lambda_j}, \quad k = 1, \dots, m-1,$$

where p_k is the conditional probability that a request reaching segment k continues into segment $k+1$. The constraint says that thresholds may not shrink faster than the continuation traffic as we move deeper into the decode process; if $n_{k+1}/n_k \leq p_k$, later segments are starved relative to the load that flows into them. We therefore set n_k proportional to the downstream traffic $f_k = \sum_{j \geq k} \lambda_j$ reaching segment k , with a small additive margin so that, after integer rounding against the budget tl , the ratio n_{k+1}/n_k is strictly greater than p_k . In practice, the head segments receive the largest in-flight limits and the tail the smallest. When tl is modest, some late segments may remain dormant. Thus the operator chooses a single scalar rather than m separate thresholds, while the fluid-derived rule determines their relative allocation across segments, consistent with the asymptotic-optimality analysis of Theorem 2.

Number of segments m	Latency at $\lambda = 60$ (s)	Latency at $\lambda = 90$ (s)	Stability boundary λ^*
10	12.6	>40 (unstable)	≈ 55
25	9.2	25.0	≈ 62
50	8.5	23.6	≈ 65
100	9.0	25.2	≈ 62
200	10.8	29.5	≈ 58

Table 3 Effect of the segment count m on Nested WAIT's mean latency and stability boundary on the lmsys workload (Poisson arrivals, fixed tl). Too few segments blur meaningful decode-stage differences, whereas too many fragment the per-segment in-flight limits.

Results. Figure 12 sweeps the arrival rate λ from 10 to 150 in steps of 10. On this real workload, the ordering is less clean than on synthetic. vLLM deteriorates first, with mean latency jumping from 3.2 s at $\lambda = 30$ to 15.4 s at $\lambda = 40$. Sarathi and Nested WAIT remain close through $\lambda = 40$; at $\lambda = 50$, Nested WAIT is slightly above Sarathi (4.4 s versus 3.9 s), which is the only observed arrival rate where Sarathi is lower. From $\lambda = 60$ onward, however, Nested WAIT is consistently below Sarathi: it attains 8.5 s versus 12.2 s at $\lambda = 60$, 27.1 s versus 32.1 s at $\lambda = 100$, and 37.7 s versus 42.2 s at $\lambda = 150$. Thus Nested WAIT wins at 14 of the 15 observed arrival rates against Sarathi, with the gain concentrated in the medium-to-high-load regime rather than uniformly across the whole sweep. These points should be read as outcomes on the observed sweep grid rather than as a formal estimate of the real-workload stability boundary, which would require a denser local sweep and a separate long-horizon validation. The mechanism remains the same as in synthetic. The real workload combines many short responses with a smaller long-decode tail. Nested WAIT's segment-specific thresholds keep that tail from expanding too far into later decode stages, so once the workload enters the higher-load regime it remains materially below the single-policy schedulers.

7. Extensions

The WAIT and Nested WAIT algorithms assume constant arrival rates and a fixed number of segments matching the number of prompt types. In practice, however, LLM inference systems face two additional challenges: (i) **time-varying workloads**, where arrival rates fluctuate throughout the day (e.g., peak hours vs. off-hours); (ii) **numerous prompt types**, where maintaining separate thresholds for hundreds of output lengths becomes infeasible. We address these challenges through two extensions that preserve the core algorithmic principles—eviction prevention and approaching load balance—while adapting to realistic deployment scenarios.

First, we extend Nested WAIT to handle time-varying arrival rates λ_j^t by replacing constant-rate threshold conditions with accumulated arrival constraints over each iteration window. The coupling-

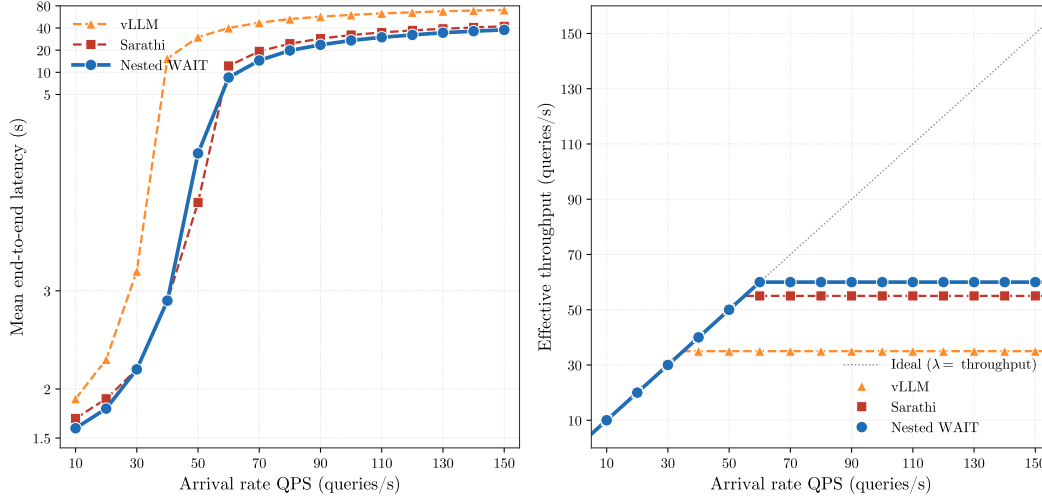


Figure 12 Real workload (lmsys-chat-1m, 50,000 prompts): mean end-to-end latency (left) and effective throughput (right) versus arrival rate λ . On the observed grid $\lambda \in \{10, 20, \dots, 150\}$, Nested WAIT is close to Sarathi at low load, is slightly higher only at $\lambda = 50$, and remains lower from $\lambda = 60$ onward.

based proof technique extends naturally, requiring only that threshold conditions hold at every time instant (Section 7.1). Second, we introduce a **segment-wise design** that groups m prompt types into $L \leq m$ segments, reducing algorithmic complexity while maintaining asymptotic optimality (Section 7.2). Numerical validation confirms that a moderate segment count (e.g., $L = 10$) suffices in practice, with memory overhead remaining logarithmic.

7.1. Time-Varying Arrival Rates

Time-varying arrival rates capture daily patterns in real LLM services: morning traffic surges, afternoon lulls, evening peaks. The threshold mechanism remains valid as long as the accumulated arrivals during each iteration window stay below the completion capacity. Formally, we replace constant rates λ_j with time-dependent rates λ_j^t , assumed continuous and uniformly bounded by $\lambda_{\max} < \infty$ to prevent unbounded bursts.

We analyze the asymptotic optimality of the Nested WAIT algorithm under time-varying arrival rates, denoted λ_j^t for prompt type $j \in [m]$ at time $t \in [0, T]$.

We define the accumulated arrivals of type j over the interval $[t_1, t_2]$ as $\lambda_j[t_1, t_2] := \int_{t_1}^{t_2} \lambda_j^t dt$. Recall from Equation (10) that $\Delta T_{[1,2,\dots,m]}(n_1, \dots, n_m)$ represents the per-iteration time cost when processing exactly n_k prompts at each stage in segment k , for all $k \in [m]$, as in (11), where n_k is the threshold defined in Algorithm 2.

For the Nested WAIT policy defined by $\pi = [n_1, \dots, n_m]$, we consider the following conditions for all $t \in [0, T]$, $0 < \Delta t \leq \Delta T_{[1,2,\dots,m]}(n_1, \dots, n_m)$ and $k = 1, 2, \dots, m-1$, analogous to (11):

$$\begin{aligned} & \sum_{j=1}^m \lambda_j[t, t + \Delta t] \\ & \leq \sum_{j=1}^m \lambda_j[t, t + \Delta T_{[1,2,\dots,m]}(n_1, \dots, n_m)] < n_1, \\ & n_{k+1}/n_k > p_k[t, t + \Delta t], \end{aligned} \tag{14}$$

where $p_k[t, t + \Delta t] = (\sum_{j=k+1}^m \lambda_j[t, t + \Delta t]) / (\sum_{j=k}^m \lambda_j[t, t + \Delta t])$.

We define the total number of iterations over time horizon $[0, T]$ as B . Therefore, we have $T \geq \sum_{b=1}^B \Delta T_{b\text{-th Batch}} \geq B \cdot \min \Delta T_{\text{Batch}}$, where $\Delta T_{b\text{-th Batch}}$ denotes the time cost of the b -th iteration and $\Delta T_{\min} = \min_b \{\Delta T_{\text{Batch}}\}$. The following theorem extends Theorem 2 to time-varying arrival rates.

THEOREM 3. *Suppose the thresholds $\pi = [n_1, \dots, n_m]$ satisfy (14), and the memory $M^{(\zeta, \pi)}$ fulfills:*

$$\begin{aligned} M^{(\zeta, \pi)} & \geq M^\pi + \sum_{k=2}^m (l + l'_{k-1}) \left(n_k + \theta_k^{-1} \ln \left(\frac{m\zeta B}{\delta} \right) \right) \\ & = O \left(2M^\pi + \sum_{k=1}^m \theta_k^{-1} (l + l'_{k-1}) \ln \left(\frac{m\zeta B}{\delta} \right) \right), \end{aligned}$$

where the memory usage M^π is defined as (12) and θ_k is given in (20) with the lower bound in Equation (21). The following asymptotic bounds hold:

$$\begin{aligned} \text{Throughput}^* - \mathbb{E} [\text{Throughput}^{(\zeta, \pi)}] & = O \left((\zeta T)^{-1} \right), \\ \mathbb{E} [\text{Latency}^{(\zeta, \pi)}], \mathbb{E} [\text{TFT}^{(\zeta, \pi)}] & = O(1). \end{aligned}$$

Additionally, the memory usage remains below $M^{\zeta, \pi}$ with probability at least $1 - \delta$ over the time horizon $[0, T]$.

The extension to time-varying rates preserves the dimensionality reduction structure from Theorem 2. The key adaptation is that threshold conditions (14) must hold at every time instant $t \in [0, T]$, replacing constant rates λ_j with accumulated arrivals $\lambda_j[t, t + \Delta t]$ over each iteration window. Because arrival rates are continuous and bounded, the coupling construction remains valid: prompts transitioning from segment k to $k + 1$ still couple to a Poisson process, now with time-varying rate $\sum_{j=k+1}^m \lambda_j^t$. The Lindley recursion and Doob's maximal inequality apply identically, bounding queue lengths with the same logarithmic overhead. Appendix D provides the detailed proof.

7.2. Segment Design

When LLM systems handle hundreds of distinct output lengths (e.g., $m = 500$ in the LMSYS dataset [Zheng et al. \(2023\)](#)), maintaining individual thresholds for each type becomes impractical: the algorithm must track queue lengths separately for every output length, increasing memory overhead and implementation complexity. The **segment-wise design** addresses this by grouping similar output lengths into $L \leq m$ segments, where each segment k aggregates types with comparable decode lengths. This reduces algorithmic complexity from $O(m)$ to $O(L)$ while preserving asymptotic optimality—prompts still classify themselves on-the-fly, but now at segment boundaries rather than individual type boundaries.

We assume constant arrival rates for clarity, though the approach extends to time-varying rates as discussed in Section 7.1. Consider m prompt types with decode lengths $l'_1 < \dots < l'_m$ and arrival rates $\lambda_1, \lambda_2, \dots, \lambda_m$. Our objective is to merge these into L segments, where $L \leq m$. For notational convenience, let $\Delta L := m/L$. We partition the m types into L segments as follows: types 1 to ΔL , types $\Delta L + 1$ to $2\Delta L$, $\dots$, types $(L-1)\Delta L + 1$ to m .

For each segment $k \in \{1, \dots, L\}$, the total arrival rate is defined as $\lambda'_k = \sum_{(k-1)\Delta L + 1 \leq j \leq k\Delta L} \lambda_j$. Additionally, we define $\lambda'(k_1 \rightarrow k_2) := \sum_{r=k_1}^{k_2} \lambda'_r$ for $1 \leq k_1 \leq k_2 \leq L$.

Given a policy $\pi = [n_1, \dots, n_L]$ with threshold n_k for the k -th segment in Algorithm 2, we formulate a linear system analogous to Equation (11):

$$\begin{aligned} \Delta T_{[1:L]}(n_{1:L}) &\leq \frac{n_1}{\sum_{r=1}^L \lambda'_r}, \\ \frac{n_{k+1}}{n_k} &> p_k, \quad \forall k < L, \end{aligned} \tag{15}$$

where $p_k = \frac{\lambda'(k+1 \rightarrow L)}{\lambda'(k \rightarrow L)}$, $\Delta T_{[1,\dots,L]}(n_1, \dots, n_L) = d_0 + d_1 M^\pi(n_1, \dots, n_L)$, and the memory M^π running with n_k at each stage in segment k is given by:

$$M^\pi(n_1, \dots, n_L) = \sum_{k=1}^L n_k \left(l + \frac{k}{2} \Delta L \right) \Delta L.$$

As in Section 7.1, we define the total number of iterations over time horizon $[0, T]$ as B , satisfying $T \geq \sum_{b=1}^B \Delta T_{b\text{-th Batch}} \geq B \cdot \min \Delta T_{\text{Batch}}$.

The segment-wise design achieves two practical goals: (i) **robustness** to varying or uncertain type distributions—if actual output lengths differ slightly from predictions, grouping into segments

absorbs the variation; (ii) **simplicity** in deployment—maintaining $L = 10$ segments is far easier than tracking $m = 500$ individual types. The theoretical guarantee shows that this simplification incurs no asymptotic cost: throughput remains $O(1/(\zeta T))$ away from optimal, with the same logarithmic safety buffer.

THEOREM 4. *For any fixed number of segments L and thresholds $\pi = [n_1, \dots, n_L]$ satisfying Equation (15), if the memory capacity $M^{(\zeta, \pi)}$ satisfies:*

$$\begin{aligned} M^{(\zeta, \pi)} &\geq M^\pi + \sum_{k=2}^L (l + l'_{k-1}) n_k \\ &\quad + \sum_{k=2}^L (l + l'_{k-1}) \theta_k^{-1} \ln \left(\frac{m \zeta B}{\delta} \right) \\ &= O \left(2M^\pi + \sum_{k=1}^L (l + l'_{k-1}) \theta_k^{-1} \ln \left(\frac{m \zeta B}{\delta} \right) \right), \end{aligned} \tag{16}$$

where θ_k is given by Equation (19) with lower bound $8(n_k - n_{k-1} p_k)/n_k$. The performance of Nested WAIT satisfies:

$$\begin{aligned} \text{Throughput}^* - \mathbb{E} [\text{Throughput}^{(\zeta, \pi)}] &= O \left((\zeta T)^{-1} \right), \\ \mathbb{E} [\text{Latency}^{(\zeta, \pi)}] &= O(1), \\ \mathbb{E} [\text{TTFT}^{(\zeta, \pi)}] &= O(1). \end{aligned}$$

Moreover, memory is not exceeded with probability at least $1 - \delta$ over the total batch counts $b \in \{1, \dots, B\}$ as well as the time horizon $t \in [0, T]$.

Clustering m types into L segments preserves the core threshold structure from Theorem 2. The key observation is that aggregation commutes with the coupling argument: individual arrival rates $(\lambda_1, \dots, \lambda_m)$ are replaced by aggregated rates $(\lambda'_1, \dots, \lambda'_L)$ where $\lambda'_k = \sum_{j \in \text{segment } k} \lambda_j$, but the delayed arrival process for segment k still couples to a Poisson process with rate $\sum_{r=k+1}^L \lambda'_r$. The Lindley recursion and Doob's maximal inequality apply identically at the segment level, yielding the same logarithmic overhead in Equation (16). The proof follows the structure of Theorem 2 in Appendix C.

We validate the segment-wise design numerically using the LMSYS dataset Zheng et al. (2023) (Section 6) with $m = 500$ prompt types, decode lengths from 1 to 500, and fixed prefill length 62.

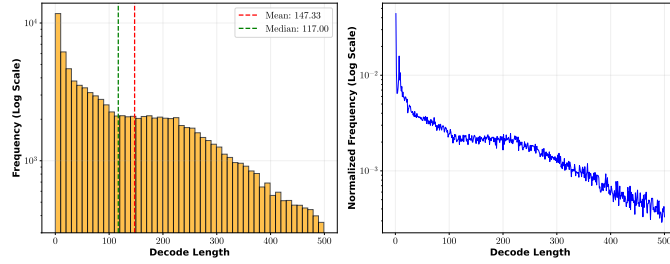


Figure 13 Arrival rate construction from real data, with rates proportional to normalized frequencies.

The distribution of arrival rates, shown in Figure 13, is proportional to normalized frequencies from real user traffic.

We analyze the three components of the memory bound in Equation (16): (1) the peak batch memory usage $\sum_{k=1}^L n_k \left(l + \frac{k}{2} \Delta \right) \Delta$, (2) the queue length bound $\sum_{k=2}^L n_k (l + l'_{k-1})$, and (3) the high-probability queue length bound $\sum_{k=2}^L \theta_k^{-1} (l + l'_{k-1}) \ln \left(\frac{(L-1)(T+1)}{\delta} \right)$.

Figures 14, 15, and 16 plot these three memory components (y-axis) as functions of the number of segments L (x-axis), under varying arrival rates, time horizons T , and confidence levels δ . Each panel shows: Term 1 (blue circles) representing the fluid equilibrium memory from batching, Term 2 (green squares) representing the deterministic queue bound, Term 3 (red triangles) representing the stochastic safety buffer with logarithmic dependence on T and δ , and their sum (black crosses) representing the total memory requirement $M^{(\zeta, \pi)}$. Three key observations emerge: (i) Term 1 (fluid equilibrium memory) dominates Terms 2 and 3 across all parameter settings, confirming that the safety buffer remains modest relative to the base batching requirement. (ii) The total memory (black line) follows a **U-shaped curve** with respect to L : too few segments waste memory by overprovisioning for heterogeneous types (high Term 1 at small L), while too many segments increase overhead from maintaining separate queues. The minimum occurs around $L = 5$ -10, validating the choice of $L = 10$ in Section 6. (iii) Term 3 exhibits smooth growth as δ decreases (comparing left vs. right panels in Figures 14 and 15) and as T increases (comparing Figure 16), consistent with the logarithmic bound in the theoretical analysis.

These extensions demonstrate that the core threshold-based design, eviction prevention through controlled admission, remains robust to realistic deployment conditions. Time-varying arrival rates and segment-wise grouping introduce no fundamental barrier; the coupling-based analysis extends naturally with logarithmic overhead. Section 6 validates these theoretical guarantees through

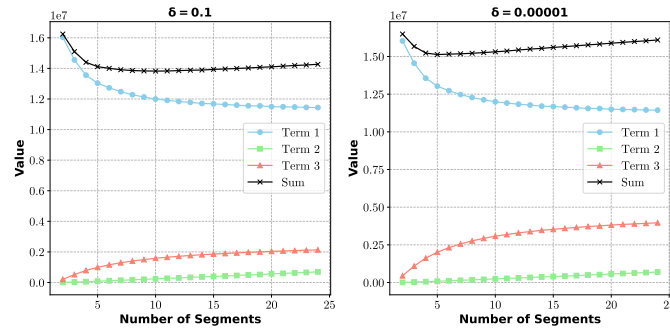


Figure 14 Memory usage proportions under low arrival rate (total rate = 50, $T = 200$). Left: $\delta = 0.1$; Right: $\delta = 10^{-5}$.

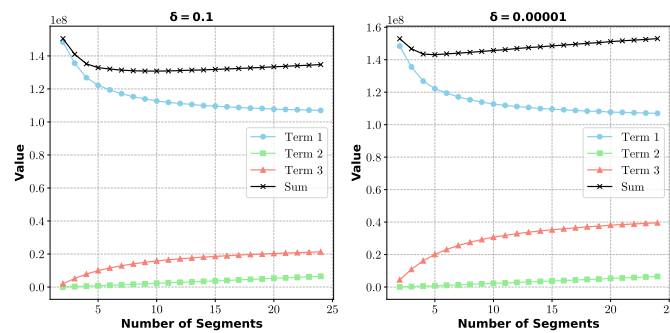


Figure 15 Memory usage proportions under high arrival rate (total rate = 500, $T = 200$). Left: $\delta = 0.1$; Right: $\delta = 10^{-5}$.

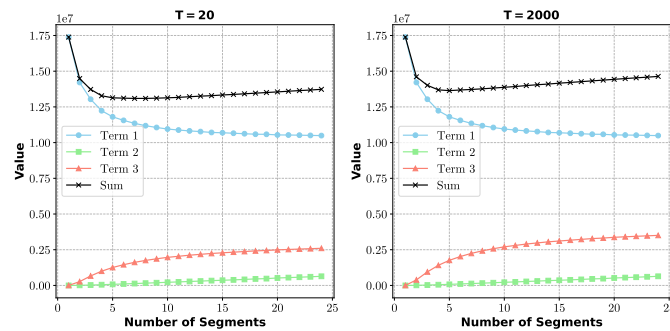


Figure 16 Memory usage proportions with varying time horizons (total rate = 50, $\delta = 0.001$). Left: $T = 20$; Right: $T = 2000$.

experiments on real workloads, showing that Nested WAIT with $L = 10$ segments matches the performance of the full m -type algorithm while significantly reducing implementation complexity.

8. Conclusion and Future Directions

This paper studies online scheduling for LLM inference, a setting in which each request's memory consumption grows during decoding and exceeding capacity triggers evictions that can destabilize a system whose capacity is theoretically sufficient for stability. We develop a multi-stage online

scheduling model that captures these dynamics and analyze a fluid approximation whose equilibrium identifies the stability region and the memory level M^* required for stability at its boundary. Guided by this fluid benchmark, we design the (Nested) WAIT algorithm, a threshold-based admission rule that keeps the stochastic system close to the fluid equilibrium. The threshold mechanism performs a dimensionality reduction: it decouples interactions across prompt types and replaces the continuously growing KV cache dynamics with a discrete-time per-type recursion. Under this reduction, we establish asymptotic optimality for the effective throughput, the completion rate net of eviction, even when output lengths are unknown at arrival. Because the algorithms operate near the fluid equilibrium and prevent the eviction cascades that destabilize memory-reactive baselines, they both enlarge the stability region and reduce mean end-to-end latency across the underloaded, near-capacity, and overloaded regimes. Numerical experiments on Llama-2-7B with an NVIDIA A100 GPU confirm both effects relative to vLLM and Sarathi.

Several questions remain open. First, our threshold design relies on knowledge of the prompt-type arrival distribution; developing distribution-free variants, or versions that adapt online under bounded misspecification, would extend the algorithm's applicability. Second, practical deployments face arrival rates that drift over time and occasionally surge. Theorem 3 handles bounded rate variation by periodically re-solving the fluid equilibrium, but sharper volatility, such as demand spikes that temporarily exceed nominal capacity, calls for further analysis; two concrete questions are how fast the thresholds can re-tune while preserving asymptotic optimality, and how transient surges interact with the safety buffer in Nested WAIT.

Finally, extending the analysis to multi-GPU deployments introduces inter-device communication cost and a choice of parallelization strategy (data, tensor, or pipeline). This regime is especially relevant for mixture-of-experts (MoE) architectures (Shazeer et al. 2017, Fedus et al. 2022, Jiang et al. 2024), in which token-level routing distributes experts across GPUs and creates skewed memory and compute load that our single-GPU analysis does not capture. A unified framework that models inter-device communication inside the iteration-time model, and coordinates admission thresholds across GPUs in response to the routing distribution, is a natural open direction.

Endnotes

References

- Agrawal A, Kedia N, Mohan J, Panwar A, Kwatra N, Gulavani B, Ramjee R, Tumanov A (2024a) Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6:351–366.
- Agrawal A, Kedia N, Panwar A, Mohan J, Kwatra N, Gulavani B, Tumanov A, Ramjee R (2024b) Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 117–134.
- Agrawal A, Panwar A, Mohan J, Kwatra N, Gulavani BS, Ramjee R (2023) Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* .
- Aminabadi RY, Rajbhandari S, Awan AA, Li C, Li D, Zheng E, Ruwase O, Smith S, Zhang M, Rasley J, He Y (2022) Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale. *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–15 (IEEE).
- Anthropic (2023) Claude. <https://claude.ai>.
- Anthropic (2025) Claude 3.7 sonnet. URL <https://www.anthropic.com/claude/sonnet>.
- Asmussen S (2003) *Applied probability and queues*, volume 2 (Springer).
- Bai J, Bai S, Chu Y, Cui Z, Dang K, Deng X, Fan Y, Ge W, Han Y, Huang F, et al. (2023) Qwen technical report. *arXiv preprint arXiv:2309.16609* .
- Balkanski E, Rubinstein A, Singer Y (2016) The power of optimization from samples. *Advances in Neural Information Processing Systems* 29.
- Bäuerle N (2002) Optimal control of queueing networks: An approach via fluid models. *Advances in Applied Probability* 34(2):313–328.
- Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, et al. (2020) Language models are few-shot learners. *Advances in neural information processing systems* 33:1877–1901.
- Chen Z, Ye Y, Zhou Z (2025) Adaptively robust llm inference optimization under prediction uncertainty. *arXiv preprint arXiv:2508.14544* .
- Cole R, Roughgarden T (2014) The sample complexity of revenue maximization. *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, 243–252.
- DeepSeek-AI (2024) Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. URL <https://arxiv.org/abs/2405.04434>.
- DeepSeek-AI (2025) Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. URL <https://arxiv.org/abs/2501.12948>.
- Desislavov R, Martínez-Fernández S, Franch X (2021) Compute and energy consumption trends in deep learning inference. *arXiv preprint arXiv:2109.02132* .

- Devanur NR, Hayes TP (2009) The adwords problem: online keyword matching with budgeted bidders under random permutations. *Proceedings of the 10th ACM conference on Electronic commerce*, 71–78.
- Devlin J, Chang MW, Lee K, Toutanova K (2019) Bert: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 4171–4186.
- Durrett R (2019) *Probability: theory and examples*, volume 49 (Cambridge university press).
- Fedus W, Zoph B, Shazeer N (2022) Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23(120):1–39.
- Fu Y, Zhu S, Su R, Qiao A, Stoica I, Zhang H (2025) Efficient llm scheduling by learning to rank. *Advances in Neural Information Processing Systems* 37:59006–59029.
- García-Martín E, Rodrigues CF, Riley G, Grahn H (2019) Estimation of energy consumption in machine learning. *Journal of Parallel and Distributed Computing* 134:75–88.
- GitHub (2022) Copilot. URL <https://github.com/features/copilot>.
- Google (2023) Bard. <https://bard.google.com>.
- Hooper C, Kim S, Mohammadzadeh H, Mahoney MW, Shao S, Keutzer K, Gholami A (2025) Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems* 37:1270–1303.
- Im S, Moseley B (2013) Online batch scheduling for flow objectives. *Proceedings of the twenty-fifth annual ACM symposium on Parallelism in algorithms and architectures*, 102–104.
- Jaillet P, Jiang J, Mellou K, Molinaro M, Podimata C, Zhou Z (2025) Online scheduling for llm inference with kv cache constraints. URL <https://arxiv.org/abs/2502.07115>.
- Jiang AQ, Sablayrolles A, Roux A, Mensch A, Savary B, Bamford C, Chaplot DS, Casas Ddl, Hanna EB, Bressand F, et al. (2024) Mixtral of experts. URL <https://arxiv.org/abs/2401.04088>.
- Kang H, Zhang Q, Kundu S, Jeong G, Liu Z, Krishna T, Zhao T (2024) Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm. *arXiv e-prints* arXiv–2403.
- Kaplan J, McCandlish S, Henighan T, Brown TB, Chess B, Child R, Gray S, Radford A, Wu J, Amodei D (2020) Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Kingman JFC (1962) On queues in heavy traffic. *Journal of the Royal Statistical Society: Series B (Methodological)* 24(2):383–392.
- Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, Gonzalez J, Zhang H, Stoica I (2023) Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626.
- Lattanzi S, Lavastida T, Moseley B, Vassilvitskii S (2020) Online scheduling via learned weights. *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1859–1877 (SIAM).

- Li R, Pal S, Pullu VN, Ryoo JH, John LK, Yadwadkar NJ (2025a) Oneiros: Kv cache optimization through parameter remapping for multi-tenant llm serving. *arXiv preprint arXiv:2507.11507* To appear in ACM SoCC 2025.
- Li W, Wang L, Chai X, Yuan H (2020) Online batch scheduling of simple linear deteriorating jobs with incompatible families. *Mathematics* 8(2):170.
- Li Y, Dai J, Peng T (2025b) Throughput-optimal scheduling algorithms for llm inference and ai agents. URL <https://arxiv.org/abs/2504.07347>, accessed: 2025-04-11.
- Liu P, Lu X (2015) Online unbounded batch scheduling on parallel machines with delivery times. *Journal of Combinatorial Optimization* 29:228–236.
- Liu Y, Whitt W (2011) A network of time-varying many-server fluid queues with customer abandonment. *Operations research* 59(4):835–846.
- Lucier B, Menache I, Naor J, Yaniv J (2013) Efficient online scheduling for deadline-sensitive jobs. *Proceedings of the twenty-fifth annual ACM symposium on Parallelism in algorithms and architectures*, 305–314.
- Maglaras C (2000) Discrete-review policies for scheduling stochastic networks: Trajectory tracking and fluid-scale asymptotic optimality. *The Annals of Applied Probability* 10(3):897–929.
- Mandelbaum A, Massey WA, Reiman MI (1998) Strong approximations for markovian service networks. *Queueing Systems* 30(1):149–201.
- Microsoft (2023) Bing ai. <https://www.bing.com/chat>.
- OpenAI (2024) Gpt-4 technical report. URL <https://arxiv.org/abs/2303.08774>.
- Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, Zhang C, Agarwal S, Slama K, Ray A, et al. (2022) Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35:27730–27744.
- Patel D (2023) Peeling The Onion's Layers: Large Language Models' Cost, Context, and Feature Breakdown. Semianalysis blog, URL <https://semianalysis.com/2023/02/13/peeling-the-onions-layers-large-language/>, accessed: [Insert Date Here].
- Patel P, Choukse E, Zhang C, Shah A, Goiri Í, Maleki S, Bianchini R (2024) Splitwise: Efficient generative llm inference using phase splitting. in 2024 acm/ieee 51st annual international symposium on computer architecture (isca). *IEEE Computer Society, Los Alamitos, CA, USA* 118–132.
- Patterson D, Gonzalez J, Le Q, Liang C, Munguia LM, Rothchild D, So D, Texier M, Dean J (2021) Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*.
- Pope R, Douglas S, Chowdhery A, Devlin J, Bradbury J, Heek J, Xiao K, Agrawal S, Dean J (2023) Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5:606–624.
- Shazeer N, Mirhoseini A, Maziarz K, Davis A, Le Q, Hinton G, Dean J (2017) Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations (ICLR)*.

- Sheng Y, Zheng L, Yuan B, Li Z, Ryabinin M, Chen B, Liang P, Ré C, Stoica I, Zhang C (2023) Flexgen: High-throughput generative inference of large language models with a single gpu. *International Conference on Machine Learning*, 31094–31116 (PMLR).
- Strait P (1974) On the maximum and minimum of partial sums of random variables. *Pacific Journal of Mathematics* 52(2):585–593.
- Strubell E, Ganesh A, McCallum A (2019) Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243*.
- Tay Y, Dehghani M, Bahri D, Metzler D (2022) Efficient transformers: A survey. *ACM Computing Surveys* 55(6):1–28.
- Touvron H, Lavril T, Izacard G, Martinet X, Lachaux MA, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, et al. (2023) Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Vee E, Vassilvitskii S, Shanmugasundaram J (2010) Optimal online assignment with forecasts. *Proceedings of the 11th ACM conference on Electronic commerce*, 109–118.
- vLLM Team (2025) vllm v1: A major upgrade to vllm’s core architecture. https://docs.vllm.ai/en/latest/getting_started/v1_user_guide.html, blog post, January 27, 2025. ”In vLLM V1, the default preemption mode is RECOMPUTE rather than SWAP, as recomputation has lower overhead in the V1 architecture”.
- Wang M, Ye Y, Zhou Z (2025) Llm serving optimization with variable prefill and decode lengths. *arXiv preprint arXiv:2508.06133*.
- Wei J, Tay Y, Bommasani R, Raffel C, Zoph B, Borgeaud S, Yogatama D, Bosma M, Zhou D, Metzler D, et al. (2022a) Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*.
- Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D, et al. (2022b) Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35:24824–24837.
- Wu CJ, Raghavendra R, Gupta U, Acun B, Ardalani N, Maeng K, Chang G, Aga F, Huang J, Bai C, et al. (2022) Sustainable ai: Environmental implications, challenges and opportunities. *Proceedings of Machine Learning and Systems* 4:795–813.
- Yu GI, Jeong JS, Kim GW, Kim S, Chun BG (2022) Orca: A distributed serving system for {Transformer-Based} generative models. *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 521–538.
- Zheng L, Chiang WL, Sheng Y, Li T, Zhuang S, Wu Z, Zhuang Y, Li Z, Lin Z, Xing EP, et al. (2023) Lmsys-chat-1m: A large-scale real-world llm conversation dataset. *arXiv preprint arXiv:2309.11998*.
- Zhong Y, Liu S, Chen J, Hu J, Zhu Y, Liu X, Jin X, Zhang H (2024) {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 193–210.

Appendix A: Proofs of Propositions

A.1. Proof of Proposition 1

Proof Outline. We show that when $\lambda_j d_1 (l'_j + 1)(l_j + l'_j/2) \geq 1$, arrivals consistently exceed completions regardless of batch size, causing queue length to grow unboundedly.

Consider a system with only type- j prompts and suppose n_j prompts are in the batch. If equilibrium were to hold, prompts would be uniformly distributed across stages $0, 1, \dots, l'_j$, with $n_j/(l'_j + 1)$ prompts at each stage. The total memory consumption is:

$$M_j = \frac{n_j}{l'_j + 1} \sum_{s=0}^{l'_j} (l_j + s) = n_j \left(l_j + \frac{l'_j}{2} \right).$$

The iteration time to process this batch is:

$$\tau_j = d_0 + d_1 M_j = d_0 + d_1 n_j \left(l_j + \frac{l'_j}{2} \right).$$

During this iteration, the expected number of arriving prompts is:

$$\text{Arrivals}_j = \tau_j \cdot \lambda_j = d_0 \lambda_j + \lambda_j d_1 n_j \left(l_j + \frac{l'_j}{2} \right),$$

while the number of completions is $n_j/(l'_j + 1)$ (prompts at stage l'_j complete). The arrival-completion gap satisfies:

$$\text{Arrivals}_j - \frac{n_j}{l'_j + 1} = d_0 \lambda_j + n_j \left[\lambda_j d_1 \left(l_j + \frac{l'_j}{2} \right) - \frac{1}{l'_j + 1} \right] \geq d_0 \lambda_j > 0,$$

where the inequality follows from $\lambda_j d_1 (l'_j + 1)(l_j + l'_j/2) \geq 1$.

Since this gap is positive for all $n_j > 0$, the queue length after N iterations satisfies:

$$\text{Queue Length}(N) \geq d_0 \lambda_j \cdot N.$$

The latency for a prompt arriving at iteration N grows as $\Omega(N)$, establishing instability.

A.2. Proof of Proposition 2

Proof Outline. The expected throughput of any online policy cannot exceed the fluid throughput **Throughput***, since throughput is bounded by the arrival rate of output tokens.

In the fluid model, the equilibrium parameters are:

$$\Delta T = \frac{d_0}{1 - d_1 \sum_{j=1}^m \lambda_j (l'_j + 1)(l_j + l'_j/2)}, \quad n_j^* = \Delta T \cdot \lambda_j (l'_j + 1),$$

$$M^* = \Delta T \cdot \sum_{j=1}^m \lambda_j (l'_j + 1)(l_j + l'_j/2), \quad \text{Throughput}^* = \sum_{j=1}^m \lambda_j l'_j.$$

In any stochastic system, throughput is counted only after prompts complete. Over time horizon $[0, T]$, let Arrivals_j^T denote the total arrivals of type- j prompts. The total output tokens generated cannot exceed the total output length of all arrivals:

$$\text{Throughput}^{(\pi, T)} \leq \sum_{j=1}^m \text{Arrivals}_j^T \cdot l'_j.$$

Taking expectations and dividing by T :

$$\mathbb{E}[\text{Throughput}^{(\pi, T)}] \leq \sum_{j=1}^m \mathbb{E} \left[\frac{\text{Arrivals}_j^T}{T} \right] \cdot l'_j = \sum_{j=1}^m \lambda_j l'_j = \text{Throughput}^*.$$

A.3. Proof of Proposition 3

Proof Outline. We construct a two-type example where FCFS suffers from cascading evictions even when capacity equals equilibrium memory M^* , leading to $\Omega(1)$ throughput gap.

Consider two prompt types: type 1 with $(l_1, l'_1, \lambda_1) = (1, 1, 1)$ and type 2 with $(l_2, l'_2, \lambda_2) = (1, 2, 1)$.

Fluid Equilibrium. The equilibrium equations $d_0 + d_1(1.5n_1 + 2n_2) = n_1/2 = n_2/3$ yield:

$$K = \frac{d_0}{1 - 9d_1}, \quad n_1^* = 2K, \quad n_2^* = 3K, \quad M^* = 9K.$$

Setting $K = 1$ (without loss of generality), we have $n_1^* = 2, n_2^* = 3, M^* = 9$.

Equilibrium Batch Composition. In equilibrium, the batch contains:

- Type 1: 1 prompt at stage 0 (memory = 1), 1 prompt at stage 1 (memory = 2, completing)
- Type 2: 1 prompt at stage 0 (memory = 1), 1 at stage 1 (memory = 2), 1 at stage 2 (memory = 3, completing)

Overflow Probability. Let (X_1, X_2) denote arrivals during one iteration, where $X_j \sim \text{Poisson}(1)$. The combinations that do not cause memory overflow within two iterations are: $(0, 0), (1, 0), (0, 1), (1, 1), (2, 0)$. All other combinations lead to overflow. The overflow probability is:

$$p = 1 - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{2e^2} = 1 - \frac{9}{2e^2} \approx 0.391.$$

Since overflow probability is bounded away from zero, FCFS experiences eviction with constant probability at each iteration. This leads to queue length growth $\Omega(t)$ and throughput gap $\Omega(1)$.

A.4. Proof of Proposition 5

Proof Outline. Using the same two-type example, we show that when output lengths are unknown until completion, any online policy faces either memory underutilization or constant overflow probability.

Consider the same setup as Proposition 3: type 1 with $(l_1, l'_1, \lambda_1) = (1, 1, 1)$ and type 2 with $(l_2, l'_2, \lambda_2) = (1, 2, 1)$. The fluid equilibrium gives $M^* = 9$.

The key constraint is that prompt types cannot be distinguished until a type-2 prompt enters stage 2 (generates its second output token). At stage 0 and stage 1, both types appear identical to the scheduler.

Consider any admissible batching policy. For each possible batch configuration, one of the following must hold:

1. **Memory underutilization:** The batch leaves significant memory unused, causing expected arrivals to exceed completions during the iteration.
2. **Overflow risk:** The batch fully utilizes memory, but faces constant probability $\rho > 0$ of overflow when arrivals exceed expectations.

Since throughput is counted only upon prompt completion, and the fluid throughput equals expected arrivals' output length per unit time, any policy that either underutilizes memory or experiences overflow incurs a throughput gap of at least $\Omega(\rho)$ compared to **Throughput***.

Appendix B: Proof of Theorem 1

Proof Outline. This proof establishes asymptotic optimality of the WAIT algorithm when output lengths are known. The structure proceeds in two parts:

1. **Single-type case:** We construct a coupled dominating process using the Lindley recursion and apply Kingman's bound for queue length expectations.
2. **Multiple-type case:** We extend to m types by applying the coupling construction type-by-type, leveraging that the fixed threshold n_j decouples inter-type dynamics.

Key techniques include: Lindley recursion for queue length bounds (Asmussen 2003, Proposition 6.3), coupling construction to dominate the original process, Kingman's bound for negative drift processes (Kingman 1962), and standard random walk results (Strait 1974).

Throughout this appendix, we use b to denote the *batch index* (the count of completed batch iterations), which is distinct from the stage index $s \in \{0, 1, \dots, l'_j\}$ used in the main text for decode

stages. We use t for continuous time and T (or B) for the total number of batches over the time horizon.

We first consider the single-type case, then extend to multiple types.

B.0.1. Single-Type Case Consider a single prompt type with threshold n . For general scheduling algorithms, analyzing this system in continuous time is challenging: KV cache grows dynamically during decode, arrivals are stochastic, and eviction risks complicate the dynamics.

The WAIT policy's *threshold mechanism achieves a dimensionality reduction*. By waiting until exactly n prompts accumulate before initiating a batch, the algorithm enforces a fixed batch size, which ensures constant processing time ΔT per batch. This decouples the continuous-time memory-constrained dynamics into a tractable discrete-time process: a memory-constrained queueing system becomes a simple random walk. The reduction is enabled by the algorithm design and is what makes the subsequent drift analysis apply.

Specifically, let $\tilde{\lambda}$ denote the continuous-time arrival rate. The WAIT policy's fixed batch size implies that arrivals during each batch follow $\text{Poisson}(\tilde{\lambda} \cdot \Delta T)$. Defining $\lambda = \tilde{\lambda} \cdot \Delta T$ (expected arrivals per batch), we can analyze the system as an *embedded Markov chain* observed at batch completion times, indexed by $b \in \mathbb{N}$. Setting $n = \lambda$ corresponds to critical loading. The throughput loss arises from “stuck” iterations where insufficient prompts are available.

LEMMA 1 (Queue Length and Stuck Time). *This lemma connects queue accumulation to throughput loss through stuck iterations. Let X^b denote the number of arrivals during batch b , where $X^b \sim \text{Poisson}(\lambda)$. Let W^b denote the queue length after batch b , with $W^0 = 0$. The state transition is:*

$$W^{b+1} = W^b + X^b - \lambda \cdot \mathbf{1}\{W^b + X^b \geq \lambda\}.$$

Define B_{stuck} as the number of stuck iterations (where $W^b + X^b < \lambda$). Then:

$$\mathbb{E}[W^B] = \lambda \cdot \mathbb{E}[B_{\text{stuck}}].$$

The proof is in Appendix E.1. This lemma connects queue accumulation to throughput loss: stuck iterations directly translate to waiting prompts. To bound $\mathbb{E}[W^B]$, we construct a coupled dominating process that admits tractable analysis via classical queueing techniques.

LEMMA 2 (Coupled Dominating Process). *The coupled process starts with a safety buffer (2λ vs 0) and maintains dominance throughout. Define $\{\tilde{W}^b\}$ by:*

$$\tilde{W}^0 = 2\lambda, \quad \tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\}.$$

Then $\tilde{W}^b \geq W^b + \lambda$ for all $b \in \mathbb{N}$.

The proof is in Appendix E.2.

Lindley recursion. To analyze the coupled process, we apply the Lindley recursion representation (Asmussen 2003, Proposition 6.3):

LEMMA 3 (Lindley Representation). *Let $S^k = \sum_{r=1}^k (X^r - \lambda)$ be the partial sum process. Then:*

$$\tilde{W}^B = 2\lambda + \max(S^B, S^B - S^1, \dots, S^B - S^{B-1}, 0).$$

This classical queueing representation is made applicable by the WAIT policy's threshold design. Define $\xi^k = X^k - \lambda$. We use time-reversal symmetry to simplify the maximum expression:

$$\max(S^B, S^B - S^1, \dots, S^B - S^{B-1}, 0) = \max_{1 \leq k \leq B} \left(\sum_{r=k}^B \xi^r \right)^+ \stackrel{d}{=} \max_{1 \leq k \leq B} \left(\sum_{r=1}^k \eta^r \right)^+, \quad \eta^r \stackrel{d}{=} X^1 - \lambda.$$

Thus $\tilde{W}^B \stackrel{d}{=} 2\lambda + \max_{1 \leq k \leq B} (\sum_{r=1}^k \eta^r)^+$.

Since $\tilde{W}^B \geq W^B + \lambda$, we have $\mathbb{E}[W^B] \leq \mathbb{E}[\tilde{W}^B] - \lambda$. Let $Y^B = \max_{1 \leq k \leq B} (\sum_{r=1}^k \eta^r)^+$. By (Strait 1974, Lemma 2):

LEMMA 4 (Random Walk Maximum).

$$\mathbb{E}[Y^B] = \mathbb{E} \left[\max_{1 \leq k \leq B} (S^k)^+ \right] = \sum_{k=1}^B \frac{1}{k} \mathbb{E}[(S^k)^+].$$

This standard result (Strait 1974) decomposes the maximum into a sum over batch indices. For $\mathbb{E}[(S^k)^+]$, by the Cauchy-Schwarz inequality and $\text{Var}(S^k) = \lambda k$, we have $\mathbb{E}[(S^k)^+] \leq \sqrt{\lambda k}$. Hence:

$$\mathbb{E}[Y^B] \leq \sqrt{\lambda} \sum_{k=1}^B k^{-1/2} \sim \sqrt{\lambda B} \quad \text{as } B \rightarrow \infty.$$

LEMMA 5 (Throughput Gap Bound).

$$\mathbb{E}[W^B] - \lambda \leq \mathbb{E}[Y^B] \leq \sqrt{\lambda} \sum_{k=1}^B \frac{1}{\sqrt{k}} \sim \sqrt{\lambda B}, \quad B \rightarrow \infty.$$

The throughput gap is thus $\frac{1}{B} \mathbb{E}[W^B] = O(B^{-1/2})$.

Asymptotic Regime. In the asymptotic regime where the time horizon scales as $\zeta \rightarrow \infty$:

$$\frac{1}{B^\zeta} \mathbb{E}[W^{B,\zeta}] = \frac{\lambda^\zeta}{B^\zeta} + \frac{(\lambda B)^{\zeta/2}}{B^\zeta} = \left(\frac{\lambda}{B}\right)^\zeta + \lambda^{\zeta/2} B^{-\zeta/2} \rightarrow 0 \quad \text{as } \zeta \rightarrow \infty.$$

B.0.2. Multiple-Type Case We now extend to m prompt types. Consider the WAIT policy $\pi = [n_1, \dots, n_m]$, where n_j is the threshold for type- j prompts.

Policy Class Π . Define the processing time for a full batch (containing all m types):

$$\Delta T_{[1,\dots,m]} = d_0 + d_1 \cdot \sum_{j=1}^m n_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right). \quad (17)$$

Approaching Load Balance. The constraint $\Delta T_{[1,\dots,m]} \leq n_j / \lambda_j$ (Equation 18) embodies the "approaching load balance" principle. For each type j , expected arrivals during a full batch equal $\lambda_j \cdot \Delta T_{[1,\dots,m]}$. The constraint ensures arrivals do not exceed batch size n_j , preventing queue accumulation and eviction risk. The equilibrium policy π^* achieves equality (arrivals = completions), maximizing throughput while maintaining stability.

The policy parameters must satisfy:

$$\begin{aligned} \Delta T_{[1,\dots,m]} &\leq \frac{n_j}{\lambda_j} \text{ for all } j \in [m], \\ M^\pi &= \sum_{j=1}^m n_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right) \geq M^*. \end{aligned} \quad (18)$$

These constraints ensure that for any $\pi \in \Pi$: when all m types are batched together, arrivals do not exceed completions for each type (i.e., $\text{Arrival}_j \leq \text{Completion}_j$). The equilibrium policy corresponds to $\pi^* = [\frac{n_1^*}{l'_1+1}, \dots, \frac{n_m^*}{l'_m+1}]$ when $M^\pi = M^*$.

Notation. Let b denote the batch index, J^b the set of types in batch b , $t_s(b)$ the start time and $t_e(b)$ the end time of batch b . The waiting time $t_s(b+1) - t_e(b) > 0$ occurs only when no type meets its threshold.

State Transition. At batch $b+1$:

$$W_{(j)}^{t_e(b+1)} = W_{(j)}^{t_e(b)} + X_{(j)}^{[t_e(b), t_s(b+1)]} + Y_{(j)}^{[t_s(b+1), t_e(b+1)]} - n_j \cdot \mathbf{1}\{j \in J^{b+1}\}, \quad j = 1, \dots, m,$$

where $X_{(j)}^{[t_e(b), t_s(b+1)]} \sim \text{Poisson}(\lambda_j \cdot (t_s(b+1) - t_e(b)))$ and $Y_{(j)}^{[t_s(b+1), t_e(b+1)]} \sim \text{Poisson}(\lambda_j \cdot (t_e(b+1) - t_s(b+1)))$ are independent arrival counts during waiting and processing periods.

Batch Processing Time. Recall from (17) that $\Delta T_{[1,\dots,m]}$ denotes the processing time for a full batch. Under fixed thresholds, this time is constant across batches.

Type Decoupling. Fixed thresholds n_j enable type decoupling. Each type j has independent arrivals during batch processing: $X_{(j)}^b \sim \text{Poisson}(\lambda_j \cdot \Delta T_{[1, \dots, m]})$. Arrivals depend only on the type-specific rate λ_j and batch time $\Delta T_{[1, \dots, m]}$, not on other types' queue states. This independence allows us to analyze each type separately and sum the resulting bounds.

LEMMA 6 (Multiple-Type Coupled Process). *Define a coupled process $\{\tilde{W}_{(j)}^b\}$ indexed by batch b :*

$$\tilde{W}_{(j)}^0 = 2n_j, \quad \tilde{W}_{(j)}^{b+1} = \max\{2n_j, \tilde{W}_{(j)}^b + X_{(j)}^b - n_j\}, \quad X_{(j)}^b \sim \text{Poisson}(\lambda_j \cdot \Delta T_{[1, \dots, m]}).$$

Then $\tilde{W}_{(j)}^b \geq W_{(j)}^b$ for all $b \geq 0$ and $j = 1, \dots, m$.

The proof is in Appendix E.3. Define $Y_{(j)}^b = X_{(j)}^b - n_j$. The coupled process becomes:

$$\tilde{W}_{(j)}^0 = 2n_j, \quad \tilde{W}_{(j)}^{b+1} = \max\{2n_j, \tilde{W}_{(j)}^b + Y_{(j)}^b\}.$$

Case 1: Zero Drift ($\Delta T_{[1, \dots, m]} = n_j / \lambda_j$). When $\lambda_j \cdot \Delta T_{[1, \dots, m]} = n_j$, we have $\mathbb{E}[Y_{(j)}^b] = 0$ (zero drift). By Lemma 5:

$$\mathbb{E}[W_{(j)}^B] \leq \mathbb{E}[\tilde{W}_{(j)}^B] \leq \lambda_j + \sqrt{\lambda_j B}.$$

Case 2: Negative Drift ($\Delta T_{[1, \dots, m]} < n_j / \lambda_j$). When $\mathbb{E}[Y_{(j)}^b] = \lambda_j \Delta T_{[1, \dots, m]} - n_j < 0$, the coupled process has negative drift. Since

$$\text{Var}(Y_{(j)}^b) = \lambda_j \cdot \Delta T_{[1, \dots, m]}, \quad \left| \mathbb{E}[Y_{(j)}^b] \right| = n_j - \lambda_j \cdot \Delta T_{[1, \dots, m]},$$

by Kingman's bound (Kingman 1962):

$$\mathbb{E}[W_{(j)}^B] \leq \mathbb{E}[\tilde{W}_{(j)}^B] \leq 2n_j + \frac{\lambda_j \cdot \Delta T_{[1, \dots, m]}}{2(n_j - \lambda_j \cdot \Delta T_{[1, \dots, m]})}.$$

Equilibrium Case ($C = M^*$). When $\pi^* = [\frac{n_1^*}{l_1^*+1}, \dots, \frac{n_m^*}{l_m^*+1}]$, all types satisfy $\Delta T_{[1, \dots, m]} = n_j / \lambda_j$ (zero drift). Summing over all types:

$$\frac{1}{B} \sum_{j=1}^m \mathbb{E}[W_{(j)}^B] \leq \frac{1}{B} \sum_{j=1}^m \lambda_j + \sum_{j=1}^m \sqrt{\frac{\lambda_j}{B}} = O(B^{-1/2}).$$

Asymptotic Regime. As $\zeta \rightarrow \infty$:

$$\frac{1}{\zeta B} \sum_{j=1}^m \lambda_j + \sum_{j=1}^m \sqrt{\frac{\lambda_j}{\zeta B}} \rightarrow 0.$$

The expected end-to-end latency scales as $O(B^{-1/2})$.

Strictly Negative Drift Case. When $\Delta T_{[1,\dots,m]} < n_j/\lambda_j$ for all j , the throughput gap is $O(B^{-1})$:

$$\frac{1}{B} \sum_{j=1}^m \mathbb{E} [W_{(j)}^B] \leq \frac{1}{B} \sum_{j=1}^m \left(2n_j + \frac{\lambda_j \cdot \Delta T_{[1,\dots,m]}}{2(n_j - \lambda_j \cdot \Delta T_{[1,\dots,m]})} \right) = O(B^{-1}).$$

Appendix C: Proof of Theorem 2

Proof Outline. This proof establishes asymptotic optimality of Nested WAIT when output lengths are unknown. The algorithm achieves *on-the-fly classification*: prompts classify themselves by their actual output lengths at segment boundaries, requiring no prediction. The structure proceeds as follows:

1. **First segment analysis:** All prompts enter segment 1 with Poisson arrivals. We couple with a Lindley process and apply Kingman's inequality for bounded expectations.
2. **Subsequent segments analysis:** After completing segment $k - 1$, prompts naturally reveal whether they continue to segment k . This type revelation follows Binomial thinning: a fraction p_k of completions continue, while short prompts finish without being delayed by longer ones. We apply Lindley coupling to each segment.
3. **High-probability bounds prevent eviction:** Expected bounds ensure average stability, but stochastic fluctuations risk overflow. We construct an exponential martingale and apply Doob's inequality to obtain high-probability memory bounds, preventing eviction cascades.

We adopt notation consistent with the main text (Section 5). Let l denote the common input length (tokens after prefill), and let l'_j denote the output length for type- j prompts. We use b to denote the *batch index* (count of completed iterations) and $k \in \{1, \dots, m\}$ to denote the *segment index*, where segment k contains prompts with output lengths in $(l'_{k-1}, l'_k]$. The threshold for segment k is n_k (from Equation (11)). We denote by $W_{(k)}^b$ the queue length for segment k after batch b , and by B the total number of batches over the time horizon. The thinning probability $p_k = (\sum_{j=k}^m \lambda_j) / (\sum_{j=k-1}^m \lambda_j)$ represents the fraction of prompts completing segment $k - 1$ that continue to segment k .

C.1. First Segment Analysis

All prompts initially enter segment 1 with Poisson arrivals at aggregate rate $\sum_{j=1}^m \lambda_j$. We construct a dominating process and apply Kingman's inequality to bound expected queue length.

For the first segment ($k = 1$), the state transition is:

$$W_{(1)}^{b+1} = W_{(1)}^b + Y_{(1)}^b - n_1 \cdot \mathbf{1}\{W_{(1)}^b + Y_{(1)}^b \geq n_1\}, \quad Y_{(1)}^b \sim \text{Poisson}(\Delta T_b \cdot \sum_{j=1}^m \lambda_j),$$

where ΔT_b denotes the processing time of batch b . Since $\Delta T_b \leq \Delta T_{[1,\dots,m]}$ (the time for a full batch), we construct a dominating process:

$$\bar{W}_{(1)}^{b+1} = \bar{W}_{(1)}^b + \bar{Y}_{(1)}^b - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^b + \bar{Y}_{(1)}^b \geq n_1\}, \quad \bar{Y}_{(1)}^b \sim \text{Poisson}(\Delta T_{[1,\dots,m]} \cdot \sum_{j=1}^m \lambda_j).$$

This is further dominated by a Lindley process:

$$\tilde{W}_{(1)}^0 = 2n_1, \quad \tilde{W}_{(1)}^{b+1} = \max\{2n_1, \tilde{W}_{(1)}^b + \bar{Y}_{(1)}^b - n_1\}.$$

Since $\Delta T_{[1,\dots,m]} \cdot \sum_{j=1}^m \lambda_j < n_1$ (negative drift), by Kingman's inequality (Kingman 1962):

$$\mathbb{E} \left[W_{(1)}^b \right] \leq 2n_1 + \frac{\Delta T_{[1,\dots,m]} \cdot \sum_{j=1}^m \lambda_j}{2 \left(n_1 - \Delta T_{[1,\dots,m]} \cdot \sum_{j=1}^m \lambda_j \right)}.$$

Remark. Prompts waiting in the first segment are stored in CPU and do not consume GPU memory.

C.2. Subsequent Segments Analysis ($k \geq 2$)

For segment k ($2 \leq k \leq m$), the arrival process reflects on-the-fly classification: prompts that completed segment $k-1$ reveal whether their output lengths exceed l'_{k-1} . This type revelation follows binomial thinning with probability p_k .

Arrivals come from prompts that completed segment $k-1$:

$$Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k), \quad \text{where } p_k = \frac{\sum_{j=k}^m \lambda_j}{\sum_{j=k-1}^m \lambda_j}.$$

On-the-fly Classification via Binomial Thinning. The arrival process $Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k)$ reflects the *on-the-fly classification mechanism*, not merely a modeling choice. After segment $k-1$ processing, exactly n_{k-1} prompts complete that stage. Among these, a fraction $p_k = \frac{\sum_{j=k}^m \lambda_j}{\sum_{j=k-1}^m \lambda_j}$ have output lengths exceeding l'_{k-1} . These prompts naturally continue to segment k . Prompts classify themselves by their actual output lengths—no prediction needed. Short prompts complete quickly without being delayed by longer ones.

The state transition is:

$$W_{(k)}^{b+1} = W_{(k)}^b + Y_{(k)}^b - n_k \cdot \mathbf{1}\{W_{(k)}^b + Y_{(k)}^b \geq n_k\},$$

where b indexes the batches containing segment $k-1$. Note that each segment has its own batch index, but all are bounded by the total batch count B .

Define $X_{(k)}^b = Y_{(k)}^b - n_k$. We construct a coupled dominating process:

LEMMA 7 (Coupled Process for Segment k). Define $\{\tilde{W}_{(k)}^b\}$ by:

$$\begin{aligned}\tilde{W}_{(k)}^0 &= n_k, \\ \tilde{W}_{(k)}^{b+1} &= \max\{n_k, \tilde{W}_{(k)}^b + X_{(k)}^b\}.\end{aligned}$$

Then $\tilde{W}_{(k)}^b \geq W_{(k)}^b$ for all $b \in \mathbb{N}$.

The proof is in Appendix E.4. The process $\{\tilde{W}_{(k)}^b\}$ has the Lindley representation:

$$\begin{aligned}\tilde{W}_{(k)}^{b+1} &= n_k + \max_{0 \leq i \leq b} \{S_{(k)}^i\}, \\ \text{where } S_{(k)}^i &= \sum_{r=1}^i X_{(k)}^r, \quad S_{(k)}^0 = 0.\end{aligned}$$

Expected Queue Length. Since $\mathbb{E}[X_{(k)}^b] = n_{k-1}p_k - n_k < 0$ (negative drift), by Kingman's inequality (Kingman 1962):

$$\mathbb{E}[W_{(k)}^b] \leq \mathbb{E}[\tilde{W}_{(k)}^b] \leq n_k + \frac{n_{k-1} \cdot p_k (1 - p_k)}{2(n_k - n_{k-1} \cdot p_k)}.$$

Negative drift ensures queue stability, preventing unbounded growth that would lead to eviction. This bound ensures finite expected queue length for each segment $k \geq 2$, controlling expected GPU memory consumption. The segment structure ensures shorter prompts complete without waiting for longer ones to finish.

C.3. Throughput Gap Bound

Using B as the total batch count (upper bound for each segment's batch index), the throughput gap is:

$$\frac{1}{B} \sum_{k=1}^m \mathbb{E}[W_{(k)}^B] \leq \frac{2n_1}{B} + \frac{\Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j}{2B(n_1 - \Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j)} + \frac{1}{B} \sum_{k=2}^m n_k + \frac{1}{B} \sum_{k=2}^m \frac{n_{k-1} \cdot p_k (1 - p_k)}{2(n_k - n_{k-1} \cdot p_k)}.$$

Asymptotic Regime. As $\zeta \rightarrow \infty$, the throughput gap is $O((\zeta B)^{-1})$. The expected latency and TTFT are $O(1)$.

C.4. High-Probability Memory Bound

High-Probability Bounds Prevent Eviction. Expected queue length bounds ensure average stability. However, stochastic fluctuations may cause $W_{(k)}^b$ to exceed these limits, risking memory overflow and eviction. We derive high-probability bounds to quantify this overflow risk. By choosing capacity C to ensure overflow probability $< \delta$ across all batches and segments, we prevent eviction cascades.

Martingale Construction. To obtain high-probability bounds on queue length, we need to control tail probabilities of the random walk maximum $\max_{0 \leq i \leq s-1} \{S_{(k)}^i\}$. Expected bounds alone cannot prevent rare but catastrophic overflow events. We construct an exponential martingale that enables Doob's inequality, converting expected bounds into probabilistic guarantees.

We need to estimate

$$P\left(\max\{0, S_{(k)}^1, \dots, S_{(k)}^{s-1}\} \geq c\right) = P\left(\max_{0 \leq i \leq s-1} \{S_{(k)}^i\} \geq c\right), \forall c > 0, \text{ where } S_{(k)}^0 = 0.$$

Define the exponential martingale $M_{(k)}^i = e^{\theta S_{(k)}^i}$. For this to be a martingale, the moment generating function must equal 1, ensuring zero expected drift in log space:

$$\phi_{(k)}(\theta) = \mathbb{E}\left[e^{\theta X_{(k)}^i}\right] = e^{-\theta n_k}(1 - p_k + p_k e^\theta)^{n_{k-1}} = 1.$$

The following lemma establishes existence and properties of the solution $\theta_k > 0$ that satisfies this martingale condition (proof in Appendix E.5).

LEMMA 8. *Suppose $n_{k-1} > n_k > n_{k-1}p_k$ and $p_k \in (0, 1)$. Then there exists a unique solution $\theta_k > 0$ to the equation*

$$e^{-\theta_k n_k}(1 - p_k + p_k e^{\theta_k})^{n_{k-1}} = 1, \quad (19)$$

where θ_k is strictly increasing with respect to $D = n_k - n_{k-1}p_k$ and satisfies the lower bound

$$\theta_k \geq \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}.$$

When $\theta_k > 0$ satisfies $\phi(\theta_k) = 1$, the process $M_{(k)}^i$ is a martingale starting at $M_{(k)}^0 = e^{\theta_k \cdot 0} = 1$. This zero-drift property in log space enables Doob's inequality to bound tail probabilities.

By Doob's martingale inequality (Durrett 2019), we have that

$$P\left(\max_{1 \leq i \leq b-1} \{e^{\theta_k S_{(k)}^i}\} \geq a\right) \leq \frac{\mathbb{E}[M^i]}{a} = \frac{\mathbb{E}[M^0]}{a} = \frac{1}{a}.$$

Exponentiating both sides of the inequality event, we obtain:

$$P\left(\max_{1 \leq i \leq b-1} \{e^{\theta_k S_{(k)}^i}\} \geq e^{\theta_k c}\right) \leq \frac{1}{e^{\theta_k c}} \Rightarrow P\left(\max_{1 \leq i \leq b-1} \{S_{(k)}^i\} \geq c\right) \leq e^{-\theta_k c}.$$

Using the dominance $\tilde{W}_{(k)}^b = n_k + \max_{0 \leq i \leq b-1} \{S_{(k)}^i\} \geq W_{(k)}^b$, we have

$$P\left(W_{(k)}^b \geq c\right) < P\left(\tilde{W}_{(k)}^b \geq c\right) = P\left(\max_{1 \leq i \leq b-1} \{S_{(k)}^i\} \geq c - n_k\right) \leq e^{-\theta_k(c - n_k)}$$

For all $2 \leq k \leq m$ and $1 \leq b \leq B$:

$$P\left(W_{(k)}^b \geq c\right) < e^{-\theta_k(c-n_k)},$$

where $\theta_k > 0$ is the solution to (19).

Union bound for safety guarantee. To ensure memory consumption does not exceed the bound at any batch $b \in \{1, \dots, B\}$ with probability at least $1 - \delta$, we apply a union bound across all segments and batches. For each segment $k \in \{2, \dots, m\}$ and batch index $b \in \{1, \dots, B\}$, we have:

$$P\left(W_{(k)}^b \geq c\right) < e^{-\theta_k(c-n_k)},$$

There are $m - 1$ segments (from $k = 2$ to m) and B batches (from $b = 1$ to B). We allocate the total failure probability δ across all segments and batches. The total number of events is $(m - 1) \cdot B$. Set the overflow probability for each segment and batch to:

$$P\left(W_{(k)}^b \geq c_k\right) \leq \frac{\delta}{(m - 1) \cdot B} \Rightarrow e^{-\theta_k(c_k - n_k)} \leq \frac{\delta}{(m - 1) \cdot B} \Rightarrow c_k \geq n_k + \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k}$$

If we choose

$$c_k = n_k + \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k},$$

then we have that

$$P\left(W_{(k)}^b \geq c_k\right) \leq \frac{\delta}{(m - 1) \cdot B}.$$

Next, we compute the joint probability of overflow across all segments and batches using a union bound:

$$P\left(\bigcup_{k=2}^m \bigcup_{b=1}^B \{W_{(k)}^b \geq c_k\}\right) \leq \sum_{k=2}^m \sum_{b=1}^B P\left(W_{(k)}^b \geq c_k\right) \leq (m - 1) \cdot B \cdot \frac{\delta}{(m - 1) \cdot B} = \delta$$

Thus:

$$P\left(W_{(k)}^b \leq c_k \text{ for all } \begin{matrix} k \in \{2, \dots, m\}, \\ b \in \{1, \dots, B\} \end{matrix}\right) \geq 1 - \delta.$$

So the memory consumption is bounded:

$$\text{Memory Consumption}^b = M^\pi + \sum_{k=2}^m W_{(k)}^b \cdot (l + l'_{k-1}) \leq M^\pi + \sum_{k=2}^m c_k \cdot (l + l'_{k-1})$$

where M^π is the peak batch memory usage as defined in Equation (12):

$$M^\pi = \sum_{j=1}^m n_j \left(l + \frac{L'_j}{2} \right) \Delta l'_j, \quad \text{with } L'_j = \sum_{i=1}^j l'_i, \Delta l'_j = l'_j - l'_{j-1}, l'_0 = 0.$$

Note that this differs from the M^π formula for the WAIT algorithm (Theorem 1), because Nested WAIT groups prompts into segments based on their unknown output lengths, leading to segment-wise memory accumulation

Substitute $c_k = n_k + \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k}$, we have that

$$\text{Memory Consumption}^b \leq M^\pi + \sum_{k=2}^m \left(n_k + \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k} \right) \cdot (l + l'_{k-1}).$$

Define the memory bound $\mathbf{M}^B$:

$$\mathbf{M}^B = M^\pi + \sum_{k=2}^m n_k \cdot (l + l'_{k-1}) + \sum_{k=2}^m \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k} \cdot (l + l'_{k-1}).$$

By the lower bound in Lemma 8, we can obtain the upper bound of the third term:

$$\sum_{k=2}^m \frac{\ln\left(\frac{(m-1) \cdot B}{\delta}\right)}{\theta_k} \cdot (l + l'_{k-1}) \leq \sum_{k=2}^m \frac{n_{k-1}}{8(n_k - n_{k-1}p_k)} \ln\left(\frac{(m-1) \cdot B}{\delta}\right) \cdot (l + l'_{k-1})$$

And this $\mathbf{M}^B$ ensures that with probability at least $1 - \delta$, the memory consumption does not exceed $\mathbf{M}^B$ at any batch index $b \in \{1, \dots, B\}$.

Appendix D: Proof of Theorem 3

Proof Outline. This proof extends Nested WAIT to time-varying arrival rates $\lambda_j(t)$. Time-varying rates make exact load balance impossible, since the optimal threshold depends on the instantaneous arrival pattern. The *worst-case domination strategy* maintains load balance even at peak rates: by designing thresholds to handle worst-case arrivals, we ensure system stability across all scenarios. This conservative approach prevents eviction throughout the time horizon. The structure proceeds as follows:

1. **First segment analysis:** Poisson arrivals with time-varying rates are dominated by a time-invariant process using worst-case aggregate rate $\Lambda^\pi = \sup_b \left\{ \sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}] \right\}$. We couple with a Lindley process and apply Kingman's inequality.

2. **Subsequent segments analysis:** Binomial arrivals with time-varying thinning probabilities $p_k(b)$ are dominated by worst-case $p_k^* = \sup_b \{p_k(b)\}$. This ensures safety across all batch compositions. We apply Lindley coupling to each segment.

3. **Throughput and high-probability bounds:** Asymptotic optimality follows from the time-invariant dominated processes. High-probability memory bounds follow the same martingale construction, using worst-case parameters to prevent eviction.

We use b to denote the *batch index* and $k \in \{1, \dots, m\}$ to denote the *segment index*. The total number of batches is denoted by B .

D.1. First Segment Analysis

For the first segment ($k = 1$), the state transition rule is

$$W_{(1)}^{b+1} = W_{(1)}^b + Y_{(1)}^b - n_1 \cdot \mathbf{1}\{W_{(1)}^b + Y_{(1)}^b \geq n_1\}, \quad Y_{(1)}^b \sim \text{Poisson}\left(\sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_b]\right),$$

where $t(b)$ denotes the start time of batch b and ΔT_b denotes the processing time of batch b .

Worst-case domination. Time-varying arrival rates prevent exact load balance since optimal thresholds depend on instantaneous arrival patterns. To maintain stability across all scenarios, we use worst-case domination: design thresholds to handle peak arrival rates, ensuring eviction prevention even under worst-case conditions. Since $\Delta T_b \leq \Delta T_{[1, \dots, m]}$ (the time for a full batch containing all m types), we construct a dominating process $\{\bar{W}_{(1)}^b\}$:

$$\bar{W}_{(1)}^{b+1} = \bar{W}_{(1)}^b + \bar{Y}_{(1)}^b - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^b + \bar{Y}_{(1)}^b \geq n_1\}, \quad \bar{Y}_{(1)}^b \sim \text{Poisson}\left(\sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}]\right).$$

By (14), the aggregate arrival rate during any batch is bounded by the worst-case aggregate rate:

$$\sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}] \leq \sup_{\forall b} \left\{ \sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}] \right\} < n_1.$$

Define $\Lambda^\pi = \sup_{\forall b} \left\{ \sum_{j=1}^m \lambda_j[t(b), t(b) + \Delta T_{[1, \dots, m]}] \right\} < n_1$ as the worst-case aggregate arrival rate.

This enables a time-invariant dominating process that provides tractable bounds:

$$\tilde{W}_{(1)}^{b+1} = \tilde{W}_{(1)}^b + \tilde{Y}_{(1)}^b - n_1 \cdot \mathbf{1}\{\tilde{W}_{(1)}^b + \tilde{Y}_{(1)}^b \geq n_1\}, \quad \tilde{Y}_{(1)}^b \sim \text{Poisson}(\Lambda^\pi).$$

This time-invariant process enables classical queueing analysis. Define $X_{(1)}^b = \tilde{Y}_{(1)}^b - n_1$. The process $\{\tilde{W}_{(1)}^b\}$ is dominated by a Lindley process:

$$\bar{W}_{(1)}^0 = 2n_1, \quad \bar{W}_{(1)}^{b+1} = \max\{2n_1, \bar{W}_{(1)}^b + X_{(1)}^b\}, \quad \forall b \in \mathbb{N}.$$

Thus we have the dominance chain:

$$W_{(1)}^b \leq \bar{W}_{(1)}^b \leq \tilde{W}_{(1)}^b \leq \overline{W}_{(1)}^b, \quad \forall b \in \mathbb{N}.$$

Since $n_1 > \Lambda^\pi$, the process $\{\bar{W}_{(1)}^b\}$ has negative drift. By Kingman's inequality (Kingman 1962), with

$$\text{Var}(X_{(1)}^b) = \Lambda^\pi, \quad \left| \mathbb{E} \left[X_{(1)}^b \right] \right| = n_1 - \Lambda^\pi,$$

we obtain

$$\mathbb{E} \left[W_{(1)}^b \right] \leq 2n_1 + \frac{\Lambda^\pi}{2(n_1 - \Lambda^\pi)}.$$

D.2. Subsequent Segments Analysis ($k \geq 2$)

For segment k ($2 \leq k \leq m$), arrivals come from prompts that completed segment $k - 1$. The batch index b for segment k counts only batches containing segment $k - 1$. These indices differ across segments but are bounded by the total batch count B .

Time-varying thinning probabilities. Consider the b -th arrival to segment k , which comes from the b -th batch containing segment $k - 1$. The thinning probability $p_k(b)$ depends on the prompt composition in this batch and varies over time. Although exact arrival times are unknown, arrivals occur within some interval $[t, t + \Delta t]$ where $t + \Delta t \leq t(b, k - 1)$, the starting time of the b -th batch containing segment $k - 1$. To handle this time variation, we use the worst-case thinning probability. By (14):

$$p_k(b) = \frac{\sum_{j=k+1}^m \lambda_j[t, t + \Delta t]}{\sum_{j=k}^m \lambda_j[t, t + \Delta t]} \leq p_k^* < \frac{n_k}{n_{k-1}}, \quad \forall b \in \mathbb{N}.$$

The arrival process is

$$Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k(b)), \quad \forall b \geq 0,$$

and the state transition rule is

$$W_{(k)}^{b+1} = W_{(k)}^b + Y_{(k)}^b - n_k \cdot \mathbf{1}\{W_{(k)}^b + Y_{(k)}^b \geq n_k\},$$

where b indexes batches containing segment $k - 1$, since new arrivals to segment k occur only when segment $k - 1$ is processed.

Define $X_{(k)}^b = Y_{(k)}^b - n_k$. Although the distribution $Y_{(k)}^b \sim \text{Binomial}(n_{k-1}, p_k(b))$ is time-varying, the coupling construction follows Lemma 7:

LEMMA 9 (Coupling for Time-Varying Case). Define a coupled process $\{\tilde{W}_{(k)}^b\}$ by:

$$\begin{aligned}\tilde{W}_{(k)}^0 &= n_k, \\ \tilde{W}_{(k)}^{b+1} &= \max\{n_k, \tilde{W}_{(k)}^b + X_{(k)}^b\}, \\ X_{(k)}^b &= Y_{(k)}^b - n_k.\end{aligned}$$

Then $\tilde{W}_{(k)}^b \geq W_{(k)}^b$ for all $b \in \mathbb{N}$.

To apply Kingman's inequality (Kingman 1962), we construct a time-invariant dominating process:

$$\begin{aligned}\bar{W}_{(k)}^0 &= n_k, \\ \bar{W}_{(k)}^{b+1} &= \max\{n_k, \bar{W}_{(k)}^b + \bar{X}_{(k)}^b\}, \\ \bar{X}_{(k)}^b &= \bar{Y}_{(k)}^b - n_k, \\ \bar{Y}_{(k)}^b &\sim \text{Binomial}(n_{k-1}, p_k^*), \quad \forall b \in \mathbb{N}.\end{aligned}$$

Then we have the dominance chain:

$$\bar{W}_{(k)}^b \geq \tilde{W}_{(k)}^b \geq W_{(k)}^b, \quad \forall b \in \mathbb{N}.$$

The process $\bar{W}_{(k)}^b$ has the Lindley representation:

$$\begin{aligned}\bar{W}_{(k)}^0 &= n_k, \\ \bar{W}_{(k)}^{b+1} &= n_k + \max_{0 \leq i \leq b} \{\bar{S}_{(k)}^i\}, \\ \text{where } \bar{S}_{(k)}^0 &= 0, \bar{S}_{(k)}^i = \sum_{r=1}^i \bar{X}_{(k)}^r, \quad 1 \leq i \leq b.\end{aligned}$$

Since

$$\mathbb{E} \left[\bar{X}_{(k)}^b \right] = n_{k-1} \cdot p_k^* - n_k < 0, \quad \forall b \in \mathbb{N},$$

the coupled process $\{\bar{W}_{(k)}^b\}$ has negative drift. With

$$\text{Var}(\bar{X}_{(k)}^b) = n_{k-1} p_k^* (1 - p_k^*), \quad \left| \mathbb{E} \left[\bar{X}_{(k)}^b \right] \right| = n_k - p_k^* n_{k-1},$$

Kingman's inequality (Kingman 1962) yields:

$$\mathbb{E} \left[\max_{0 \leq i \leq b} \{\bar{S}_{(k)}^i\} \right] \leq \frac{n_{k-1} p_k^* (1 - p_k^*)}{2(n_k - n_{k-1} p_k^*)}.$$

Thus, the expected queue length is bounded:

$$\mathbb{E} \left[W_{(k)}^b \right] \leq \mathbb{E} \left[\tilde{W}_{(k)}^b \right] \leq \mathbb{E} \left[\bar{W}_{(k)}^b \right] \leq n_k + \frac{n_{k-1} p_k^* (1 - p_k^*)}{2(n_k - n_{k-1} p_k^*)}.$$

D.3. Throughput and High-Probability Bounds

The discussion of asymptotic optimal throughput under bounded latency and TTFT follows Appendix C. For high-probability bounds, the analysis follows the same martingale construction applied to the time-invariant coupled process $\{\bar{W}_{(k)}^b\}$ defined above. By the same argument as in Appendix C, with $\bar{X}_{(k)}^r = \bar{Y}_{(k)}^r - n_k$, $\bar{Y}_{(k)}^r \sim \text{Binomial}(n_{k-1}, p_k^*)$, and $p_k^* < n_k/n_{k-1}$, the memory bound $\mathbf{M}^B$ is:

$$\mathbf{M}^B = M^\pi + \sum_{k=2}^m n_k \cdot (l + l'_{k-1}) + \sum_{k=2}^m \theta_k^{-1} \ln \left(\frac{(m-1) \cdot B}{\delta} \right) \cdot (l + l'_{k-1}),$$

where θ_k ($2 \leq k \leq m$) is the unique positive solution to:

$$e^{-\theta_k n_k} (1 - p_k^* + p_k^* e^{\theta_k})^{n_{k-1}} = 1. \quad (20)$$

Since $p_k^* = \sup_b \{p_k(b)\}$ represents the worst-case thinning probability, by the strictly increasing property of θ_k in Lemma 8, this yields the smallest θ_k and hence the largest memory bound term. The lower bound for θ_k follows from the same analysis in Appendix C.

Appendix E: Proof of Lemmas

Throughout this section, we use b for the batch index and B for the total number of batches.

E.1. Proof of Lemma 1

To prove this lemma, we take expectations on both sides of the state transition equation:

$$\mathbb{E} [W^{b+1}] = \mathbb{E} [W^b] + \lambda - \lambda \mathbb{P}(W^b + X^b \geq \lambda).$$

Summing from $b = 0$ to $B - 1$, and noting that $W^0 = 0$, we obtain:

$$\mathbb{E} [W^B] = \lambda B - \lambda \mathbb{E} [B - B_{\text{stuck}}] = \lambda \cdot \mathbb{E} [B_{\text{stuck}}].$$

E.2. Proof of Lemma 2

We prove by induction on the batch index b . The coupled process $\{\tilde{W}^b\}$ starts with a safety buffer of 2λ (compared to the real process starting at 0), ensuring it remains above $W^b + \lambda$ throughout. For the inductive step, we consider two cases based on whether the system can process a batch or experiences a stuck iteration.

Base case. For $b = 0$: $\tilde{W}^0 = 2\lambda \geq \lambda = W^0 + \lambda$.

Inductive step. Assume $\tilde{W}^b \geq W^b + \lambda$ for some $b \geq 0$. We show $\tilde{W}^{b+1} \geq W^{b+1} + \lambda$ by considering two cases:

Case 1 (Batch processed): If $W^b + X^b \geq \lambda$, the queue has accumulated enough prompts to process a batch, so $W^{b+1} = W^b + X^b - \lambda$. Then

$$\tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\} \geq \tilde{W}^b + X^b - \lambda \geq (W^b + \lambda) + X^b - \lambda = W^{b+1} + \lambda.$$

Case 2 (Stuck iteration): If $W^b + X^b < \lambda$, insufficient prompts are available and the system cannot process a batch (stuck iteration). The queue simply accumulates arrivals: $W^{b+1} = W^b + X^b$. The safety buffer ensures dominance:

$$\tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\} \geq 2\lambda = \lambda + \lambda > W^{b+1} + \lambda.$$

By induction, $\tilde{W}^b \geq W^b + \lambda$ for all $b \in \mathbb{N}$.

E.3. Proof of Lemma 6

Proof idea. The coupled process dominates the real process by maintaining a consistent full-batch processing schedule, while the real process may skip type j in some batches when $W_{(j)}^b < n_j$. We analyze periods when type j actively joins batches and show the coupled process makes at most one additional batch iteration, preserving dominance.

Proof. Since $\tilde{W}_{(j)}^b \geq 2n_j$, when $W_{(j)}^b < n_j$, we trivially have $\tilde{W}_{(j)}^b \geq W_{(j)}^b$.

Initially, $W_{(j)}^0 = 0$. As arrivals accumulate, $W_{(j)}^b$ reaches n_j and type j joins the batch. After processing, $W_{(j)}^b$ may drop below n_j , and the cycle repeats.

Consider a period $[b_0, b_1]$ when $W_{(j)}^b \geq n_j$, meaning type j is actively joining batches in the real process. Let $\tau = \max\{b \leq b_0 : W_{(j)}^b = n_j\}$ be the last time before b_0 when the queue length equals exactly n_j . We compare the number of batch iterations between the real and coupled processes over the interval $[\tau, b]$.

For any $b \in [b_0, b_1]$, the elapsed time is $t(b) - t(\tau)$, where $t(b)$ denotes the time at batch b . Since each batch takes at most $\Delta T_{[1, \dots, m]}$ time (the full-batch processing time), we can bound the number of batch iterations $I_{\tau \rightarrow b}$ (real process) and $\tilde{I}_{\tau \rightarrow b}$ (coupled process):

$$I_{\tau \rightarrow b} \geq \lfloor \frac{t(b) - t(\tau)}{\Delta T_{[1, \dots, m]}} \rfloor, \quad \tilde{I}_{\tau \rightarrow b} \leq 1 + \lfloor \frac{t(b) - t(\tau)}{\Delta T_{[1, \dots, m]}} \rfloor \Rightarrow \tilde{I}_{\tau \rightarrow b} \leq 1 + I_{\tau \rightarrow b}.$$

Thus, the coupled process makes at most one more iteration than the real process after τ .

At time τ , the real process may be in a batch without type j (since not all batches contain all types), while the coupled process just completed a full batch containing all types. This yields the initial dominance $\tilde{W}_{(j)}^\tau \geq n_j = W_{(j)}^\tau$.

For batches in the interval $(\tau, b]$, the arrival dynamics are identical for both processes. Since type j joins every batch in the real process during this period (as $W_{(j)}^b \geq n_j$), but the coupled process completes at most one additional batch, the dominance is maintained. Thus $\tilde{W}_{(j)}^b \geq W_{(j)}^b$ for all $b \geq 0$.

E.4. Proof of Lemma 7

The proof follows the same induction structure as Lemma 2, adapted to the Nested WAIT setting where arrivals follow binomial thinning from segment $k - 1$ completions.

Base case. For $b = 0$: $\tilde{W}_{(k)}^0 = n_k \geq 0 = W_{(k)}^0$.

Inductive step. Assume $\tilde{W}_{(k)}^b \geq W_{(k)}^b$ for some $b \geq 0$. We show $\tilde{W}_{(k)}^{b+1} \geq W_{(k)}^{b+1}$ by case analysis:

Case 1 (Segment batch processed): If $W_{(k)}^b + Y_{(k)}^b \geq n_k$, then $W_{(k)}^{b+1} = W_{(k)}^b + Y_{(k)}^b - n_k$, so

$$\tilde{W}_{(k)}^{b+1} = \max\{n_k, \tilde{W}_{(k)}^b + X_{(k)}^b\} \geq \tilde{W}_{(k)}^b + X_{(k)}^b \geq W_{(k)}^b + Y_{(k)}^b - n_k = W_{(k)}^{b+1}.$$

Case 2 (Segment stuck): If $W_{(k)}^b + Y_{(k)}^b < n_k$, then $W_{(k)}^{b+1} = W_{(k)}^b + Y_{(k)}^b$, so

$$\tilde{W}_{(k)}^{b+1} = \max\{n_k, \tilde{W}_{(k)}^b + X_{(k)}^b\} \geq n_k > W_{(k)}^{b+1}.$$

By induction, $\tilde{W}_{(k)}^b \geq W_{(k)}^b$ for all $b \in \mathbb{N}$.

E.5. Proof of Lemma 8

This lemma establishes existence and properties of the parameter $\theta_k > 0$ that makes the exponential process $e^{\theta_k S_{(k)}^i}$ a martingale. This enables Doob's inequality to convert expected queue length bounds into high-probability bounds, preventing eviction cascades.

Existence and uniqueness. For simplicity, we denote $\theta = \theta_k$. Define the function

$$f(\theta) = e^{-\theta n_k} (1 - p_k + p_k e^\theta)^{n_{k-1}}.$$

We seek $\theta > 0$ such that $f(\theta) = 1$. First, evaluate $f(\theta)$ at $\theta = 0$:

$$f(0) = e^0 (1 - p_k + p_k \cdot 1)^{n_{k-1}} = 1.$$

Thus, $\theta = 0$ is a trivial solution, but we need $\theta > 0$ to obtain useful high-probability bounds. To analyze $f(\theta)$ for $\theta > 0$, we consider the logarithm

$$g(\theta) = \ln f(\theta) = -\theta n_k + n_{k-1} \ln(1 - p_k + p_k e^\theta),$$

$$g'(\theta) = -n_k + n_{k-1} \cdot \frac{p_k e^\theta}{1 - p_k + p_k e^\theta}.$$

At $\theta = 0$,

$$g(0) = 0, \quad g'(0) = -n_k + n_{k-1} p_k.$$

Since $\mathbb{E}[X_{(k)}^b] = n_{k-1} p_k - n_k < 0$, we have $g'(0) < 0$, so $f(\theta)$ is decreasing near $\theta = 0$. Consider the second derivative:

$$g''(\theta) = n_{k-1} \cdot \frac{p_k e^\theta (1 - p_k)}{(1 - p_k + p_k e^\theta)^2} > 0 \quad \text{for } \theta > 0,$$

since $p_k \in (0, 1)$ and $n_{k-1} > 0$. Thus, $g(\theta)$ is strictly convex. Convexity ensures that $g(\theta)$ can cross zero at most once for $\theta > 0$, guaranteeing uniqueness. Now, consider the limit as $\theta \rightarrow \infty$:

$$f(\theta) \approx p_k^{n_{k-1}} e^{\theta(n_{k-1} - n_k)}.$$

By our settings, $n_{k-1} > n_k$, which ensures $f(\theta) \rightarrow \infty$ as $\theta \rightarrow \infty$. Thus, $f(\theta)$ is continuous, convex, starts at $f(0) = 1$, decreases below 1 for small $\theta > 0$ (since $g'(0) = n_{k-1} p_k - n_k < 0$ by negative drift), and increases to infinity. By the intermediate value theorem and convexity, there exists a unique $\theta > 0$ such that $f(\theta) = 1$.

Monotonicity in drift. Next, we prove that the solution $\theta > 0$ is strictly increasing with respect to the drift $D = n_k - n_{k-1} p_k$. This shows that tighter capacity (larger D , more negative drift) leads to larger θ and thus tighter high-probability bounds. Recall that $g(0) = 0$, $g'(0) = -D < 0$, and $g''(\theta) > 0$ for $\theta > 0$, so $g(\theta)$ is strictly convex. The unique positive solution $\theta > 0$ occurs where $g(\theta)$ crosses zero after initially decreasing from $g(0) = 0$.

Consider two values D_1 and D_2 such that $D_1 < D_2$, with corresponding functions $g_1(\theta) = -\theta(n_{k-1} p_k + D_1) + n_{k-1} \ln(1 - p_k + p_k e^\theta)$ and $g_2(\theta) = -\theta(n_{k-1} p_k + D_2) + n_{k-1} \ln(1 - p_k + p_k e^\theta)$, and solutions θ_1 and θ_2 , respectively. At $\theta = 0$, we have $g'_1(0) = -D_1 > g'_2(0) = -D_2$, meaning $g_2(\theta)$ decreases more steeply from zero than $g_1(\theta)$.

For a fixed $\theta > 0$, the partial derivative $\frac{\partial g}{\partial D} = -\theta < 0$, so increasing D decreases $g(\theta)$ pointwise. Since $g(\theta)$ is convex and approaches infinity as $\theta \rightarrow \infty$, the zero crossing of $g(\theta; D)$ shifts to a larger θ when D increases. Hence, if $D_2 > D_1$, then $\theta_2 > \theta_1$. Operationally, this means larger safety margins yield tighter high-probability bounds.

Drift-to- θ scaling. Finally, we characterize how the solution $\theta > 0$ scales with drift D , n_{k-1} , and p_k . When the drift $D = n_k - n_{k-1}p_k$ is small (system near critical load), the solution θ is also small. To derive an explicit approximation, we expand $g(\theta)$ around $\theta = 0$ using Taylor's theorem:

$$g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(0)\theta^2 + O(\theta^3) = 0 - D\theta + \frac{1}{2}n_{k-1}p_k(1 - p_k)\theta^2 + O(\theta^3).$$

Setting $g(\theta) = 0$ and neglecting higher-order terms, we obtain the quadratic approximation

$$-D\theta + \frac{1}{2}n_{k-1}p_k(1 - p_k)\theta^2 \approx 0 \implies \theta \approx \frac{2D}{n_{k-1}p_k(1 - p_k)}.$$

This shows that for small D , the solution scales linearly: $\theta = O(D)$. Specifically, $\theta \approx \frac{2(n_k - n_{k-1}p_k)}{n_{k-1}p_k(1 - p_k)}$. This linear scaling implies that the high-probability memory bound grows logarithmically in B/δ , with the coefficient inversely proportional to the drift D .

General lower bound. To obtain a uniform lower bound valid for all $D > 0$ (not just small D), we apply Taylor's theorem with remainder:

$$g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(c)\theta^2 = -D\theta + \frac{1}{2}g''(c)\theta^2 = 0 \implies \theta = \frac{2D}{g''(c)}. \quad \text{for some } c \in (0, \theta)$$

To find an upper bound for $g''(c)$, we write $g''(x) = n_{k-1} \cdot \frac{p_k e^x (1 - p_k)}{(1 - p_k + p_k e^x)^2}$ and define $h(y) = \frac{p_k y (1 - p_k)}{(1 - p_k + p_k y)^2}$ where $y = e^x \geq 1$. By differentiation, $h(y)$ achieves a maximum of $\frac{1}{4}$ over $y \geq 1$ and $0 < p_k < 1$ (attained when the numerator and denominator are balanced). Thus we have that $g''(x) \leq n_{k-1} \cdot \frac{1}{4} = \frac{n_{k-1}}{4} \implies \theta = \frac{2D}{g''(c)} \geq \frac{2D}{n_{k-1}/4} = \frac{8D}{n_{k-1}}$. So a general lower bound is:

$$\theta \geq \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}. \quad (21)$$

Appendix F: Experimental Configuration

This appendix supplies the per-workload parameter values and the per-scheduler hyperparameters used in Section 6. The end-to-end real-GPU validation, including the iteration-time comparison against the Vidur simulator, is reported separately in Appendix G.2.

F.1. Per-workload parameters

Table 4 summarizes the per-workload settings used in Section 6: the total in-flight limit tl for WAIT and Nested WAIT, and the KV cache capacity C of the simulated pool. Throughout, one KV cache block holds 16 tokens, so $C = 29,952$ blocks corresponds to roughly 479,000 tokens of KV cache on an A100 80 GB.

Workload	Algorithm	tl	C (blocks)
Single-type p512d20	WAIT	{20, 25, ..., 40} by rate	29,952
Long-decode p512d1000	WAIT	{20, 25, ..., 40} by rate	29,952
Two-type (0.7p512d20, 0.3p512d50)	Nested WAIT	{20, 25, ..., 40} by rate	29,952
Real (lmsys-chat-1m)	Nested WAIT	{20, 25, ..., 40} by rate	29,952

Table 4 Per-workload settings for WAIT and Nested WAIT in Section 6. In the experiments, we do not set the thresholds directly. Instead, we specify a total in-flight limit tl and derive the per-type thresholds n_j of WAIT and the per-segment thresholds n_k of Nested WAIT so as to satisfy Equations (10) and (11), respectively. Thus tl determines the overall scale of the threshold vector, while the equations determine its relative ratios across types or segments. On the lmsys workload, tl is selected from the grid $\{20, 25, \dots, 40\}$ at each arrival rate.

F.2. Scheduler hyperparameters

vLLM. We use vLLM’s default Vidur configuration, which admits new arrivals first-come-first-served and reserves a KV cache watermark fraction of 0.01 (i.e. 1% of the pool) as a safety margin against concurrent growth of in-flight decodes.

Sarathi. We use Sarathi’s default Vidur configuration, which combines chunked prefill with an admission cap of 512 requests per iteration.

WAIT (single-type workloads). For each arrival rate, tl is selected from the grid $\{20, 25, \dots, 40\}$. We use WAIT for both p512d20 and the long-decode workload p512d1000; in each case, the per-type threshold n_1 is then computed so as to satisfy Equation (10).

Nested WAIT (two-type synthetic workload). Under Nested WAIT, m denotes the number of decode-stage segments. For each arrival rate, tl is selected from the grid $\{20, 25, \dots, 40\}$. For the two-type synthetic workload (0.7p512d20, 0.3p512d50), meaning that each arrival is p512d20 with probability 0.7 and p512d50 with probability 0.3, we use $m = 2$ with decode-stage segments 1–20 and 21–50, and compute the per-segment thresholds (n_1, n_2) so as to satisfy Equation (11). For example, with $tl = 30$, a consistent choice is $(n_1, n_2) \approx (1, 1/3)$, which yields segment in-flight limits of about 20 and 10 after integerization.

Nested WAIT (real lmsys workload). For the real lmsys workload, Nested WAIT again uses a finite partition of the decode stages, and Section 6.2 reports the corresponding sensitivity to the segment count m . For each arrival rate, tl is selected from the grid $\{20, 25, \dots, 40\}$, and the per-segment thresholds n_k are then computed so as to satisfy Equation (11).

Appendix G: Additional Experiments

This appendix collects experiments that address specific reviewer comments without crowding the main text. Section G.1 compares WAIT with a threshold-only variant that skips the accumulation wait, the alternative algorithm suggested by Reviewer 2. Section G.2 reports the real-GPU validation in three parts: a simulator-vs-GPU iteration-time comparison, a single-type GPU sweep that mirrors Section 6.1, and a real-data GPU sweep that mirrors Section 6.2. Section G.3 reports multi-seed reproducibility. Section G.4 extends the evaluation to a prefill-decode-disaggregated deployment.

G.1. Threshold without waiting

Reviewer 2 asks: “what happens if the algorithm does not wait – i.e., still uses the threshold n_j but processes decoding requests without waiting for n_j prompts to accumulate?” We refer to this variant as *WAIT-no-wait*: the per-segment thresholds n_k are still enforced (so the KV cache footprint remains bounded as in the main WAIT algorithm), but a segment is scheduled in each iteration regardless of whether its entry-stage inventory has reached n_k . The accumulation requirement is the only thing removed.

Table 5 reports mean end-to-end latency for six Vidur workload scenarios that span single-type, multi-type, and memory-limited regimes. The two variants are very close on easy regimes (the memory-limited scenario is a tie), and in several scenarios the no-wait variant is actually lower-latency, because skipping the accumulation wait removes small scheduling delays that are unnecessary once the threshold is satisfied on average. The real-GPU experiments on SGLang (Section G.2, Table 6) corroborate this picture but also identify a regime in which the accumulation wait pays off: on the lmsys-chat-1m sweep, the per-rate tuned WAIT configuration attains its largest improvement of +9.2% at $\lambda = 0.4$, where the accumulation wait is the active mechanism. The choice between the two variants is therefore regime-dependent. The threshold itself is what provides the asymptotic stability guarantees of Section 4; the accumulation wait acts as a latency refinement that helps in specific operating regimes.

G.2. Real-GPU validation

We complement the simulator-based experiments of Section 6 with end-to-end runs on a physical GPU. The setup is a single NVIDIA A100 80 GB serving Llama-2-7B through the SGLang 0.5.7 production serving engine; WAIT is implemented as a native SGLang scheduler module, so the comparison is independent of Vidur. We report three pieces of GPU evidence: a check that the simulator’s iteration-time predictions agree with direct GPU measurements; an end-to-end comparison

Scenario (arrival rate)	WAIT (s)	WAIT-no-wait (s)	Preferred
Single-type short decode ($\lambda = 18$)	0.95	0.70	no-wait
Single-type long decode ($\lambda = 4$)	154.9	17.5	no-wait
Memory-limited ($\lambda = 3$, margin 0.6)	79.8	79.8	tied
Balanced multi-type ($\lambda = 20$)	1.37	0.71	no-wait
Heterogeneous multi-type ($\lambda = 20$)	2.12	0.63	no-wait
Same prefill, different decode ($\lambda = 20$)	1.95	1.84	no-wait

Table 5 Threshold-only (WAIT-no-wait) versus full WAIT, on six representative Vidur workload scenarios. Removing the accumulation wait never hurts by a meaningful margin in these Vidur scenarios; on the real-GPU sweep (Table 6), however, the per-rate tuned WAIT configuration attains its largest single-rate improvement at $\lambda = 0.4$. The threshold itself is what provides the stability guarantees of Section 4.

on a single-type workload that mirrors the synthetic experiment of Section 6.1; and an end-to-end comparison on a real-data workload that mirrors the lmsys-chat-1m experiment of Section 6.2.

Simulator vs GPU. To verify that the simulator’s iteration-time model matches real GPU behavior in the regime relevant to Section 6, we compare Vidur’s predictions against direct measurements on the same A100 80 GB GPU running Llama-2-7B served by vLLM. We evaluate 25 batch-size configurations $B \in \{1, 2, 4, \dots, 128\}$ at fixed prefill length 256 and decode length 20, which together span the batch-size range used in our experiments. As Figure 17 shows, Vidur’s predictions track the GPU measurements to within mean absolute percentage error 1.51% ($R^2 = 0.9957$). The fitted iteration-time intercept and slope are $d_0 \approx 276$ ms and $d_1 \approx 0.0115$ ms per KV cache token, in line with the linear time model of Equation (1). This justifies using the simulator as the basis for the throughput and stability comparisons in Section 6.

Single-type workload on GPU. This sweep mirrors the synthetic experiment of Section 6.1: random prompts with input length 512 and output length 20, with Poisson arrivals at $\lambda \in \{1, 2, \dots, 15\}$ requests/s and 500 prompts per rate. The SGLang baseline uses `chunked_prefill_size=256` and `prefill_max_requests=8` (its default configuration triggers OOM at several mid-to-high rates and so cannot serve as a fair baseline). WAIT uses `tl=21`, applied uniformly across all rates, with an underload bypass that defers to SGLang’s native scheduler when the request queue is empty. Figure 18 shows mean end-to-end latency versus λ . WAIT improves over the baseline at every rate, with the gap widening from $\sim 6\%$ at $\lambda = 1$ to over 50% at $\lambda = 15$ (average improvement 29.7% across the sweep). The stability and latency ordering from Section 6.1 carries over to the GPU.

Real-data workload on GPU. This sweep mirrors the lmsys-chat-1m experiment of Section 6.2: prompts sampled from the same dataset (heavy-tailed prefill lengths concentrated near 30 tokens; decode lengths spanning 1 to 500 tokens with median 95 and 95th percentile 380), with Poisson

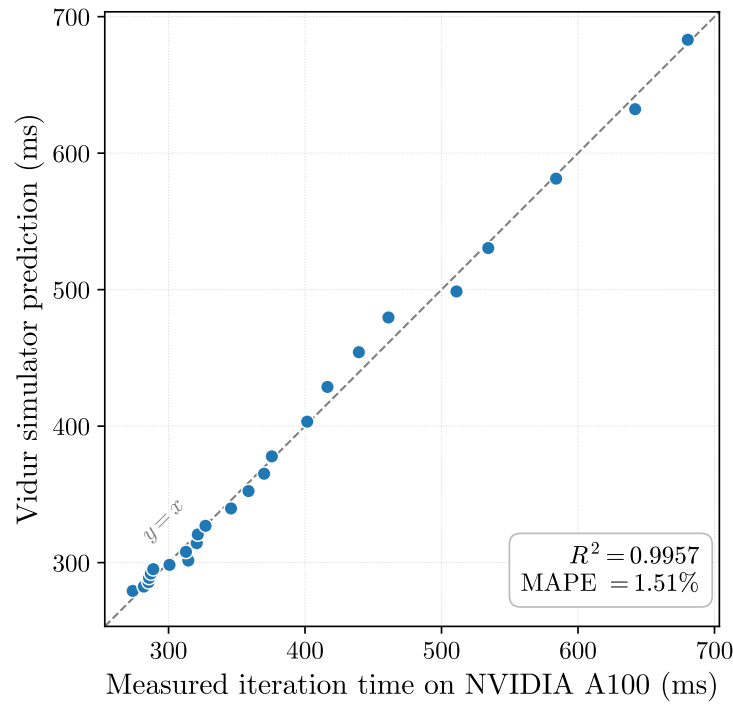


Figure 17 Validation of the Vidur simulator against real NVIDIA A100 80 GB GPU measurements. Each point is one batch-size configuration $B \in \{1, 2, 4, \dots, 128\}$ ($N = 25$), with prefill 256 and decode 20, serving Llama-2-7B via vLLM. The dashed line is $y = x$. The linear iteration-time model of Equation (1) achieves $R^2 = 0.9957$ and mean absolute percentage error 1.51%.

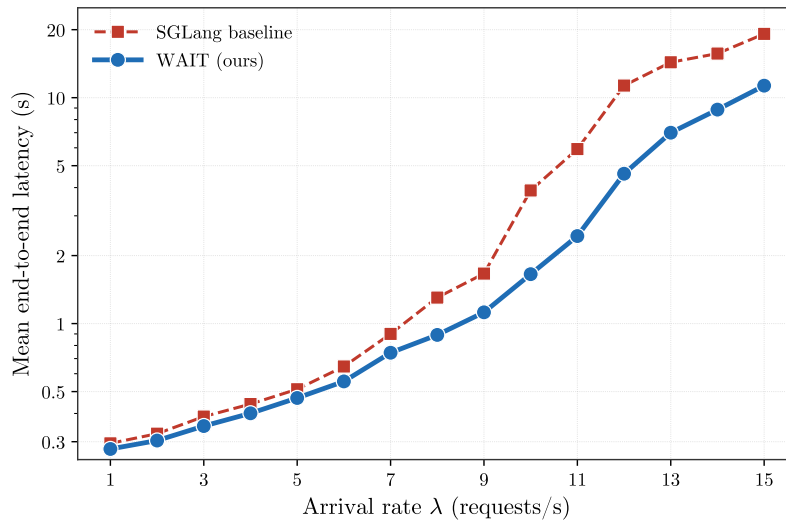


Figure 18 Single-type GPU sweep on SGLang 0.5.7 (NVIDIA A100 80 GB, Llama-2-7B, random prompts with input length 512 and output length 20): mean end-to-end latency versus arrival rate. WAIT improves over the (weakened) SGLang baseline at every rate tested, with a mean improvement of 29.7% across the sweep.

arrivals at $\lambda \in \{0.1, 0.2, \dots, 0.7\}$ requests/s and 200 prompts per rate. The SGLang server is restarted between rates so that each WAIT run is paired against a fresh baseline at the same rate. WAIT uses the total in-flight limit tl tuned per rate; the per-rate value of tl is given in Table 6. Table 6 reports the per-rate paired-baseline improvement: WAIT improves over the SGLang baseline at every swept rate, with peak improvement of +9.2% at $\lambda = 0.4$ and an average of +4.7% across the seven rates. The stability and latency ordering from Section 6.2 (Figure 12) carries over to the GPU.

λ (req/s)	tl	Improvement (%)
0.1	52	+3.4
0.2	52	+2.6
0.3	59	+3.4
0.4	70	+9.2
0.5	72	+2.8
0.6	72	+6.5
0.7	104	+5.0

Table 6 Real-data GPU sweep on SGLang 0.5.7 with the lmsys-chat-1m prompt distribution, NVIDIA A100 80 GB, Llama-2-7B, 200 prompts per rate, fresh paired baseline at each rate. Improvement is $1 - (\text{WAIT mean E2E})/(\text{baseline mean E2E})$. All configurations use Nested WAIT; the total in-flight limit tl is tuned per rate. Peak improvement of +9.2% is attained at $\lambda = 0.4$.

G.3. Multi-seed reproducibility

The Figures in Section 6 report a single seed per configuration; the stability boundaries are insensitive to seed selection because they are defined as the divergence point of a long-horizon mean, but finite- N variance could in principle move individual data points. Table 7 reports the result of re-running the near-boundary arrival rates $\lambda \in \{22, 23\}$ on the single-type and multi-type workloads over five independent seeds.

WAIT's sample standard deviation is consistently an order of magnitude smaller than Sarathi's and vLLM's, because WAIT is deep in its stable region at these rates while the baselines are past theirs; the smaller variance reflects bounded-queue behavior. The range of five seeds preserves the stability ordering at every (workload, λ) combination.

G.4. Prefill-decode-disaggregated deployment

Production LLM serving increasingly uses a *prefill-decode disaggregation* (PD) architecture: prefill is computed by one service and decode by another, with the KV cache shipped between them. The decode-side scheduler sees a stream of prompts whose prefill has already completed and needs to allocate decode-stage compute. This setting is a clean fit for our algorithm: every admitted prompt

Workload	λ	vLLM (s)	Sarathi (s)	WAIT (s)
Single-type	22	15.11 ± 2.73	1.85 ± 0.33	1.00 ± 0.04
Single-type	23	24.33 ± 3.48	8.65 ± 2.11	1.25 ± 0.11
Multi-type W3	22	25.54 ± 2.40	5.67 ± 1.66	2.72 ± 0.58
Multi-type W3	23	35.07 ± 2.40	14.67 ± 2.09	10.24 ± 2.06

Table 7 Multi-seed reproducibility check at the near-boundary arrival rates. Each entry is mean $\pm$ sample standard deviation over 5 independent seeds, each simulated for 10,000 requests. WAIT's standard deviation is substantially smaller because WAIT is deep in its stable region at these rates.

proceeds immediately to decode, so WAIT's threshold directly bounds the number of prompts that the decode-side service keeps in memory.

We implement this as a p630d20 workload (post-prefill context length 630 tokens, decode length 20) on the decode side of a PD setup, with Poisson arrivals sweeping $\lambda \in \{100, 150, \dots, 500\}$ requests/s. The baseline is vLLM's own PD scheduler (vLLM (PD)). Figure 19 shows mean end-to-end decode latency: WAIT improves over vLLM at every rate, with the relative gap ranging from 45% at $\lambda = 100$ to 55% at $\lambda = 500$. The algorithm's advantage therefore transfers directly to the PD-disaggregated architecture without modification.

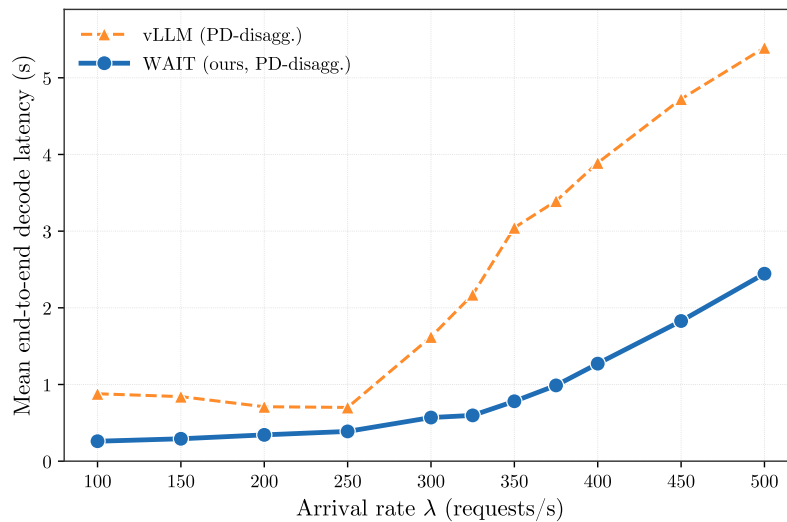


Figure 19 Prefill-decode-disaggregated deployment (p630d20, decode-only): mean end-to-end decode latency versus arrival rate. WAIT attains between 45% and 55% latency reduction over the vLLM PD baseline across the swept range.